

# Committed Vector Oblivious Linear Evaluation and Its Applications

Yunqing Sun\*    Hanlin Liu\*    Kang Yang†    Yu Yu‡    Xiao Wang\*  
Chenkai Weng§

## Abstract

We introduce the notion of committed vector oblivious linear evaluation (C-VOLE), which allows a party holding a pre-committed vector to generate VOLE correlations with multiple parties on the committed value. It is a unifying tool that can be found useful in zero-knowledge proofs (ZKPs) of committed values, actively secure multi-party computation, private set intersection (PSI), etc.

To achieve the best efficiency, we design a tailored commitment scheme and matching C-VOLE protocols, both based on the learning parity with noise assumption. In particular, exploiting the structures of the carefully designed LPN-based commitment minimizes the cost of ensuring consistency between the committed vector and VOLE correlation. As a result, we achieve an  $28\times$  improvement over the protocol proposed in prior work (Usenix 2021) that uses ZKP to prove the correct opening of the commitment. We also apply C-VOLE to design a PSI protocol that allows one server to run PSI repeatedly with multiple clients while ensuring that the same set is used across all executions. Compared with the state-of-the-art PSI (CCS 2024) with similar security requirements, our protocol reduces the communication overhead by a factor of  $35\times$ .

## 1 Introduction

Vector oblivious linear evaluation (VOLE) is a type of protocol that sets up linear correlations between two parties. It can be viewed as an arithmetic extension of the correlated oblivious transfer [NNOB12]. By running a VOLE protocol between  $P_A$ , who has an input vector  $\mathbf{x} \in \mathbb{F}^m$ , and  $P_B$ ,  $P_A$  would obtain a uniform vector  $M[\mathbf{x}] \in \mathbb{F}^m$  as output, and  $P_B$  would obtain  $(\Delta, K[\mathbf{x}]) \in \mathbb{F} \times \mathbb{F}^m$  such that they satisfy the correlation  $M[\mathbf{x}] = K[\mathbf{x}] + \mathbf{x} \cdot \Delta$ . VOLE correlation can be viewed as an information-theoretic message authentication code (IT-MAC) on  $P_A$ 's input, hence serving as a commitment or materials to construct more useful correlations for MPC like Beaver triples. Therefore, extensive practical applications in zero-knowledge proofs (ZKPs) [BMRS21, DIO21, WYKW21], multi-party computation [BDOZ11, DILO22a, RS22], private set intersection (PSI) [RR22a, RS21], etc, have been found recently.

In many VOLE-based protocols, it is often desirable to allow  $P_A$  to 1) first non-interactively commit to its input, which can be distributed to all parties easily, and later 2) run VOLE with other parties while ensuring that  $P_A$  uses the committed input to the VOLE protocol. Such a capability enforces  $P_A$  to use the same input across multiple executions of VOLE, potentially with

---

\*Northwestern University, {yunqing.sun, hanlin.liu, wangxiao}@northwestern.edu

†State Key Laboratory of Cryptology, yangk@sklc.org

‡Shanghai Jiao Tong University and Shanghai Qi Zhi Institute, yuyu@yuyu.hk

§Arizona State University, Chenkai.Weng@asu.edu

different parties. Moreover, the commitment could be pre-distributed to parties offline using cheaper mechanisms like content distribution networks. In more detail:

1. VOLE-based ZKP [WYKW21, BMRS21, DIO21] is a recent paradigm for constructing lightweight ZKP protocols. However, they are designated verifier ZK and do not provide a cheap mechanism if the statement needs to use a consistent witness across different ZKPs. A recent work [WYX<sup>+</sup>21] explored proving the decommitment phase of a symmetric-key-based commitment; however, its performance is fairly limited due to the large size of SHA-256 and AES. Other works [AGR<sup>+</sup>16, GKR<sup>+</sup>21, AAB<sup>+</sup>20, GKS23] have explored designing ZK-friendly hash functions, but the cost is high as the underlying commitment schemes always require non-black-box use.
2. Among existing dishonest-majority MPC protocols, the most practical way to achieve malicious security is to use IT-MACs to authenticate secret shares [BDOZ11, DPSZ12, WRK17, RS22]. These protocols often require running pair-wise VOLE across all parties so that even if all but one parties are corrupted, the underlying secret sharing is still authenticated. However, a critical issue is ensuring that every party uses the same input vector for all VOLE executions with every other party. To achieve that goal, existing MPC protocols require a joint consistency check among all the parties, which often involves significant computation and communication.
3. Crowd PSI refers to a specific type of PSI protocol involving a server and many clients. The server has a large set and repeatedly runs a PSI protocol with many individual clients, using the same set across different PSI executions. It is useful when in services where many clients need to privately look up some private information on the server side, like password checkup or private contact discovery. Prior work [SKR<sup>+</sup>24] in this setting with malicious security requires a public-key-based pseudorandom function (PRF) and its oblivious evaluation for every element. To adapt existing VOLE-based PSI protocols [RS21, RR22a], the most performant PSI protocols right now, to work in this setting, one would need to ensure that the same input vector from the server is used for all VOLE executions, which in turn ensures consistency in the server's set across all executions.

To summarize, the above examples either require  $P_A$ 's VOLE input vector  $\mathbf{x}$  to be publicly committed beforehand or require  $P_A$ 's VOLE input vector  $\mathbf{x}$  to remain the same among multiple VOLE applications with different parties, which can also be ensured if all parties have the same commitment of  $\mathbf{x}$ . This paper provides a unified approach to solve all these issues efficiently in a modular way. To this end, we propose *committed-VOLE (C-VOLE)*, which allows the VOLE sender's input to be pre-committed non-interactively.

## 1.1 Our Contribution

In this work, we propose concretely efficient C-VOLE protocols based on the assumption of learning parity with noise (LPN). Then, we show how C-VOLE can be used in multiple important protocols and how our new protocols can enhance the efficiency of these applications.

**LPN-based C-VOLE protocol.** We propose a C-VOLE protocol based on the LPN assumption. Prior work [WYX<sup>+</sup>21] proposed a protocol using a commitment scheme based on random oracle and AES and proved the correct decommitment, which involves computing SHA256 and AES within ZK. We design an LPN-based commitment scheme specifically for C-VOLE to achieve the best possible efficiency. Then, we exploit the fact that this commitment scheme and the state-of-the-art VOLE protocols share a similar structure, i.e., the LPN-like structure, to design a zero-knowledge

proof of decommitment with very small communication. As a result, our C-VOLE protocol achieves significantly better online communication efficiency than the corresponding VOLE protocol, while requiring  $2\times$  the online running time. We show how our protocol can be based on either dual-LPN-based VOLE or primal-LPN-based VOLE.

**C-VOLE applications and their performance.** We provided some concrete case studies of using C-VOLE in prior applications and how it can improve the efficiency of previous works. Additionally, we implement our C-VOLE protocol and benchmark its practical performance.

For our C-VOLE protocol, the offline commitment is nearly the same size as the data being committed. The online communication is almost constant, with a running time of about 900 ns and 420 ns per element in a LAN setting using 4 and 16 threads, respectively, for a vector of length  $2^{26}$ . Note that running VOLE with a chosen input requires essentially the same amount of communication as the input size to “fix” a random correlation to the chosen input. Therefore, ours communication overhead is near optimal while most communication is offline. We also investigate the protocol change and the concrete efficiency when applying our C-VOLE protocol in various applications.

1. C-VOLE directly enables ZK proof on publicly committed values. Compared to Mystique, we observe a  $28\times$  improvement in running time and  $7\times$  improvement in communication cost.
2. Regarding the multi-party VOLE in generic MPC protocols, C-VOLE avoids the final consistency check, which contains heavy peer-to-peer and broadcast communications. In the context of multi-party VOLE in dishonest-majority MPC and multi-verifier ZKPs [RS22, HSS20, WRK17, EPSW24], C-VOLE similarly avoids the costly final consistency check, significantly reducing the round complexity.
3. By adapting C-VOLE to the VOLE-PSI protocol, we can convert state-of-the-art, actively secure one-shot PSI protocol to the crowd setting without assuming the server’s honest behavior. The C-VOLE PSI protocol requires around 750 ns per server element for computing one-time commitment and around 900 ns per server element for the online interaction with each client, given the server set size of  $2^{24}$ . It reduces online communication overhead by  $35\times$  compared to the state-of-the-art PSI protocol in the crowd setting [SKR<sup>+</sup>24].

**Other related notions.** Note that a different notion of committing VOLE is recently defined in [CDKs24] to construct MPC with an input-revealing identifiable abort guarantee. Their focus is a VOLE variant where parties inputs and outputs are committed so that they can later be opened. Therefore, their notion does not provide input consistency between different executions of VOLE. Thus, their notion is more similar to committing OT notation proposed by Jawurek et al. [JKO13].

## 2 Technical Overview

In this section, we provide some intuition of our constructions.

**Prior VOLE constructions** Recall that a *random* VOLE protocol outputs  $(\mathbf{u}, \mathbf{M}[\mathbf{u}]) \in \mathbb{F}^m \times \mathbb{F}^m$  to  $\mathcal{P}_A$  and  $(\Delta, \mathbf{K}[\mathbf{u}]) \in \mathbb{F} \times \mathbb{F}^m$  to  $\mathcal{P}_B$ , such that all values are uniform under the constraint that

$$\mathbf{M}[\mathbf{u}] = \mathbf{K}[\mathbf{u}] + \mathbf{u} \cdot \Delta.$$

When two parties hold VOLE correlations for vector  $\mathbf{u}$  like the above, we also write  $\llbracket \mathbf{u} \rrbracket$  for simplicity. All of the state-of-the-art VOLE protocols [WYKW21, BCG<sup>+</sup>22, RRT23] compute such

random correlation because this kind of correlations can be easily compressed to achieve sublinear communication and they can be easily derandomized to chosen input. More specifically, if  $P_A$  wants to correct  $\mathbf{u}$  to a designated vector  $\mathbf{x}$ , it simply sends the difference  $\mathbf{d} = \mathbf{u} - \mathbf{x}$  to  $P_B$  and defines  $M[\mathbf{x}] = M[\mathbf{u}]$ ;  $P_B$  can define  $K[\mathbf{x}] = K[\mathbf{u}] + \mathbf{d} \cdot \Delta$ . Now these new values form a correlation

$$M[\mathbf{x}] = M[\mathbf{u}] = K[\mathbf{u}] + \mathbf{u} \cdot \Delta = K[\mathbf{u}] + (\mathbf{u} - \mathbf{x}) \cdot \Delta + \mathbf{x} \cdot \Delta = K[\mathbf{x}] + \mathbf{x} \cdot \Delta.$$

To efficiently construct random VOLE, state-of-the-art protocols follow the pseudorandom correlation generator (PCG) framework [BCGI18]. For example, the dual version of the learning parity with noise (LPN) assumption states that

$$(\mathbf{H}, \mathbf{u}) \text{ is computationally indistinguishable from } (\mathbf{H}, \mathbf{e}_u \mathbf{H}),$$

where  $\mathbf{H} \in \mathbb{F}^{cm \times m}$  is a public random but structured matrix and  $\mathbf{e}_u \in \mathbb{F}^{cm}$  is a sparse vector. To allow more efficient constructions, state-of-the-art protocols also assume that  $\mathbf{e}_u$  is regular, meaning that it partition  $\mathbf{e}_u$  into  $w$  number of shorter vectors  $(\mathbf{e}_u^1, \dots, \mathbf{e}_u^w)$  each of length  $2^L$  such that each shorter vector is non-zero only at one random location. With this assumption, one can construct an efficient VOLE by first constructing “simple” VOLE on each  $\mathbf{e}_u^i$ ’s to obtain  $\llbracket \mathbf{e}_u^i \rrbracket$ , following [BCG<sup>+</sup>19, BCGI18, BCG<sup>+</sup>17] with cost  $O(L\kappa)$ , where  $\kappa$  is the computational security parameter. Then, parties can assemble  $\llbracket \mathbf{e}_u \rrbracket = \llbracket \mathbf{e}_u^1 \rrbracket \parallel \dots \parallel \llbracket \mathbf{e}_u^w \rrbracket$  locally and further compute  $\llbracket \mathbf{e}_u \mathbf{H} \rrbracket = \llbracket \mathbf{e}_u \rrbracket \cdot \mathbf{H}$  due to linear homomorphism. As a result the total communication cost is sublinear. However, when  $P_A$  interacts with different parties, there is no guarantee that they will use the same randomness and hence no guarantee about the consistency of  $P_A$ ’s input.

Our main contribution is to design a customized commitment scheme and proof of consistency to force  $P_A$  using the same value with every party who holds the commitment.

**Blueprint of our protocol.** From a high-level view, our protocol functions as follows. First  $P_A$ , with an input vector  $\mathbf{x} \in \mathbb{F}^m$ , can locally compute a commitment of size roughly  $m \cdot \log |\mathbb{F}|$  bits. This commitment is distributed to all relevant parties by various means, such as a broadcasting protocol or public bulletin board, and its genuineness can be verified with a signature, assuming PKI exists. Because the same commitment is distributed to all parties, it can enjoy network caching at a reduced cost. Then  $P_A$  can interact with each individual party acting as  $P_B$  in VOLE and execute the online phase of the C-VOLE protocol such that they obtain VOLE correlation about  $\mathbf{x}$  while the protocol ensures that  $P_A$  must use  $\mathbf{x}$  as its input (otherwise the protocol would abort). The commitment is used for  $P_A$  to correct VOLE of random vector to its input  $\mathbf{x}$ . Different parties acting as  $P_B$  all know that  $P_A$  uses the same input under the pre-distributed commitment.

At the core of the protocol is the online phase, where a consistency check is needed to show that the committed  $\mathbf{x}$  is consistent with the  $\mathbf{x}$  in VOLE. Assuming a commitment scheme  $\text{Com}$  and a pseudorandom generator (PRG)  $\text{PRG}$ , one can convert the problem to ensuring the consistency of a seed as follows. The commitment to  $\mathbf{x}$  is now  $\text{com}_{\mathbf{x}} = (\text{PRG}(\mathbf{r}) - \mathbf{x}, \text{Com}(\mathbf{r}))$ . During the online phase, two parties obtain the VOLE correlation for  $\mathbf{r}$  and  $\mathbf{x}$ ; then, they can prove using VOLE-ZK that  $\mathbf{r}$  satisfies two properties. First, when fed to the PRG, it yields a vector consistent with the first value in the commitment  $\text{com}_{\mathbf{x}}$ , i.e.,

$$\text{PRG}(\llbracket \mathbf{r} \rrbracket) - \llbracket \mathbf{x} \rrbracket = \text{com}_{\mathbf{x}}[0].$$

Second,  $\llbracket \mathbf{r} \rrbracket$  is consistent with  $\text{Com}(\mathbf{r})$ . Parties execute a proof of decommitment to obtain  $\llbracket \text{decom} \rrbracket = \text{Open}(\text{com}_{\mathbf{x}}[1])$ . Then,  $P_A$  proves that

$$\llbracket \text{decom} \rrbracket = \llbracket \mathbf{r} \rrbracket.$$

Both are statements relevant to VOLE values; thus, VOLE-based ZK only requires one extra message from  $P_A$  but the communication complexity is linear to the circuit size of PRG and  $\text{Open}(\cdot)$ . The rest of this section discusses how to construct PRG and Com so that proving the above two statements using VOLE-based ZK involves almost no communication.

**Commitment based on LPN assumption.** Since state-of-the-art VOLE uses LPN assumptions to generate pseudorandom bits, our goal is to design the commitment for it. Rigorously, the underlying PRG can be defined as  $\text{PRG} \in \{0, 1\}^{wL} \times \mathbb{F}^w \rightarrow \{0, 1\}^m$ :

$$\text{PRG}(k_1, \dots, k_w; a_1, \dots, a_w) = \text{Unary}(k_1, a_1) \parallel \dots \parallel \text{Unary}(k_w, a_w) \cdot \mathbf{H},$$

where  $\mathbf{H} \in \mathbb{F}^{w2^L \times m}$  is a public matrix and  $\text{Unary}(k_i, a_i)$  is defined as an all zero vector with  $a_i$  at position  $k_i$ . The seed of this PRG consists of  $wL$  bits and  $w$  elements in  $\mathbb{F}$ , while the output is a vector of  $m$  elements. In practice, to maintain 128-bit computational security, we set  $w = 224$ ,  $L = 23$ , and  $m = 2^{28}$ , which means the output is much larger than the input, hence a good PRG.

The next step is to design a commitment scheme for  $\mathbf{k} = (k_1, \dots, k_w)$  and  $\mathbf{a} = (a_1, \dots, a_w)$ . We propose to use an LPN-based commitment [JKPT12] due to its linearity. For example, to commit the vector  $\mathbf{a}$ , one can compute

$$\mathbf{a} \cdot \mathbf{H}_1 + \hat{\mathbf{r}} \cdot \mathbf{H}_2 + \hat{\mathbf{e}},$$

where  $\mathbf{H}_1$  and  $\mathbf{H}_2$  are two public matrices,  $\hat{\mathbf{r}}$  is a uniform vector, and  $\hat{\mathbf{e}}$  is a sparse vector, all with appropriate length. It can be shown that when picking the dimension of the two matrices and the sparsity of  $\hat{\mathbf{e}}$  appropriately,  $\hat{\mathbf{r}} \cdot \mathbf{H}_2 + \hat{\mathbf{e}}$  is pseudorandom and thus provides hiding, while it is still difficult to find two pairs of  $(\mathbf{a}, \hat{\mathbf{r}}, \hat{\mathbf{e}})$  and  $(\mathbf{a}', \hat{\mathbf{r}}', \hat{\mathbf{e}}')$  that lead to the same value, hence binding.

Proving the consistency is fairly cheap:  $P_A$  and  $P_B$  can first run the VOLE protocol with  $P_A$  providing  $\mathbf{k}$  and  $\mathbf{a}$ . As part of the VOLE protocol, two parties would obtain VOLE on values  $\mathbf{e}_u^i = \text{Unary}(k_i, a_i)$ , namely  $\llbracket \mathbf{e}_u^i \rrbracket$  for each  $i \in [w]$ . One can easily check the correct value of  $a_i$  by checking  $a_i = \sum_{j \in [2^L]} \mathbf{e}_u^i[j]$ , which is free. Checking the sparsity of  $\mathbf{e}_u^i$  is no longer free but can still be done with linear cost by running the binary-to-unary circuit in ZKP from  $k_i \in \{0, 1\}^L$  to a Boolean vector of length  $2^L$ , which takes a total of  $2m$  non-linear gates. The cost is orders of magnitude better than Mystique, which requires more than 50 non-linear gate per output due to a large AES circuit.

**C-VOLE with sublinear online communication.** The above protocol is much faster than prior work, but the online communication cost is still linear to the vector size. Note that we already send linear communication as part of the offline commitment, so we ideally want the online phase to be even smaller. This means the consistency check can only prove a circuit of size sublinear to the output size!

In the above protocol, the most expensive operation is proof of sparsity; it is required for the binding property. Our crucial observation is that the VOLE protocol already ensures that each  $\llbracket \mathbf{e}_u^i \rrbracket$  is sparse; we need to prove the binary-to-unary circuit only to ensure that it is consistent with the committed randomness. To this end, we modify our commitment. Instead of committing to the seed of the LPN-PRG, we commit to some intermediate values in the PRG, i.e., the structured sparse vector, directly. More specifically, our commitment scheme no longer handles any input but only sparse vectors  $\mathbf{e}_u$ :

$$\mathbf{e}_u \cdot \mathbf{H}_1 + \mathbf{e}_r \cdot \mathbf{H}_2,$$

where  $\mathbf{H}_1$  and  $\mathbf{H}_2$  are two public matrices, and  $\mathbf{e}_r$  is a sparse vector. We can show that it is computationally infeasible to obtain two  $(\mathbf{e}_u, \mathbf{e}_r)$  and  $(\mathbf{e}'_u, \mathbf{e}'_r)$  such that the above commitments are the same and that  $\mathbf{e}'_u$  is also regular with specified sparsity. Note that because the output is much

shorter than  $|e_u|$ , finding a collision is possible if the sparsity is not guaranteed by other means; but again, in our application, VOLE protocols already ensure it for free.

Now, our protocol is extremely efficient. Two parties obtain  $\llbracket e_u \rrbracket$  as part of the normal VOLE protocol. They just need to obtain  $\llbracket e_r \rrbracket$  and check if the  $\llbracket e_u \rrbracket \cdot \mathbf{H}_1 + \llbracket e_r \rrbracket \cdot \mathbf{H}_2$  matches the commitment. We show that  $|e_r| \ll |e_u|$ , and thus, this extra communication cost is very small compared to the cost of the VOLE protocol. In addition to the VOLE protocol, the main computational cost is the evaluation of  $\llbracket e_u \rrbracket \cdot \mathbf{H}_1$ , which is only a local computation. We provide the detailed description and analysis of LPN-based commitment in Section 4.1 and Section A.1

**Applications to VOLE-ZK, MPC, and MVZK.** C-VOLE brings immediate performance improvement when used in several recent VOLE-ZK and MPC protocols.

- In VOLE-based ZKP protocols, C-VOLE directly leads to a way to convert public commitments to VOLE-like authentication ready for VOLE-ZK. Compared to the previous work [WYX<sup>+</sup>21], it avoids proving large SHA-256 and AES circuits in ZKP and achieves 28× concrete improvement.
- For generic secure multi-party computation, it can be used to execute multi-party VOLE among all parties [RS22, HSS20, WRK17]. By using C-VOLE, they no longer require heavy peer-to-peer and broadcast communication in the final consistency check. MVZK with dishonest-majority verifiers [EPSW24] also relies on similar consistency checks, which can be avoided by C-VOLE.

We defer the details to Section 6.

**Application to crowd PSI.** In the crowd PSI setting, we have one server holding a large set  $X$  and a set of clients, each holding a small set  $Y_i$ , such that each pair want to obtain the intersection  $X \cap Y_i$ . State-of-the-art PSI protocols based on VOLE [RS21, RR22a] can provide malicious security but cannot ensure consistency of  $X$ . A prior work [SKR<sup>+</sup>24] works in this setting but uses public-key based Oblivious PRF (OPRF), which is highly expensive.

We observe that the only place where VOLE-PSI protocols use VOLE is for the server to feed its input set to the VOLE. The server’s input set is encoded into a vector  $\mathbf{p}$ , such that  $\text{Decode}(\mathbf{p}, x) = H_1(x)$  for  $x \in X$ . The server identifies any client’s set elements in the intersection using an OPRF defined by  $\llbracket \mathbf{p} \rrbracket$ . Specifically, the client encodes its set elements by  $F(x) = H_2(\text{Decode}(\mathbf{K}[\mathbf{p}], x) + \Delta \cdot H_1(x), x)$ , where  $H_1$  and  $H_2$  are two hash functions, and sends the encodings to the server. The server encodes its set elements by  $X' := \{H_2(\text{Decode}(\mathbf{M}[\mathbf{p}], x), x) | x \in X\}$  and checks for matches with the client’s encoding. The two encodings match if the client’s set element is located in the intersection, which satisfies  $\text{Decode}(\mathbf{p}, x) = H_1(x)$ . In this case, VOLE is utilized as a black-box component in the VOLE-PSI protocol. Compared to VOLE, C-VOLE enforces the consistency of  $\mathbf{p}$  across different VOLE executions with other clients, ensuring consistent use of the server’s set. Thus, we can prove that if we replace VOLE with C-VOLE, we can obtain a PSI protocol in the crowd setting.

As a result, we upgrade the state-of-the-art single-client PSI without the set-committing feature into a crowd PSI. The C-VOLE PSI requires around 900 ns per server element for online computation and less than 1.64 B per server element for online communication of set size  $2^{20}$ , which is roughly 2× online time consuming but 13× better online communication overhead compared with VOLE-PSI. Compared with prior work [SKR<sup>+</sup>24] in the crowd setting, C-VOLE PSI reduces initialization time with a 8-fold improvement for set size  $2^{20}$ . It also decreases online communication to 1.73 MB with clients having  $2^{10}$  elements, reduced by a factor of 35×.



### 3 Preliminaries

#### 3.1 Notation

Define statistical and computational security parameters  $\lambda$  and  $\kappa$ . We let  $\text{negl}(\cdot)$  denote a negligible function, and use  $\log$  to denote logarithms in base 2.  $\mathbb{F}$  denotes a finite field. We write  $x \leftarrow \mathbb{F}$  to denote sampling  $x$  uniformly from  $\mathbb{F}$ . We write  $[n] = \{1, \dots, n\}$ . We use bold upper-case letters like  $\mathbf{A}$  for matrices. For  $\mathbf{A} \in \mathbb{F}^{m \times n}$ ,  $m$  denotes the number of rows, and  $n$  denotes the number of columns. We use bold lower-case letters like  $\mathbf{a}$  for row vectors, and let  $\mathbf{a}[i]/a_i$  denote the  $i$ th component of  $\mathbf{a}$  (with  $\mathbf{a}[1]/a_1$  the first entry).  $\mathbf{a}^T$  denotes the transposed vector of  $\mathbf{a}$ . We use  $\parallel$  to represent the concatenation. Define a regular- $w$  vector  $\mathbf{e}$  as  $\mathbf{e} = (\mathbf{e}^1 \parallel \dots \parallel \mathbf{e}^w) \in \mathbb{F}^{w \cdot 2^L}$ , where  $\mathbf{e}^i \in \mathbb{F}^{2^L}$  is a unary vector. We also denote  $\mathbf{e}$  as a multi-point vector and  $\mathbf{e}^i$  as a single-point vector. We use  $\mathbf{K}[\mathbf{a}]$  to represent  $\mathbf{K}[a_1] \parallel \dots \parallel \mathbf{K}[a_n]$ , and let  $\mathbf{M}[\mathbf{a}] = \mathbf{M}[a_1] \parallel \dots \parallel \mathbf{M}[a_n]$ .

#### 3.2 Learning Parity with Noise

The Learning Parity with Noise (LPN) problem is defined as recovering a secret vector given a noisy system of linear equations. It has found many applications in cryptography, as it has strong security guarantees. The hardness definition of primal-LPN and dual-LPN [BFKL94] are presented in Definition 1 and Definition 2 respectively as follows:

**Definition 1.** (Primal LPN) Let  $\mathcal{C}$  be a probabilistic code generation algorithm such that  $\mathcal{C}(N, k, \mathbb{F})$  outputs a matrix  $\mathbf{A} \in \mathbb{F}^{k \times N}$ . We say that the decisional primal-LPN( $N, k, t$ ) problem is  $(T, \varepsilon)$ -computationally hard for every probabilistic distinguisher  $\mathcal{D}$  running in time  $T$  if

$$|\Pr[\mathcal{D}(\mathbf{A}, \mathbf{A} \cdot \mathbf{x} + \mathbf{e}) = 1] - \Pr[\mathcal{D}(\mathbf{A}, \mathbf{u}) = 1]| \leq \varepsilon,$$

where  $\mathbf{x} \leftarrow \mathbb{F}^N$ ,  $\mathbf{u} \leftarrow \mathbb{F}^k$ , and  $\mathbf{e} \leftarrow \mathbb{F}^k$  with weight  $t$ .

**Definition 2.** (Dual LPN) Let  $\mathcal{C}^\perp$  be a probabilistic code generation algorithm such that  $\mathcal{C}^\perp(N, k, \mathbb{F})$  outputs a matrix  $\mathbf{H} \in \mathbb{F}^{k \times N}$ . We say that the decisional dual-LPN( $N, k, t$ ) problem is  $(T, \varepsilon)$ -computationally hard for every probabilistic distinguisher  $\mathcal{D}$  running in time  $T$  if

$$|\Pr[\mathcal{D}(\mathbf{H}, \mathbf{H} \cdot \mathbf{e}) = 1] - \Pr[\mathcal{D}(\mathbf{H}, \mathbf{u}) = 1]| \leq \varepsilon,$$

where  $\mathbf{e} \leftarrow \mathbb{F}^N$  with weight  $t$ , and  $\mathbf{u} \leftarrow \mathbb{F}^k$ .

#### 3.3 Commitment Scheme

A commitment scheme is defined as a sender committing to a chosen value while keeping it hidden from others and opening the value later to let others verify it.

1. **ParamGen:** The sender samples public parameter  $k$  from a parameter space  $K$  and publish  $k$  to other parties.
2. **Commit:** The sender picks message  $\mathbf{x} \leftarrow \mathbb{F}^m$  and computes  $\{\text{com}_{\mathbf{x}}, \text{decom}_{\mathbf{x}}\} \leftarrow \text{Com}(\mathbf{x})$ , where  $\text{decom}_{\mathbf{x}} \in \mathbb{F}^*$ . The sender sends  $\text{com}_{\mathbf{x}}$  to the receiver.
3. **Open:** The sender sends  $x$  and  $\text{decom}_{\mathbf{x}}$  to the receiver. The receiver computes  $0/1 \leftarrow \text{Open}(\text{com}_{\mathbf{x}}, \text{decom}_{\mathbf{x}}, \mathbf{x})$

A commitment scheme with computational hiding and statistical binding is defined as follows.

### Functionality $\mathcal{F}_{C-VOLE}$

**Public Parameters:** A public commitment of  $\mathbf{x}$ ,  $\text{Hcom}_{\mathbf{x}}$ . A hybrid commitment scheme  $\text{HCom}(\cdot)$  defined in Section 4.2.

1. Upon receiving  $(\text{init}, \mathbf{x}, \text{Hdecom}_{\mathbf{x}})$  from  $P_A$ , and  $(\text{init})$  from  $P_B$ , the functionality stores  $(\mathbf{x}, m = |\mathbf{x}|)$  and checks whether  $(\text{Hcom}_{\mathbf{x}}, \text{Hdecom}_{\mathbf{x}})$  opens to  $\mathbf{x}$ . If not, sends **abort** to both parties.
2. Samples  $K[\mathbf{x}] \leftarrow \mathbb{F}^m$ ,  $\Delta \leftarrow \mathbb{F}$ . If  $P_B$  is malicious, receives  $(K[\mathbf{x}], \Delta) \in \mathbb{F}^{m+1}$  from adversary. Computes  $M[\mathbf{x}] := K[\mathbf{x}] + \Delta \cdot \mathbf{x} \in \mathbb{F}^m$ . If  $P_A$  is malicious, receives  $M[\mathbf{x}] \in \mathbb{F}^m$  from the adversary, and recomputes  $K[\mathbf{x}] := M[\mathbf{x}] - \Delta \cdot \mathbf{x} \in \mathbb{F}^m$ . Sends  $M[\mathbf{x}]$  to  $P_A$  and  $(\Delta, K[\mathbf{x}])$  to  $P_B$ .

Figure 1: The committed-VOLE functionality.

1. **Computational hiding:** with overwhelming probability over the choice of  $k$ , for every  $\mathbf{x}, \mathbf{x}' \in \mathbb{F}^m$  and  $\{\text{com}_{\mathbf{x}}, \text{decom}_{\mathbf{x}}\} \leftarrow \text{Com}(\mathbf{x})$ ,  $\{\text{com}'_{\mathbf{x}'}, \text{decom}'_{\mathbf{x}'}\} \leftarrow \text{Com}(\mathbf{x}')$ , the distributions of  $\text{com}_{\mathbf{x}}$  and  $\text{com}'_{\mathbf{x}'}$  are computationally indistinguishable.
2. **Statistical binding:** with overwhelming probability over the choice of  $k$ , the following equation holds with probability  $\text{negl}(\lambda)$ ,

$$\text{Open}(\text{com}_{\mathbf{x}}, \text{decom}_{\mathbf{x}}, \mathbf{x}) \wedge \text{Open}(\text{com}_{\mathbf{x}}, \text{decom}'_{\mathbf{x}'}, \mathbf{x}') = 1$$

### 3.4 Ideal Functionalities

**Committed-vector Oblivious Linear Evaluation.** Committed-vector oblivious linear evaluation (C-VOLE) is a variant of VOLE that allows two parties to generate VOLE relations for a pre-committed vector  $\mathbf{x} \in \mathbb{F}^m$ . The commitment of  $\mathbf{x}$ , denoted as  $\text{Hcom}_{\mathbf{x}}$ , is public. The holder of  $\mathbf{x}$  serves as  $P_A$ , while any party knows the public commitment  $\text{Hcom}_{\mathbf{x}}$  can act as  $P_B$  and execute the functionality depicted in Figure 1 with  $P_A$ . The functionality  $\mathcal{F}_{C-VOLE}$  first checks whether  $P_A$  holds the correct  $(\mathbf{x}, \text{Hdecom}_{\mathbf{x}})$  that satisfies the public commitment  $\text{Hcom}_{\mathbf{x}}$ . It then samples a vector  $K[\mathbf{x}] \leftarrow \mathbb{F}^m$  and a field element  $\Delta \leftarrow \mathbb{F}$  to  $P_B$  and delivers  $M[\mathbf{x}] = K[\mathbf{x}] + \mathbf{x} \cdot \Delta \in \mathbb{F}^m$  to  $P_A$ , as the VOLE functionality typically does.

**Vector Oblivious Linear Evaluation.** Vector Oblivious Linear Evaluation (VOLE) is generating VOLE relation of a designated vector rather than a random vector. The ideal functionality is shown in Figure 2. The functionality samples field element  $\Delta \leftarrow \mathbb{F}$  to  $P_B$  for initialization. Then, for  $P_A$ 's input vector  $\mathbf{x} \in \mathbb{F}^m$ , the functionality samples  $K[\mathbf{x}] \leftarrow \mathbb{F}^m$  to  $P_B$  and delivers  $M[\mathbf{x}] = K[\mathbf{x}] + \mathbf{x} \cdot \Delta \in \mathbb{F}^m$  to  $P_A$ . VOLE is typically instantiated from Random VOLE (RVOLE). It also serves as a building block in the instantiation of  $\mathcal{F}_{C-VOLE}$ .

**Multi-point Vector Oblivious Linear Evaluation.** Multi-point Vector Oblivious Linear Evaluation (MPVOLE) is a variant of VOLE that generates the VOLE relation of a multi-point vector, a regular sparse vector formed by concatenating multiple single-point (unary) vectors. Its ideal functionality is shown in Figure 3 and consists of two phases: the functionality samples field element  $\Delta \leftarrow \mathbb{F}$  to  $P_B$  for initialization; Then, for  $P_A$ 's input multi-point vector  $\mathbf{e}_u \in \mathbb{F}^{t \cdot 2^h}$ , where  $t \cdot 2^h = n$ , the functionality samples  $K[\mathbf{e}_u] \leftarrow \mathbb{F}^n$  to  $P_B$  and delivers  $M[\mathbf{e}_u] = K[\mathbf{e}_u] + \mathbf{e}_u \cdot \Delta \in \mathbb{F}^n$  to  $P_A$ . We provide the construction of MPVOLE in Section A.5, which is a slightly variant of [BCGI18, WYKW21]. It serves as an important building block in instantiating  $\mathcal{F}_{C-VOLE}$ .



**Functionality  $\mathcal{F}_{\text{VOLE}}$**

**Initialize:** Upon receiving (init) from both  $P_A$  and  $P_B$ , sample  $\Delta \leftarrow \mathbb{F}$  and send it to  $P_B$ . The functionality stores  $\Delta$ . If  $P_B$  is malicious, receive  $\Delta \in \mathbb{F}$  from  $P_B$ .

**Extend:** Upon receiving (extend,  $\mathbf{x} \in \mathbb{F}^m, m$ ) from  $P_A$  and (extend,  $m$ ) from  $P_B$ :

1. Samples  $K[\mathbf{x}] \leftarrow \mathbb{F}^m$ . If  $P_B$  is malicious, receive  $K[\mathbf{x}] \in \mathbb{F}^m$  from adversary.
2. Computes  $M[\mathbf{x}] := K[\mathbf{x}] + \Delta \cdot \mathbf{x} \in \mathbb{F}^m$ . If  $P_A$  is malicious, receives  $M[\mathbf{x}] \in \mathbb{F}^m$  from the adversary, and recomputes  $K[\mathbf{x}] := M[\mathbf{x}] - \Delta \cdot \mathbf{x} \in \mathbb{F}^m$ .
3. Sends  $M[\mathbf{x}]$  to  $P_A$  and  $K[\mathbf{x}]$  to  $P_B$ .

Figure 2: The VOLE functionality.

**Functionality  $\mathcal{F}_{\text{MPVOLE}}$**

**Initialize:** Upon receiving (init) from both  $P_A$  and  $P_B$ , sample  $\Delta \leftarrow \mathbb{F}$  to  $P_B$ . The functionality stores  $\Delta$ . If  $P_B$  is malicious, receive  $\Delta \in \mathbb{F}$  from  $P_B$ .

**Extend:** Upon receiving (extend,  $\mathbf{e}_u \in \mathbb{F}^{t \cdot 2^h}, h, t$ ) from  $P_A$  where  $\mathbf{e}_u$  is a regular sparse vector with  $t$  non-zero elements and  $n = t \cdot 2^h$ , and (extend,  $h, t$ ) from  $P_B$ .

1. Samples  $K[\mathbf{e}_u] \leftarrow \mathbb{F}^n$ . If  $P_B$  is malicious, receives  $K[\mathbf{e}_u] \in \mathbb{F}^n$  from adversary.
2. Computes  $M[\mathbf{e}_u] := K[\mathbf{e}_u] + \Delta \cdot \mathbf{e}_u \in \mathbb{F}^n$ . If  $P_A$  is malicious, receives  $M[\mathbf{e}_u] \in \mathbb{F}^n$  from the adversary, and recomputes  $K[\mathbf{e}_u] := M[\mathbf{e}_u] - \Delta \cdot \mathbf{e}_u \in \mathbb{F}^n$ .
3. Sends  $M[\mathbf{e}_u]$  to  $P_A$  and  $K[\mathbf{e}_u]$  to  $P_B$ .

**Selective failure:** In the malicious setting, if  $P_B$  is corrupted, the adversary is allowed to make the following selective failure query for each extend call:

1. Wait for the adversary to input  $t$  sets  $I_1, \dots, I_t \subseteq [0, n)$ . Let  $\{\alpha_i\} \in [0, n)$  be the index of the non-zero entry of  $i$ th single-point block  $\mathbf{e}_u^i$ .
2. For all  $\alpha_i \in I_i$  for  $i \in [t]$ , send **success** to  $P_B$  and continue. Otherwise, send **abort** to both parties and abort.

Figure 3: The multi-point VOLE functionality.

**Crowd Private Set Intersection.** Private set intersection (PSI) allows two distrusted parties to jointly compute the intersection of their private inputs without revealing any additional information about those inputs except for the set sizes. Crowd PSI is an extension of PSI involving a server and multiple clients, where the server executes PSI with each individual client. In Figure 4, we describe the ideal functionality of crowd PSI, with the server (party  $P_1$ ) acting as the receiver of the intersection and the clients (party  $P_2, \dots, P_n$ ) functioning as senders. The server sends its private set to the functionality for initialization. The functionality waiting for any client's private set, stores the server's set, and returns the intersection to the server.

Functionality  $\mathcal{F}_{\text{PSI}}$

There are  $n$  parties:  $P_1, \dots, P_n$

1. Upon receiving  $(\text{init}, X)$  from party  $P_1$ , and  $(\text{intersect}, Y_2)$  from a party  $P_2$ , where  $X, Y_2 \subset U$ , the functionality stores  $X$  and outputs  $(X \cap Y_2, \text{setid})$  to  $P_1$ . The functionality publishes  $\text{setid}$ .
2. Upon receiving  $(\text{init}, \text{setid})$  from party  $P_1$ , and  $(\text{intersect}, Y_j, \text{setid})$  from a party  $P_j$ , where  $Y_j \subset U$ ,  $j \in [3, n]$ , the functionality outputs  $X \cap Y_j$  to  $P_1$ .

Figure 4: The crowd private set intersection functionality.

## 4 Commitment from LPN

We introduce the commitment scheme we use, denoted as a hybrid commitment, in the committed-VOLE construction. As outlined in Section 2, the hybrid commitment scheme relies on the LPN assumption and consists of two parts: an LPN commitment to an intermediate regular sparse vector and a correction to the designated committed vector.

We construct the LPN commitment specifically for a regular sparse vector. In Theorem 1, we prove that with sparse message distribution, the new LPN commitment schemes still satisfy the requirements of binding and hiding. Then, we add a correction vector to the LPN commitment to construct a hybrid commitment scheme for a designated vector, specifically designed for the C-VOLE construction. The proposed hybrid commitment satisfy the binding and hiding property as detailed in Theorem 2.

### 4.1 LPN Commitment

Here, we define a commitment scheme  $\text{Com}(\cdot)$  based on the primal LPN assumption for committing to a regular- $w$  vector  $\mathbf{x}$ . This vector  $\mathbf{x} \in \mathbb{F}^m$ , for parameters  $w, L$  and  $m = w \cdot 2^L$ , consists of  $w$  blocks, each of size  $2^L$ , with each block contains exactly one non-zero element.

- **ParamGen.** The public commitment parameter is represented by  $\mathbf{K} = \{\mathbf{A}_1, \mathbf{A}_2\}$ , where  $\mathbf{A}_1 \leftarrow \mathbf{C}(N, m, \mathbb{F})$  and  $\mathbf{A}_2 \leftarrow \mathbb{F}^{k \times N}$ , with  $N = t \cdot 2^{L'}$ . The algorithm  $\mathbf{C}$  is a probabilistic code generation algorithm that outputs a matrix  $\mathbf{A}_1 \in \mathbb{F}^{m \times N}$ .
- **Commit.** The commitment to a regular- $w$  message  $\mathbf{x} \in \mathbb{F}^m$  is expressed as  $\text{com}_{\mathbf{x}} = \mathbf{x} \cdot \mathbf{A}_1 + \mathbf{r} \cdot \mathbf{A}_2 + \mathbf{e}$ , where  $\mathbf{r} \leftarrow \mathbb{F}^k$  and  $\mathbf{e}$  is a weight- $t$  regular noise vector of length  $N$ . The opening of the commitment is given by  $\text{decom}_{\mathbf{x}} = \mathbf{r}$ .
- **Open.** Given a commitment  $\text{com}_{\mathbf{x}}$ , a regular- $w$  message  $\mathbf{x}$ , and randomness  $\text{decom}_{\mathbf{x}}$ , a verifier accepts if  $\text{com}_{\mathbf{x}} - \mathbf{x} \cdot \mathbf{A}_1 - \text{decom}_{\mathbf{x}} \cdot \mathbf{A}_2$  is weight- $t$  regular.

The corresponding dual LPN commitment scheme  $\text{Com}(\cdot)$  for a regular- $w$  message  $\mathbf{x} \in \mathbb{F}^m$  is described below:

- **ParamGen.** The public commitment parameter is represented by  $\mathbf{K} \stackrel{\text{def}}{=} \{\mathbf{H}_1, \mathbf{H}_2\}$ , where  $\mathbf{H}_1 \leftarrow \mathbf{C}^\perp(n, m, \mathbb{F})$  and  $\mathbf{H}_2 \leftarrow \mathbb{F}^{N \times n}$ , with  $N = t \cdot 2^{L'}$ . The algorithm  $\mathbf{C}^\perp$  is a probabilistic code generation algorithm that outputs a matrix  $\mathbf{H}_1 \in \mathbb{F}^{m \times n}$ .
- **Commit.** The commitment to a regular- $w$  message  $\mathbf{x} \in \mathbb{F}^m$  is expressed as  $\text{com}_{\mathbf{x}} = \mathbf{x} \cdot \mathbf{H}_1 + \mathbf{e} \cdot \mathbf{H}_2$ , where  $\mathbf{e}$  is a weight- $t$  regular noise vector of length  $N$ . The opening of the commitment is given by  $\text{decom}_{\mathbf{x}} = \mathbf{e}$ .

- **Open.** Given a commitment  $\text{com}_{\mathbf{x}}$ , a regular- $w$  message  $\mathbf{x}$ , and randomness  $\text{decom}_{\mathbf{x}}$ , a verifier accepts if  $\text{decom}_{\mathbf{x}}$  is weight- $t$  regular and  $\text{com}_{\mathbf{x}} = \mathbf{x} \cdot \mathbf{H}_1 + \text{decom}_{\mathbf{x}} \cdot \mathbf{H}_2$ .

**Theorem 1.** Assume  $L', k, t \in \mathbb{N}$  such that the decisional primal-LPN( $N, k, t$ ) problem (with secrets of length  $k$  and  $N$  samples) and the decisional dual-LPN( $N, n = N - k, t$ ) problem are computationally hard, where  $N = t \cdot 2^{L'}$ . Then the above two commitment schemes are  $\varepsilon$ -statistically binding and computationally hiding, where  $\lambda = 2^{2wL+2tL'} / |\mathbb{F}|^{t \cdot 2^{L'} - k - 2w - 2t}$ .

*Proof.* The security properties are proved as follows:

### 1. Statistically binding.

- **For the commitment scheme based on the primal LPN problem.** Suppose there exists a commitment  $\mathbf{c}$  corresponding to two messages,  $\mathbf{x}_1 \in \mathbb{F}^m$  and  $\mathbf{x}_2 \in \mathbb{F}^m$ , with randomness pairs  $(\mathbf{r}_1, \mathbf{e}_1)$  and  $(\mathbf{r}_2, \mathbf{e}_2)$ . Define  $\mathbf{x} \stackrel{\text{def}}{=} \mathbf{x}_1 - \mathbf{x}_2 \in \mathbb{F}^m$ ,  $\mathbf{r} \stackrel{\text{def}}{=} \mathbf{r}_1 - \mathbf{r}_2 \in \mathbb{F}^k$  and  $\mathbf{e} \stackrel{\text{def}}{=} \mathbf{e}_1 - \mathbf{e}_2 \in \mathbb{F}^N$ . Given that each  $\mathbf{x}_i$  is a weight- $w$  regular vector of length  $m = w \cdot 2^L$  and each  $\mathbf{e}_i$  is a weight- $t$  regular vector of length  $N = t \cdot 2^{L'}$ , the probability that  $\mathbf{x} \cdot \mathbf{A}_1 + \mathbf{r} \cdot \mathbf{A}_2 + \mathbf{e} = 0$  is at most  $2^{2wL+2tL'} / |\mathbb{F}|^{t \cdot 2^{L'} - 2w - 2t}$ . A union bound over all possible  $\mathbf{r} \in \mathbb{F}^k$  yields the claimed bound  $2^{2wL+2tL'} / |\mathbb{F}|^{t \cdot 2^{L'} - k - 2w - 2t}$ .
- **For the commitment scheme based on the dual LPN problem.** Considering the commitment  $\mathbf{c}$  associated with two messages,  $\mathbf{x}_1 \in \mathbb{F}^m$  and  $\mathbf{x}_2 \in \mathbb{F}^m$ , along with their respective randomness  $\mathbf{e}_1$  and  $\mathbf{e}_2$ . Define  $\mathbf{x} \stackrel{\text{def}}{=} \mathbf{x}_1 - \mathbf{x}_2 \in \mathbb{F}^m$  and  $\mathbf{e} \stackrel{\text{def}}{=} \mathbf{e}_1 - \mathbf{e}_2 \in \mathbb{F}^N$ . Given that each  $\mathbf{x}_i$  is a weight- $w$  regular vector of length  $m = w \cdot 2^L$  and each  $\mathbf{e}_i$  is a weight- $t$  regular vector of length  $N = t \cdot 2^{L'}$ , the probability that  $\mathbf{x} \cdot \mathbf{H}_1 + \mathbf{e} \cdot \mathbf{H}_2 = 0$  is at most  $2^{2wL+2tL'} / |\mathbb{F}|^{t \cdot 2^{L'} - k - 2w - 2t}$ .

2. **Computationally hiding.** In the commitment scheme based on the primal LPN problem, we have  $\mathbf{c} = \mathbf{x} \cdot \mathbf{A}_1 + \mathbf{r} \cdot \mathbf{A}_2 + \mathbf{e}$ . In the commitment scheme based on the dual LPN problem, the commitment is  $\mathbf{c} = \mathbf{x} \cdot \mathbf{H}_1 + \mathbf{e} \cdot \mathbf{H}_2$ . Leveraging the security properties of the primal LPN and dual LPN problems, the terms  $\mathbf{r} \cdot \mathbf{A}_2 + \mathbf{e}$  and  $\mathbf{e} \cdot \mathbf{H}_2$  are pseudorandom, ensuring the pseudorandomness of the respective commitments.

□

## 4.2 Hybrid Commitment

We present the hybrid commitment scheme  $\text{HCom}(\cdot)$ , designed for C-VOLE construction for messages  $\mathbf{x} \in \mathbb{F}^m$ , with the following components:

- **ParamGen.** The public commitment parameter consists of the public commitment parameters of  $\text{Com}$  and a matrix  $\mathbf{H}^\top \leftarrow \mathbf{C}^\perp(N, m, \mathbb{F})$ , where  $N = w \cdot 2^L$ . The algorithm  $\mathbf{C}^\perp$  is a probabilistic code generation algorithm that outputs a matrix  $\mathbf{H} \in \mathbb{F}^{N \times m}$ .
- **Commit.** The hybrid commitment to a message  $\mathbf{x} \in \mathbb{F}^m$  is expressed as  $\text{Hcom}_{\mathbf{x}} = \{\text{com}_{\mathbf{e}}, \mathbf{c} = \mathbf{e} \cdot \mathbf{H} - \mathbf{x}\}$ , where  $\mathbf{e}$  is regular- $w$  vector of length  $N$  and  $(\text{com}_{\mathbf{e}}, \text{decom}_{\mathbf{e}}) \leftarrow \text{Com}(\mathbf{e})$ . The opening of the commitment is given by  $\text{Hdecom}_{\mathbf{x}} = \{\mathbf{e}, \text{decom}_{\mathbf{e}}\}$ .
- **Open.** Given a commitment  $\text{Hcom}_{\mathbf{x}} = \{\text{com}_{\mathbf{e}}, \mathbf{c}\}$ , a message  $\mathbf{x}$ , and randomness  $\text{Hdecom}_{\mathbf{x}} = \{\mathbf{e}, \text{decom}_{\mathbf{e}}\}$ , a verifier accepts if  $\{\mathbf{e}, \text{com}_{\mathbf{e}}, \text{decom}_{\mathbf{e}}\}$  passes verification of  $\text{Com}$ , ensuring  $\mathbf{e}$  is a weight- $w$  regular vector; and  $\mathbf{c} = \mathbf{e} \cdot \mathbf{H} - \mathbf{x}$ .

**Theorem 2.** Assume  $L, m, t \in \mathbb{N}$  such that the decisional dual-LPN( $N, m, w$ ) problem, with matrix  $\mathbf{H}^\top \leftarrow \mathbf{C}^\perp(N, m, \mathbb{F})$ , is computationally hard, where  $N = w \cdot 2^L$ . If the commitment scheme  $\text{Com}$  is  $\lambda$ -statistically binding and computationally hiding, then the aforementioned hybrid commitment scheme is also  $\lambda$ -statistically binding and computationally hiding.

*Proof.* The required security properties are as follows:

1. **Statistically binding.**  $e$  is  $\lambda$ -statistically binding according to the commitment scheme  $\text{Com}$ . Given that  $\mathbf{x} = e \cdot \mathbf{H} - \mathbf{c}$ , it follows that  $\mathbf{x}$  is also  $\lambda$ -statistically binding.
2. **Computationally hiding.** The hybrid commitment is based on the dual LPN problem, constructing the commitment  $\mathbf{c} = e \cdot \mathbf{H} - \mathbf{x}$ . Leveraging the security properties of the dual LPN problem, the term  $e \cdot \mathbf{H}$  is pseudorandom, ensuring the pseudorandomness of the resulting commitment.

□

## 5 Committed-VOLE

We construct the committed-VOLE protocol using the hybrid commitment we introduced in Section 4.2. The C-VOLE protocol starts from one party  $P_A$ , generating a hybrid commitment of its chosen vector  $\mathbf{x}$  and publishing the commitment to every other party. Then,  $P_A$  generates VOLE correlation with any other party  $P_B$  of the  $\mathbf{x}$  consistent with the published hybrid commitment. In the construction of the hybrid commitment, there are two methods to commit the intermediate vector  $e_u$ : dual-LPN commitment and primal-LPN commitment. This leads to two ways to instantiate the protocol: dual-LPN based C-VOLE and primal-LPN based C-VOLE. Here, we use the dual-LPN based hybrid commitment to  $P_A$ 's input vector  $\mathbf{x}$  and provide its corresponding C-VOLE instantiation in Figure 5. The primal-LPN based C-VOLE is introduced and evaluated in Section A.2. The concrete performance is provided in performance evaluation in Section 7.

The C-VOLE protocol based on dual-LPN hybrid commitment is constructed as follows: First,  $P_A$  commits to its chosen input vector  $\mathbf{x}$ .  $P_A$  samples an intermediate regular sparse vector  $e_u$  and computes a dual-LPN commitment  $\text{com}_{e_u}$ .  $P_A$  computes the hybrid commitment  $\text{Hcom}_{\mathbf{x}}$  of  $\mathbf{x}$  as  $\text{Hcom}_{\mathbf{x}} = \{\text{com}_{e_u}, \mathbf{x}^* = e_u \cdot \mathbf{H} - \mathbf{x}\}$ ,  $\text{Hdecom}_{\mathbf{x}} = \{e_u, \text{decom}_{e_u}\}$ , where  $\mathbf{H}$  is a public matrix and  $\mathbf{x}^*$  is a correction vector from the intermediate  $e_u$  to the designated  $\mathbf{x}$ . Then, to generate the VOLE correlation of the designated  $\mathbf{x}$  committed by  $\text{Hcom}_{\mathbf{x}}$ , we let both parties first generate the VOLE correlation of the intermediate  $e_u$  committed by  $\text{com}_{e_u}$  (part of  $\text{Hcom}_{\mathbf{x}}$ ). Given  $\llbracket e_u \rrbracket$  is consistent with  $\text{com}_{e_u}$ , both parties can locally recover  $\llbracket \mathbf{x} \rrbracket$ , which is consistent with  $\text{Hcom}_{\mathbf{x}}$ , using the publicly known correction vector  $\mathbf{x}^*$  (part of  $\text{Hcom}_{\mathbf{x}}$ ) as follows:

Given  $M[e_u] = K[e_u] + e_u \cdot \Delta$ , where  $(e_u, M[e_u])$  is held by  $P_A$  and  $(K[e_u], \Delta)$  is held by  $P_B$ , and the public correction vector  $\mathbf{x}^* = e_u \cdot \mathbf{H} - \mathbf{x}$ , both parties compute  $M[\mathbf{x}], K[\mathbf{x}]$ , respectively, as:

$$\begin{aligned} M[\mathbf{x}] &= M[e_u] \cdot \mathbf{H} \\ K[\mathbf{x}] &= K[e_u] \cdot \mathbf{H} + \mathbf{x}^* \cdot \Delta, \end{aligned}$$

such that  $M[\mathbf{x}] = K[\mathbf{x}] + \mathbf{x} \cdot \Delta$ .

In this way, generating the VOLE correlation for the committed  $\mathbf{x}$  is reduced to generating the VOLE correlation for the committed intermediate vector  $e_u$ . Notice that  $e_u$  is a regular- $w$  vector with length  $N$ , where  $N = w \cdot 2^L$ , defined in Section 4.2. Both parties call the multi-point VOLE functionality  $\mathcal{F}_{\text{MPVOLE}}$ , as depicted in Figure 3, to generate  $\llbracket e_u \rrbracket$ . The instantiation of  $\mathcal{F}_{\text{MPVOLE}}$

follows the single-point subfield VOLE  $\Pi_{\text{spsVOLE}}$  in [WYKW21] with a slight variation, which is detailed in Section A.5. In the next step,  $P_A$  needs to prove to  $P_B$  that with  $e_u$  input to  $\mathcal{F}_{\text{MPVOLE}}$ , the resulted  $\llbracket e_u \rrbracket$  is consistent with commitment  $\text{com}_{e_u}$ . We construct the proof efficiently leveraging the linear property of the dual-LPN commitment.

Recall that for dual-LPN commitment of  $e_u$ ,  $\text{com}_{e_u} = e_u \cdot \mathbf{H}_1 + e_r \cdot \mathbf{H}_2$  and  $\text{decom}_{e_u} = e_r$ . We let both parties call  $\mathcal{F}_{\text{MPVOLE}}$  again with  $P_A$  input  $\text{decom}_{e_u}(e_r)$  to generate  $\llbracket e_r \rrbracket$ . Given  $M[e_u] = K[e_u] + e_u \cdot \Delta$ , where  $(e_u, M[e_u])$  is held by  $P_A$  and  $(K[e_u], \Delta)$  is held by  $P_B$ , and  $M[e_r] = K[e_r] + e_r \cdot \Delta$ , where  $(e_r, M[e_r])$  is held by  $P_A$  and  $(K[e_r])$  is held by  $P_B$ , both parties compute  $M[\text{com}_{e_u}]$  and  $K[\text{com}_{e_u}]$ , respectively:

$$\begin{aligned} M[\text{com}_{e_u}] &= M[e_u] \cdot \mathbf{H}_1 + M[e_r] \cdot \mathbf{H}_2 \\ K[\text{com}_{e_u}] &= K[e_u] \cdot \mathbf{H}_1 + K[e_r] \cdot \mathbf{H}_2 \end{aligned}$$

Since the LPN-commitment  $\text{com}_{e_u}$  is published in the hybrid commitment  $\text{Hcom}_x$  of  $x$ , the following equation ( $\llbracket \text{com}_{e_u} \rrbracket$ ) holds,

$$M[\text{com}_{e_u}] = K[\text{com}_{e_u}] + \text{com}_{e_u} \cdot \Delta$$

if the  $\llbracket e_u \rrbracket$  and  $\llbracket e_r \rrbracket$  used in computing  $M[\text{com}_{e_u}]$  and  $K[\text{com}_{e_u}]$  are consistent with the  $e_u$  and  $e_r$  in computing  $\text{com}_{e_u}$ . We let  $P_A$  send  $H_3(M[\text{com}_{e_u}])$  to  $P_B$ .  $P_B$  verifies whether  $\llbracket \text{com}_{e_u} \rrbracket$  holds, thereby ensuring that a consistent intermediate VOLE correlation  $\llbracket e_u \rrbracket$  is generated with the intermediate commitment  $\text{com}_{e_u}$ .

The detailed protocol for instantiating  $\mathcal{F}_{\text{C-VOLE}}$  with a dual-LPN commitment is shown in Figure 5. We also proved that the dual-LPN commitment based protocol  $\Pi_{\text{C-VOLE-dual}}$  is secure against any malicious adversary, as established in Theorem 3 with a complete proof.

## 5.1 Proof of Security

**Theorem 3.** *Protocol  $\Pi_{\text{C-VOLE-dual}}$  in Figure 5 securely instantiates  $\mathcal{F}_{\text{C-VOLE}}$  in Figure 1 in  $\mathcal{F}_{\text{MPVOLE}}$ -hybrid model against any malicious adversaries.*

*Proof.* Let  $\mathcal{A}$  be a PPT adversary that corrupt  $P_A$  or  $P_B$ . We construct a PPT simulator  $\mathcal{S}$  with access to  $\mathcal{F}_{\text{C-VOLE}}$  and simulates the adversary's view. We will prove that the joint distribution over the output of  $\mathcal{A}$  and the honest party in the real world is indistinguishable from the joint distribution over the output of  $\mathcal{S}$  and the honest party in the ideal world.

**Corrupted  $P_A$ .** Let  $\mathcal{S}$  access  $\mathcal{F}_{\text{C-VOLE}}$  as an honest  $P_A$  and interact with  $\mathcal{A}$  as an honest  $P_B$ .  $\mathcal{S}$  passes all communication between  $\mathcal{A}$  and environment  $\mathcal{Z}$ .

1.  $\mathcal{S}$  holds public commitment of  $x$ , that  $\text{Hcom}_x = (\text{com}_{e_u}, x^* = e_u \cdot \mathbf{H} - x)$ .  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{MPVOLE}}$  and receives (init) from  $\mathcal{A}$ .
2.  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{MPVOLE}}$  and receives (extend,  $e'_u \in \mathbb{F}^{w \cdot 2^L}, L, w$ ) from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $M[e'_u] \in \mathbb{F}^{w \cdot 2^L}$  from  $\mathcal{A}$ .  $\mathcal{S}$  computes  $x' = e'_u \cdot \mathbf{H} - x^*$ .
3.  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{MPVOLE}}$  and receives (extend,  $e'_r \in \mathbb{F}^{t \cdot 2^{L'}}, L', t$ ) from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $M[e'_r] \in \mathbb{F}^{t \cdot 2^{L'}}$  from  $\mathcal{A}$ .  $\mathcal{S}$  sends  $(x', \text{decom}'_x = (e'_u, e'_r))$  to  $\mathcal{F}_{\text{C-VOLE}}$ . If  $\mathcal{S}$  receives abort from  $\mathcal{F}_{\text{C-VOLE}}$ ,  $\mathcal{S}$  aborts at step (4).
4.  $\mathcal{S}$  receives  $H_3(M[\text{com}_{e_u}])'$  from  $\mathcal{A}$ .  $\mathcal{S}$  checks if  $H_3(M[\text{com}_{e_u}])' = H_3(M[e'_u] \cdot \mathbf{H}_1 + M[e'_r] \cdot \mathbf{H}_2)$ . If not,  $\mathcal{S}$  aborts.
5.  $\mathcal{S}$  computes  $M[x] = M[e'_u] \cdot \mathbf{H}$  and sends  $M[x]$  to  $\mathcal{F}_{\text{C-VOLE}}$ .  $\mathcal{S}$  outputs what  $\mathcal{A}$  outputs.

**Protocol  $\Pi_{\text{C-VOLE-dual}}$**

**Parameters:** A public hybrid commitment of  $\mathbf{x} \in \mathbb{F}^m$  defined in Section 4.2, that  $\text{Hcom}_{\mathbf{x}} = (\text{com}_{e_u} \in \mathbb{F}^n, \mathbf{x}^* = e_u \cdot \mathbf{H} - \mathbf{x} \in \mathbb{F}^m)$ . A dual LPN commitment scheme defined in Section 4.1 that  $(\text{com}_{e_u} = e_u \cdot \mathbf{H}_1 + e_r \cdot \mathbf{H}_2, \text{decom}_{e_u} = e_r) \leftarrow \text{Com}(e_u)$ , where  $e_u \in \mathbb{F}^{w \cdot 2^L}$ ,  $\text{com}_{e_u} \in \mathbb{F}^n$ ,  $\text{decom}_{e_u} \in \mathbb{F}^{t \cdot 2^{L'}}$ . A Collision-Resistant Hash Function  $H_3$ .

**Inputs:**  $P_A$  holds vector  $\mathbf{x} \in \mathbb{F}^m$ ,  $\text{Hdecom}_{\mathbf{x}} = (e_u \in \mathbb{F}^{w \cdot 2^L}, e_r \in \mathbb{F}^{t \cdot 2^{L'}})$ .

**Compute:**

1. Both  $P_A$  and  $P_B$  call  $\mathcal{F}_{\text{MPVOLE}}$  with input (init).  $P_B$  receives  $\Delta \in \mathbb{F}$  from  $\mathcal{F}_{\text{MPVOLE}}$ .
2.  $P_A$  calls  $\mathcal{F}_{\text{MPVOLE}}$  with input (extend,  $e_u \in \mathbb{F}^{w \cdot 2^L}, L, w$ ).  $P_B$  calls  $\mathcal{F}_{\text{MPVOLE}}$  with input (extend,  $L, w$ ). The functionality returns  $M[e_u] \in \mathbb{F}^{w \cdot 2^L}$  to  $P_A$  and  $K[e_u] \in \mathbb{F}^{w \cdot 2^L}$  to  $P_B$  such that  $M[e_u] = K[e_u] + \Delta \cdot e_u \in \mathbb{F}^{w \cdot 2^L}$ . If either party receives **abort** from  $\mathcal{F}_{\text{MPVOLE}}$  in any of these executions, it aborts.
3.  $P_A$  calls  $\mathcal{F}_{\text{MPVOLE}}$  with input (extend,  $e_r \in \mathbb{F}^{t \cdot 2^{L'}}, L', t$ ).  $P_B$  calls  $\mathcal{F}_{\text{MPVOLE}}$  with input (extend,  $L', t$ ). The functionality returns  $M[e_r] \in \mathbb{F}^{t \cdot 2^{L'}}$  to  $P_A$  and  $K[e_r] \in \mathbb{F}^{t \cdot 2^{L'}}$  to  $P_B$  such that  $M[e_r] = K[e_r] + \Delta \cdot e_r \in \mathbb{F}^{t \cdot 2^{L'}}$ . If either party receives **abort** from  $\mathcal{F}_{\text{MPVOLE}}$  in any of these executions, it aborts.
4.  $P_A$  computes  $M[\text{com}_{e_u}] = M[e_u] \cdot \mathbf{H}_1 + M[e_r] \cdot \mathbf{H}_2 \in \mathbb{F}^n$  and sends  $H(M[\text{com}_{e_u}])$  to  $P_B$ .  $P_B$  computes  $K[\text{com}_{e_u}] = K[e_u] \cdot \mathbf{H}_1 + K[e_r] \cdot \mathbf{H}_2 \in \mathbb{F}^n$  and **value**  $= K[\text{com}_{e_u}] + \text{com}_{e_u} \cdot \Delta \in \mathbb{F}^n$ .  $P_B$  checks whether  $H_3(M[\text{com}_{e_u}]) = H_3(\text{value})$ . If not,  $P_B$  aborts.
5.  $P_A$  outputs  $(\mathbf{x}, M[\mathbf{x}])$ , where  $M[\mathbf{x}] = M[e_u] \cdot \mathbf{H} \in \mathbb{F}^m$ .  $P_B$  outputs  $(\Delta, K[\mathbf{x}])$ , where  $K[\mathbf{x}] = K[e_u] \cdot \mathbf{H} + \mathbf{x}^* \Delta \in \mathbb{F}^m$ .

Figure 5: The committed-VOLE protocol based on dual-LPN-based hybrid commitment

We are going to show the simulated execution is indistinguishable from real-world protocol execution.

**Hybrid  $\mathcal{H}_0$**  Same as real-world execution in  $\mathcal{F}_{\text{MPVOLE}}$ -hybrid model.

**Hybrid  $\mathcal{H}_1$**  Same as Hybrid  $\mathcal{H}_0$  except  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{MPVOLE}}$ . Notice that  $\mathcal{S}$  does not simulate any message to  $\mathcal{A}$ . This hybrid is identical to the previous one.

**Hybrid  $\mathcal{H}_2$**  Same as Hybrid  $\mathcal{H}_1$  except  $\mathcal{S}$  aborts at step (4) if  $\mathcal{S}$  receives **abort** from  $\mathcal{F}_{\text{C-VOLE}}$  or the received  $H_3(M[\text{com}_{e_u}])' \neq H_3(M[e'_u] \cdot \mathbf{H}_1 + M[e'_r] \cdot \mathbf{H}_2)$ .

In hybrid  $\mathcal{H}_1$ , an honest  $P_B$  aborts at step (4) if the received  $H_3(M[\text{com}_{e_u}])' \neq H_3(K[e'_u] \cdot \mathbf{H}_1 + K[e'_r] \cdot \mathbf{H}_2 + \text{com}_{e_u} \cdot \Delta)$ . There are the following abort conditions with abort probability indistinguishable from hybrid  $\mathcal{H}_2$ :

- 1)  $\mathcal{A}$  uses  $e'_u \neq e_u$  and  $e'_r \neq e_r$  to  $\mathcal{F}_{\text{MPVOLE}}$ .  $\mathcal{A}$  computes  $M[\text{com}_{e_u}]'$  honestly from  $M[e'_u] \cdot \mathbf{H}_1 + M[e'_r] \cdot \mathbf{H}_2$ . Any  $(e'_u, e'_r)$  inconsistent with  $(e_u, e_r)$  will result the inequality above with the probability of statistical binding failure mentioned in Theorem 1. In this hybrid, any inconsistency will be checked by the  $\mathcal{F}_{\text{C-VOLE}}$  with the same binding failure and result in an abort at step (4), which is indistinguishable from hybrid  $\mathcal{H}_1$ .
- 2)  $\mathcal{A}$  uses  $e'_u \neq e_u$  and  $e'_r \neq e_r$  to  $\mathcal{F}_{\text{MPVOLE}}$ .  $\mathcal{A}$  samples  $M[\text{com}_{e_u}]'$  to  $\mathcal{S}$ . The consistency check passes if and only if  $\mathcal{A}$  samples  $M[\text{com}_{e_u}]' = K[e'_u] \cdot \mathbf{H}_1 + K[e'_r] \cdot \mathbf{H}_2 + \text{com}_{e_u} \cdot \Delta$ . However, since  $K[e'_u], K[e'_r], \Delta$  are uniformly distributed over  $\mathbb{F}^{w \cdot 2^L}, \mathbb{F}^{t \cdot 2^{L'}}$  and  $\mathbb{F}$ , respectively, the consistency check fails with all but negligible probability. In this hybrid, given inconsistent  $(e'_u, e'_r)$  pair,  $\mathcal{S}$  aborts except the failure probability of binding, which is also negligible and thus indistinguishable from hybrid  $\mathcal{H}_1$ .
- 3)  $\mathcal{A}$  uses  $e'_u = e_u$  and  $e'_r = e_r$  to  $\mathcal{F}_{\text{MPVOLE}}$ .  $\mathcal{A}$  samples  $M[\text{com}_{e_u}]'$  to  $\mathcal{S}$ . The consistency check passes if and only if  $\mathcal{A}$  samples



$M[\text{com}_{e_u}]' = K[e_u'] \cdot \mathbf{H}_1 + K[e_r'] \cdot \mathbf{H}_2 + \text{com}_{e_u} \cdot \Delta$ , which is equivalent to checking  $M[\text{com}_{e_u}]' = M[e_u'] \cdot \mathbf{H}_1 + M[e_r'] \cdot \mathbf{H}_2$  in this hybrid. Thus, the abort probability is indistinguishable from hybrid  $\mathcal{H}_1$ .

Therefore, this hybrid is identically distributed as  $\mathcal{H}_1$ .

**Corrupted  $P_B$ .** Let  $\mathcal{S}$  access  $\mathcal{F}_{C\text{-Vole}}$  as an honest  $P_B$  and interact with  $\mathcal{A}$  as an honest  $P_A$ .  $\mathcal{S}$  passes all communication between  $\mathcal{A}$  and environment  $\mathcal{Z}$ .

1.  $\mathcal{S}$  emulates  $\mathcal{F}_{MPVole}$  and receives (init) from  $\mathcal{A}$ .  $\mathcal{S}$  holds public commitment of  $\mathbf{x}$ ,  $H\text{com}_{\mathbf{x}} = (\text{com}_{e_u}, \mathbf{x}^* = e_u \cdot \mathbf{H} - \mathbf{x})$ .  $\mathcal{S}$  sends (init) to  $\mathcal{F}_{C\text{-Vole}}$ .  $\mathcal{S}$  receives  $\Delta \in \mathbb{F}$  from  $\mathcal{A}$ .
2.  $\mathcal{S}$  emulates  $\mathcal{F}_{MPVole}$  and receives (extend,  $L, w$ ) from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $K[e_u] \in \mathbb{F}^{w \cdot 2^L}$  from  $\mathcal{A}$ .
3.  $\mathcal{S}$  emulates  $\mathcal{F}_{MPVole}$  and receives (extend,  $L', t$ ) from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $K[e_r] \in \mathbb{F}^{t \cdot 2^{L'}}$  from  $\mathcal{A}$ .
4.  $\mathcal{S}$  computes  $H_3(M[\text{com}_{e_u}]) = H_3(K[e_u] \cdot \mathbf{H}_1 + K[e_r] \cdot \mathbf{H}_2 + \text{com}_{e_u} \cdot \Delta)$  and sends it to  $\mathcal{A}$ .  $\mathcal{S}$  aborts if  $\mathcal{A}$  aborts.
5.  $\mathcal{S}$  computes  $K[\mathbf{x}] = K[e_u] \cdot \mathbf{H} + \mathbf{x}^* \cdot \Delta \in \mathbb{F}^m$  and sends  $(\Delta, K[\mathbf{x}]) \in \mathbb{F}^{m+1}$  to  $\mathcal{F}_{C\text{-Vole}}$ .  $\mathcal{S}$  outputs what  $\mathcal{A}$  outputs.

We are going to show the simulated execution is indistinguishable from real-world protocol execution.

**Hybrid  $\mathcal{H}_0$**  Same as real-world execution in  $\mathcal{F}_{MPVole}$ -hybrid model.

**Hybrid  $\mathcal{H}_1$**  Same as Hybrid  $\mathcal{H}_0$  except  $\mathcal{S}$  emulates  $\mathcal{F}_{MPVole}$  and simulates the messages to  $\mathcal{A}$ .

$\mathcal{S}$  simulates  $H_3(M[\text{com}_{e_u}]) = H_3(K[e_u] \cdot \mathbf{H}_1 + K[e_r] \cdot \mathbf{H}_2 + \text{com}_{e_u} \cdot \Delta)$  and sends it to  $\mathcal{A}$ . In the previous hybrid, an honest  $P_A$  computes  $M[\text{com}_{e_u}] = M[e_u] \cdot \mathbf{H}_1 + M[e_r] \cdot \mathbf{H}_2$ , which holds the property that  $M[e_u] = K[e_u] + e_u \cdot \Delta$  and  $M[e_r] = K[e_r] + e_r \cdot \Delta$ . Consequently, the  $M[\text{com}_{e_u}]$  computed by simulator equals the one computed by honest  $P_A$  in real world. Thus, this hybrid is identical to the previous one.

This concludes the proof.  $\square$

## 5.2 Complexity Analysis

We give an analysis of the communication efficiency of dual-LPN based C-Vole instantiation. As the protocol call  $\mathcal{F}_{MPVole}$  for generating  $\llbracket e_u \rrbracket$  and  $\llbracket e_r \rrbracket$ , it requires MPVole instantiations of  $e_u \in \mathbb{F}^{w \cdot 2^L}$  and  $e_r \in \mathbb{F}^{t \cdot 2^{L'}}$ . The protocol also requires a  $H_3(M[\text{com}_{e_u}])$  for consistency check and publishing a commitment of size  $n + m$ . According to the instantiation of MPVole protocol in Section A.5, we have that for sparse vector of length  $w \cdot 2^L$ , it requires communication overhead of  $wL\kappa$ . Thus, the total communication overhead of dual-LPN based C-Vole protocol is  $(wL + tL')\kappa + (n + m) \log |\mathbb{F}|$ .

## 6 Applications of C-Vole

We present more details of three applications that can benefit from C-Vole. Our design improves the overall performance of commit-and-prove Vole-ZK, simplifies the communication pattern of Vole-based generic MPC protocols, and facilitates crowd PSI. They are discussed in the following.

## 6.1 Commit-and-prove VOLE-ZK

VOLE-ZK refers to a category of private-coin interactive zero-knowledge proofs [WYKW21, YSWW21, DIO21, BMRS21]. One of its extensions is the commit-and-prove VOLE-ZK that proves a statement over publicly committed data [WYX+21]. To achieve this, [WYX+21] uses a hybrid commitment scheme that is based on the Merkle tree construction [Mer88]. Additionally, it proposes a proof of commitment opening by generically re-computing the whole Merkle tree in ZK.

For an arbitrary circuit  $C(\cdot)$  and a witness  $\mathbf{w}$ , define a VOLE-ZK that proves the validity of  $C(\mathbf{w}) = 1$  without revealing any information about  $\mathbf{w}$ . Additionally, define a Merkle tree-based commitment scheme  $\text{MerkleCom}(\cdot)$  that commits to arbitrary messages. Assume that a prover commits to the witness  $\mathbf{w}$  by  $c = \text{MerkleCom}(\mathbf{w})$ . Given  $(c, C)$ , the commit-and-prove VOLE-ZK can be viewed as proving the possession of witness  $\mathbf{w}$  such that  $c = \text{MerkleCom}(\mathbf{w}) \wedge C(\mathbf{w}) = 1$ .

In [WYX+21], the proof of the first part requires a non-black-box access to the commitment scheme. Namely, the prover needs to prove the satisfiability of a composition of hash function circuits, which incurs  $O(\ell C_{\text{Hash}})$  communication overhead assuming hash function circuits of size  $C_{\text{Hash}}$  per block and  $\mathbf{w} \in \mathbb{F}^\ell$ . Our C-VOLE protocol reduces its cost to  $O(\kappa \log \ell)$  by replacing the Merkle tree commitment with the LPN-based commitment  $\text{HCom}(\cdot)$ , which comes with an efficient proof of opening that facilitates the proof of  $c = \text{HCom}(\mathbf{w}) \wedge C(\mathbf{w}) = 1$ . We demonstrate the concrete performance improvement in Section 7.3.1.

## 6.2 Multi-Party VOLE in MPC and MVZK

It is common for multi-party protocols to rely on a multi-party VOLE protocol to generate correlated randomness. Examples include generic dishonest-majority MPC protocols [RS22, HSS20, WRK17] and multi-verifier zero-knowledge proofs [EPSW24]. In multi-party VOLE, each pair of parties  $(P_i, P_j)$  need to execute VOLE for the correlation  $M[\mathbf{x}^i]_j^i = K[\mathbf{x}^i]_j^i + \mathbf{x}^i \cdot \Delta_j^i$ . Achieving malicious security in such protocols requires that  $P_i$  uses the same  $\mathbf{x}^i$  across all VOLE instances it executes with other parties. To ensure this, a costly consistency check is performed at the end of the multi-party VOLE. In prior works, this check involves  $n$ -party joint coin-tossing and multiple rounds of peer-to-peer communication and broadcasting, which are significantly constrained by network latency.

Our C-VOLE protocol eliminates this round-heavy consistency check. Namely,  $P_i$  first publishes its commitment to  $\mathbf{x}^i$ . Then, for each  $j \in [n] \setminus \{i\}$ ,  $P_i$  interacts with  $P_j$  to execute committed VOLE, which ensures the VOLE correlations are consistent with the committed  $\mathbf{x}^i$ . As long as parties reach consensus on the commitment, there is no need for a final multi-round verification.

## 6.3 C-VOLE in Crowd PSI

Crowd PSI involves a server executing 2-party PSI with multiple clients in a distributed manner; any standalone malicious secure 2-party PSI is insufficient, as the server may use different private sets interacting with different clients. Introducing the C-VOLE could address this issue by building upon VOLE-based PSI works [RR22a, RS21], the most performant PSI works currently, and extending them to a C-VOLE based crowd PSI protocol. The protocol is constructed as follows to securely achieve  $\mathcal{F}_{\text{PSI}}$  in Figure 4. The protocol is shown in Figure 6 and its intuition is detailed as follows:

Initially,  $P_1$  computes a vector  $\mathbf{p}$  by solving a linear system of its private set elements, and publishes its commitment. Then, any client  $P_j$  executes the C-VOLE functionality with  $P_1$  to get a VOLE relation associated with  $\mathbf{p}$ . Given the VOLE relation, expressed as  $M[\mathbf{p}]_j = K[\mathbf{p}]_j + \mathbf{p} \cdot \Delta_j$ , party  $P_j$  uses  $K[\mathbf{p}]_j$  and  $\Delta_j$  to encode its private set elements and then sends the encodings to  $P_1$ .  $P_1$  uses the corresponding  $M[\mathbf{p}]_j$  to encode its private set elements  $x_i \in X, i \in [n]$ . Then,  $P_1$  is

### Protocol $\Pi_{\text{PSI}}$

**Parameters:** Party  $P_1$  with a set  $X = (x_1, \dots, x_n) \subset \mathbb{F}^n$ . Party  $P_j$  with a set  $Y_j \subset \mathbb{F}^{n'}$ . A random function  $\text{row} : \mathbb{F} \times \{0, 1\}^\kappa \rightarrow \{0, 1\}^m$ , where  $m = O(n) + O(\lambda)$  as defined by OKVS scheme in [RR22a]. A Decode algorithm  $\text{Decode} : \mathbb{F}^m \times \mathbb{F} \times \{0, 1\}^\kappa \rightarrow \mathbb{F}$ , defined by [RR22a]. A hybrid commitment scheme defined in Section 4.2, such that  $(\text{Hcom}_{\mathbf{a}}, \text{Hdecom}_{\mathbf{a}}) \leftarrow \text{HCom}(\mathbf{a})$ ,  $1 \leftarrow \text{Open}(\mathbf{a}, \text{Hdecom}_{\mathbf{a}}, \text{Hcom}_{\mathbf{a}})$ , where  $\mathbf{a} \in \mathbb{F}^m$ ,  $\text{Hcom}_{\mathbf{a}} \in \mathbb{F}^N$ ,  $\text{Hdecom}_{\mathbf{a}} \in \mathbb{F}^{m'}$ . Random oracle  $H_1 : \mathbb{F} \rightarrow \mathbb{F}$ ,  $H_2 : \mathbb{F} \times \mathbb{F} \rightarrow \{0, 1\}^{\text{out}}$ .

**Initialize:**

1.  $P_1$  samples  $r \leftarrow \{0, 1\}^\kappa$  and solves the following systems to get  $\mathbf{p} \in \mathbb{F}^m$ .

$$\begin{bmatrix} \text{row}(x_1, r) \\ \dots \\ \text{row}(x_n, r) \end{bmatrix} \mathbf{p}^T = (H_1(x_1), \dots, H_1(x_n))^T$$

2.  $P_1$  computes  $(\text{Hcom}_{\mathbf{p}}, \text{Hdecom}_{\mathbf{p}}) \leftarrow \text{HCom}(\mathbf{p})$  and publishes  $\text{Hcom}_{\mathbf{p}}$ .

**Extend:**  $P_1$  execute this phase with each party  $P_j$ , who possesses set  $Y_j$  consisting elements  $(y_1, \dots, y_{n'})$ .

3.  $P_j$  samples  $w^s \leftarrow \mathbb{F}$ , sends  $c^s := H_1(w^s)$  to  $P_1$ .
4.  $P_1$  samples  $w^r \leftarrow \mathbb{F}$ , sends  $w^r, r$  to  $P_j$ .
5.  $P_1$  calls  $\mathcal{F}_{\text{C-VOLE}}$  with input  $(\text{init}, \mathbf{p}, \text{Hdecom}_{\mathbf{p}})$ .  $P_j$  call  $\mathcal{F}_{\text{C-VOLE}}$  with input  $(\text{init})$ . The functionality sends  $M[\mathbf{p}] \in \mathbb{F}^m$  to  $P_1$  and  $(\Delta, K[\mathbf{p}]) \in \mathbb{F}^{m+1}$  to  $P_j$ , such that  $M[\mathbf{p}] = K[\mathbf{p}] + \mathbf{p} \cdot \Delta$ .
6.  $P_j$  sends  $w^s$  to  $P_1$ .  $P_1$  aborts if  $c^s \neq H_1(w^s)$ . Otherwise, both party computes  $w = w^s + w^r$ .
7.  $P_j$  computes  $ey_i = F(y_i) = H_2(\text{Decode}(K[\mathbf{p}], y_i, r) + H_1(y_i) \cdot \Delta + w, y_i)$  for each  $y_i \in Y_j$ , inserts  $ey_i$  to set  $EY$ , and sends  $EY$  to  $P_1$ .
8.  $P_1$  computes  $ex_i = F(x_i) = H_2(\text{Decode}(M[\mathbf{p}], x_i, r) + w, x_i)$  for each  $x_i \in X$ , inserts  $ex_i$  to set  $EX$ .  $P_1$  computes the intersection  $Z : \{x_i | x_i \in X, ex_i \in EX \cap EY\}$

Figure 6: Secure crowd PSI protocol.

able to compute the intersection by locally comparing the encodings. By leveraging  $\mathcal{F}_{\text{C-VOLE}}$  and pre-published commitment of  $\mathbf{p}$ , this protocol ensures that  $P_1$  consistently reuses  $\mathbf{p}$  in the extend phase with different  $P_j$ .

The correctness of encoding and computing the intersection aligns with the VOLE-based PSI protocol [RS21]. Each set element  $x_i/y_i$  is mapped to a specific row vector through random function  $\text{row}$ .  $P_j$  encodes its set elements  $y_i, i \in [n']$  by applying  $K[\mathbf{p}]$  to these row vectors to compute the inner product and then add  $H(y_i) \cdot \Delta$ .  $P_1$  encodes its set elements  $x_i, i \in [n]$  by computing the inner product of  $M[\mathbf{p}]$  with the row vectors mapped from  $x_i, i \in [n]$ . According to the computation of  $\mathbf{p}$ , for each  $y_{i'} = x_i, i \in [n], i' \in [n']$ ,  $H(x_i)/H(y_{i'})$  is equivalent to applying the corresponding row vectors of  $x_i/y_{i'}$  to  $\mathbf{p}$ . Thus, given the VOLE relation of  $\mathbf{p}$ , both parties are able to encode identical elements into identical encodings. An enhanced Oblivious Key-Value Store (OKVS) scheme from [RR22a] is used here to sample the random function  $\text{row}$ , solve the linear system of  $\mathbf{p}$ , and compute the Decode algorithm. We proved that this crowd PSI protocol is maliciously secure in each distributed two-party PSI computation below. We also want to noted that the coin-flipping procedure in generating  $w$  is not mandatory in this crowd PSI protocol as it is originally used for achieving the OPRF protocol in [RS21], which requires uniformity of OPRF output.

**Theorem 4.** *Protocol  $\Pi_{\text{PSI}}$  in Figure 6 securely instantiates  $\mathcal{F}_{\text{PSI}}$  in  $(\mathcal{F}_{\text{C-VOLE}}, H_1, H_2)$ -hybrid model against malicious adversaries.*

*Proof.* The proof is deferred to Section A.10 □

Additionally, even in semi-honest setting, implementing C-VOLE in crowd PSI could bring the correction vector, which is computed from the difference between randomized VOLE vector and the designated vector, to the initialization phase. This optimization ensures the same correction vector is used for all senders, reduces communication overhead by eliminating the need for individual correction vectors for each sender.

## 7 Performance Evaluation

We implement our proposed C-VOLE protocol and the C-VOLE based crowd PSI protocol based on the work by [RR22a]. We utilize the EMP library [WMK16] for C-VOLE implementation. For the PSI protocol, we integrate the libOT [RR] and volePSI [RR22b] libraries for Paxos operations. We instantiate all the protocols with  $\mathbb{F} = \mathbb{F}_{2^{128}}$ , the computational security parameter  $\kappa = 128$ , and the statistical security parameter  $\lambda = 40$ . All experiments were executed on AWS EC2 instances 6a.8xlarge with 32vCPU and 128GB memory. We tested our schemes in both the LAN and WAN networks, where we set the LAN network with 0 ms latency and 1Gbps bandwidth, and WAN network with 400 ms latency and 50 Mbps bandwidth.

### 7.1 Parameter Selection of Hybrid Commitment

We explain how to choose parameters for a hybrid commitment  $\text{Hcom}_{\mathbf{x}}$  of  $\mathbf{x} \in \mathbb{F}^m$ , which will influence the performance of C-VOLE as well. Notice that  $\text{Hcom}_{\mathbf{x}} = \{\text{com}_{\mathbf{e}_u}, \mathbf{x}^* = \mathbf{e}_u \cdot \mathbf{H} - \mathbf{x}\}$  consists of two parts: the LPN-based commitment  $\text{com}_{\mathbf{e}_u}$  of  $\mathbf{e}_u \in \mathbb{F}^{w \cdot 2^L}$ ; and a pseudorandom correction vector  $\mathbf{x}^*$  corrects  $\mathbf{x}$  from  $\mathbf{e}_u$ . We first select the parameters used for computing  $\mathbf{x}^*$ , which hides  $\mathbf{x}$  with 128-bit security. Our framework is compatible with any LPN coding in the VOLE context [BCG<sup>+</sup>22, RRT23]. We use the expand-accumulate (EA) code [BCG<sup>+</sup>22] to build the matrix  $\mathbf{H} \in \mathbb{F}^{w \cdot 2^L \times m}$ , where the expansion parameter  $C$  corresponds to the “expand” step in EA coding. This matrix compresses vector  $\mathbf{e}_u \in \mathbb{F}^{w \cdot 2^L}$  into a pseudorandom vector of length  $m$ . Following the conservative constraints in Section 7.1 and [BCG<sup>+</sup>22], we set the expansion parameter  $C = 10$  and the code rate  $R = 1/7$  (i.e.,  $7 * |\mathbf{x}| = |\mathbf{e}_u|$ ). Under these settings, we choose the  $(w, L)$  pair for  $\mathbf{e}_u \in \mathbb{F}^{w \cdot 2^L}$  to achieve 128-bit security. The chosen parameters are shown in Table 1. Notice that [RRT23]’s attack only applies to the aggressive parameters of EA code but not the conservative ones we use.<sup>1</sup>

Given the  $(w, L)$  pair for  $m$ , we fix the structure of  $\mathbf{e}_u$  for each  $\mathbf{x}$ . Then, we choose parameters for constructing a commitment  $\text{com}_{\mathbf{e}_u}$  with 128-bit security level, where  $\text{com}_{\mathbf{e}_u} = \mathbf{e}_u \cdot \mathbf{H}_1 + \mathbf{e}_r \cdot \mathbf{H}_2$  for the commitment from the dual LPN assumption, to achieve statistically binding and computationally hiding commitment of  $\mathbf{e}_u \in \mathbb{F}^{w \cdot 2^L}$ . According to Theorem 1, we select the  $(n, t, L')$  pair for dual LPN commitment. For each  $\mathbf{x} \in \mathbb{F}^m$  of different lengths, we compute groups of parameters  $(w, L, n, t, L')$  and select the one that minimizes the communication cost of the C-VOLE protocol from the dual LPN assumption, mentioned in Section 5.2 for implementation. We give the commitment parameter selection that achieving 128 bit security for  $\mathbf{x} \in \{\mathbb{F}^{2^{20}}, \mathbb{F}^{2^{24}}, \mathbb{F}^{2^{28}}\}$  in Table 1.

<sup>1</sup>The attack only applies to  $\mathbb{F}_2$  while our implementation is based on  $\mathbb{F}_2^{128}$ . Therefore, we also provide performance evaluation of C-VOLE under aggressive parameters in Section A.4.

| Input          | Intermediate sparse vector |     |     | Dual LPN commitment |     |      |                         |
|----------------|----------------------------|-----|-----|---------------------|-----|------|-------------------------|
| $ \mathbf{x} $ | $ e_u $                    | $w$ | $L$ | $ e_r $             | $t$ | $L'$ | $ \text{com}_{e_u} (n)$ |
| $2^{20}$       | $7 * 2^{20}$               | 224 | 15  | 1456                | 91  | 4    | 696                     |
| $2^{24}$       | $7 * 2^{24}$               | 224 | 19  | 1472                | 92  | 4    | 710                     |
| $2^{28}$       | $7 * 2^{28}$               | 224 | 23  | 1488                | 93  | 4    | 723                     |

Table 1: Dual-LPN-based hybrid commitment parameters selection of  $\mathbf{x}$  with different lengths. Achieves 128-bit security.

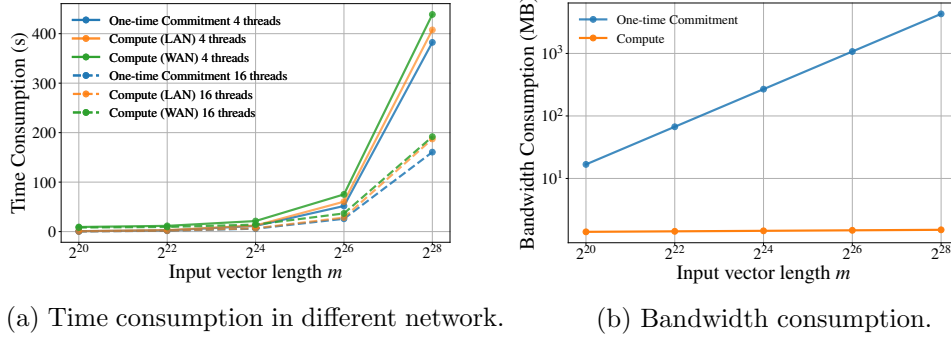


Figure 7: Performance of C-VOLE protocol in LAN and WAN setting.

## 7.2 Efficiency of Committed-VOLE

Our C-VOLE protocol includes two phases: computing a one-time commitment on the VOLE sender’s side ( $P_A$  in the protocol) for chosen input vector  $\mathbf{x}$  and a compute phase that can be executed between the sender and each individual VOLE receiver ( $P_B$  in the protocol). We show the time consumption of the two phases under LAN and WAN with different input vector lengths  $m$  and different threads in Figure 7a. The communication overhead is shown in Figure 7b. The detailed analysis is as follows.

**One-time Commitment:** In this phase, the sender, who holds the input vector  $\mathbf{x}$ , generates an LPN-based hybrid commitment of  $\mathbf{x}$  and publishes it to all the parties. We tested this process using 4 threads and 16 threads. The results show that it takes 0.84 s for  $m = 2^{20}$  and 382.39 s for  $m = 2^{28}$  using 4 threads, while it takes 0.33 s for  $m = 2^{20}$  and 160.62 s for  $m = 2^{28}$  using 16 threads. The communication overhead in this phase is publishing the hybrid commitment of  $\mathbf{x}$ , which costs 16.78 MB for  $m = 2^{20}$  and 4294.9 MB for  $m = 2^{28}$ , dominating the entire communication of the C-VOLE protocol. As the one-time commitment is published and applicable to each individual receiver interacting with the sender to generate C-VOLE of  $\mathbf{x}$ , the time consumption for the computation can be amortized to a very small portion of the overall computation. Additionally, the commitment computation is computed locally on the sender’s side and is unaffected by network settings.

**Compute:** We tested the compute phase for the sender generating VOLE of the committed  $\mathbf{x}$  with one receiver. Note that the performance of the compute phase is influenced by the parameters chosen in Section 7.1, specifically the Hamming weight and the block size of the sparse vectors in executing  $\Pi_{\text{MPVOLE}}$ . For  $m = 2^{20}$ , the compute phase takes 0.81 s under a LAN network and 9.31 s under a WAN network with a bandwidth of 50 Mbps when using 4 threads. With 16 threads, the compute phase takes 0.62 seconds under LAN network and 8.20 seconds under WAN network. For  $m = 2^{28}$ , the compute phase with 4 threads takes 407.6 s under a LAN network and 438.8 s under a WAN network. With 16 threads, the corresponding times are 187.22 s under a

| Schemes                   | 50 Mbps      | 200 Mbps     | 500 Mbps     | 1 Gbps       |
|---------------------------|--------------|--------------|--------------|--------------|
| C2A [WYX <sup>+</sup> 21] | 45 $\mu$ s   | 45 $\mu$ s   | 44 $\mu$ s   | 44 $\mu$ s   |
| C-VOLE                    | 1.61 $\mu$ s | 1.59 $\mu$ s | 1.57 $\mu$ s | 1.53 $\mu$ s |

Table 2: Time consumption compared with [WYX<sup>+</sup>21]. With different network settings, we tested C-VOLE of 128-bit data block to compare with C2A of 64-bit data block. Both evaluations are with single thread.

| Schemes                   | $2^{20}$ block | $2^{24}$ block | $2^{28}$ block |
|---------------------------|----------------|----------------|----------------|
| C2A [WYX <sup>+</sup> 21] | 114.52 MB      | 1829.42 MB     | 29317.12 MB    |
| C-VOLE                    | 18.49 MB       | 270.18 MB      | 4296.74 MB     |

Table 3: Bandwidth consumption compared with [WYX<sup>+</sup>21]. we tested the bandwidth consumption of C-VOLE and C2A under varying input blocks. For each C-VOLE input block, we used a 128-bit data block, while for C2A, a 64-bit data block was employed for comparison.

LAN network and 191.82 s under a WAN network. In terms of communication overhead, which is dominated by the cost of  $\Pi_{\text{MPVole}}$ , the cost is 1.72 MB for  $m = 2^{20}$  and 1.78 MB for  $m = 2^{28}$ . The communication overhead does not increase significantly with the increase of  $m$ , benefiting from the sublinear online communication of our intermediate structured sparse LPN commitment and corresponding consistency check construction.

### 7.3 Comparison with Prior Work

#### 7.3.1 Comparison with C2A Conversion.

We compare the proposed C-VOLE protocol with the public commitment to private authentication conversion (C2A) in [WYX<sup>+</sup>21], which is used for commit-and-prove zero-knowledge proofs. The C2A conversion in [WYX<sup>+</sup>21] assumes 64-bit data block, while our protocol is evaluated under field  $\mathbb{F}_{2^{128}}$ , implemented as data block of 128-bit. The time consumption and communication overhead are summarized in Table 2 and Table 3 and are analyzed below.

**Wallclock time.** C2A conversion is the bottleneck of computing commit-and-prove ZK. As tested from [WMK16], the C2A takes 45  $\mu$ s per element while the zero-knowledge proof only takes less than 0.1  $\mu$ s per gate [YSWW21]. We tested ours C-VOLE computation time for each block under different network setting, each result is amortized from the performance of vector of length  $2^{24}$ . The results show that our scheme requires 1.61 $\mu$ s computation for 128-bit data block, which is  $28\times$  faster than the 45 $\mu$ s computation for 64-bit data block in [WYX<sup>+</sup>21]. Note that a follow-up LPZKv2 [DILO22b] provides at most a  $2\times$  improvement over QuickSilver [YSWW21]. The advantage of C-VOLE compared to Mystique remains significant even if its underlying ZKP is replaced from QuickSilver to LPZKv2.

**Communication overhead.** We tested the communication overhead of the C2A conversion and our proposed C-VOLE for input vector length  $m = \{2^{20}, 2^{24}, 2^{28}\}$ . The communication overhead includes the published commitment, the interactive generation of VOLE, and the proof of input vector consistency. C2A conversion requires 114.52 MB of communication for  $2^{20}$  blocks while ours C-VOLE only requires 18.49 MB. For  $2^{28}$  blocks, the C2A consumes 29 GB communication overhead and ours C-VOLE consumes 4.2 GB. Note that in the C2A conversion, the commitment size is small, and the proof of consistency dominates the entire communication cost. In contrast, for our C-VOLE, the commitment size constitutes the majority of the communication overhead.



This means that when executing C2A operations multiple times on the same publicly committed data, ours C-VOLE protocol is much more communication-efficient.

### 7.3.2 Comparison with CVOLE with Input-revealing Identifiable-abort (IRIA).

We compare the committing VOLE proposed in [CDKs24] with our proposed scheme. The committing VOLE with input-revealing identifiable-abort is designed having both parties commit to their input and reveal them to prove consistency in case of an abort. To ensure a fair comparison with our work, we only require the sender to commit to its input vector and reveal it for consistency checks. Since there is no implementation available for [CDKs24], we analyze and compare its communication overhead with ours theoretically.

The C-VOLE used in [CDKs24] can be divided into two steps according to their paper: 1) Generate oblivious transfer (OT) based on PVW OT [PVW08] and extend it to VOLE using OT-multiplication protocols [DKLs18, Gil99]; 2) Run a sigma protocol to reveal the input to the other party and prove consistency of the input with previous transferred messages for each OT. For step 1), each DDH-based PVW OT requires  $8 \log |\mathbb{G}|$  communication. To extend it to VOLE with vector length  $m$ , the extension [DKLs18] requires  $m \cdot (\log |\mathbb{F}| + \kappa + 2\lambda)$  execution of OTs, along with additional consistency check of the sender’s input for malicious security, which doubles the the number of OTs. For step 2), each OT requires a communication overhead of  $4 \log |\mathbb{F}| + 8 \log |\mathbb{G}|$  for the sigma protocol to prove consistency with the commitment (transferred message). The total communication overhead for the C-VOLE with sender-committed input and revealing is  $2 \cdot m \cdot (\log |\mathbb{F}| + \kappa + 2\lambda) \cdot (8 \log |\mathbb{G}| + 4 \log |\mathbb{F}| + 8 \log |\mathbb{G}|)$ . In contrast, ours communication overhead is  $(wL + tL')\kappa + (n + m) \log |\mathbb{F}|$  as analyzed in Section 5.2. For  $m = 2^{20}$ ,  $\mathbb{G}$  with order 128 bits, the communication cost for [CDKs24] is 225 GB, whereas ours is 16.84 MB.

## 7.4 Efficiency of C-VOLE based PSI and Comparison

In this section, we evaluate the performance of our proposed C-VOLE based crowd PSI and compare it with the state-of-the-art two party PSI [RR22a] and the state-of-the-art crowd PSI protocol [SKR<sup>+</sup>24].

We integrate C-VOLE into VOLE-based PSI protocol, Blazing PSI [RR22a], which is also the state-of-the-art two-party PSI protocol, by replacing VOLE with C-VOLE to achieve malicious security in the crowd setting. As shown in Figure 6, in the C-VOLE based crowd PSI protocol, the C-VOLE sender acts as the receiver (server) of the crowd PSI protocol and computes a publicly committed vector  $\mathbf{p}$  for initialization. The server (receiver) then computes the extend phase of crowd PSI with each individual client (sender) to get the intersection. The initialization phase includes a Paxos encoding operation and the computation of a one-time commitment. The extend phase consists of the following steps: 1) Both the server and the client generate the VOLE correlation of the committed  $\mathbf{p}$ ; 2) The client sends the encoding of its set elements to the server; 3) The server performs local encoding of its set and compares the two encodings to determine the intersection with each client.

### 7.4.1 Comparison with VOLE based PSI.

The only difference between ours C-VOLE based PSI and the VOLE-based PSI [RR22a] is that we use committed-VOLE instead of VOLE. Notice that [RR22a] proposed two versions of implementation, one uses VOLE with slightly faster computational overhead, the other one uses subfield-VOLE with reduced communication overhead. We compare with the faster version (VOLE-based) and measure both the computation and the communication overhead by fixing client’s set to  $2^{10}$



elements and scaling the server’s set to  $\{2^{20}, 2^{22}, 2^{24}\}$ . The detailed comparison are presented in Table 4 and Table 5.

| Set size $(m, n)$       |            | $(2^{20}, 2^{10})$ | $(2^{22}, 2^{10})$ | $(2^{24}, 2^{10})$ |
|-------------------------|------------|--------------------|--------------------|--------------------|
| Blazing PSI [RR22a] (s) |            | 0.41               | 2.44               | 9.89               |
| C-VOLE PSI              | Init (s)   | 0.78               | 3.34               | 12.37              |
|                         | Online (s) | 1.08               | 3.51               | 14.69              |

Table 4: Time consumption compared with [RR22a].  $(m, n)$  represents the server’s set size  $m$ , and client’s set size  $n$ . PSI Init is a one-time computation and PSI online is executed between the server and each individual client. The experiments are tested with 16 threads.

| Set size $(m, n)$        |             | $(2^{20}, 2^{10})$ | $(2^{22}, 2^{10})$ | $(2^{24}, 2^{10})$ |
|--------------------------|-------------|--------------------|--------------------|--------------------|
| Blazing PSI [RR22a] (MB) |             | 23.83              | 92.34              | 366.55             |
| C-VOLE PSI               | Init (MB)   | 21.94              | 88.41              | 353.94             |
|                          | Online (MB) | 1.73               | 1.75               | 1.76               |

Table 5: Communication overhead compared with [RR22a].  $(m, n)$  represents the server’s set size  $m$ , and client’s set size  $n$ . The experiments are executed with 16 threads.

Compared with Blazing PSI [RR22a], the results show that our C-VOLE based PSI consumes approximately  $2\times$  more online running time but requires significantly less online communication overhead. This difference can be attributed to the fact that Blazing PSI employs the default VOLE implementation in the library LibOTe using EC code. In contrast, ours uses EA code with parameter  $R = \frac{1}{7}$ ,  $C = 10$  and specifically selects the structure of  $e_u$  and  $e_r$  for both security and performance in Section 7.1. Moreover, our C-VOLE based PSI has the property of having the initialization phase performed locally in advance to enhance performance. In the crowd PSI setting, when a server executes PSI with multiple clients, the initialization can be amortized to a very small amount of time. The communication can also be managed by other efficient networks such as CDN or P2P, further enhancing performance.

#### 7.4.2 Comparison with Prior Work in Crowd PSI.

There’s only one prior work that considers PSI in the crowd setting [SKR<sup>+</sup>24]. In this work, the server commits to its private input and executes PSI with each individual client using an oblivious verifiable unpredictable function (OVUF). This work allows the client to obtain the intersection, whereas our protocol enables the server to obtain the intersection. We compare our work with this work despite the slight difference in output, as it provides the relevant baseline for performance in crowd setting. We set the server’s set size to  $\{2^{20}, 2^{24}, 2^{28}\}$  elements and the client’s set size to  $\{2^{10}, 2^{17}, 2^{20}\}$  elements, and present the performance results below.

**Wallclock time.** We evaluate the time consumption for executing PSI between a server and a client using 16 threads. As shown in Table 6, the initialization of both schemes scales linearly with the server’s set size, as it computes commitment for the server’s private set elements. Ours C-VOLE PSI requires 465.23 ms for a server with  $2^{28}$  elements, while OVUF-PSI requires 1596.44 ms. For the online phase, our C-VOLE based PSI includes generating the VOLE correlation of the committed vector (with a length linear to the server set size) and computing the client’s side encoding to the server. The time consumption scales roughly linearly with the server’s set size, with a small additional increase as the client set size grows. In contrast, the OVUF-PSI scales

| Set size $(m, n)$  | C-VOLE PSI |            | OVUF-PSI [SKR <sup>+</sup> 24] |            |
|--------------------|------------|------------|--------------------------------|------------|
|                    | Init (s)   | Online (s) | Init (s)                       | Online (s) |
| $(2^{20}, 2^{10})$ | 0.78       | 1.03       | 6.24                           | 0.59       |
| $(2^{20}, 2^{17})$ | 0.79       | 1.08       | 6.24                           | 67.36      |
| $(2^{20}, 2^{20})$ | 0.79       | 1.23       | 6.24                           | 538.88     |
| $(2^{24}, 2^{10})$ | 12.37      | 14.35      | 98.14                          | 0.59       |
| $(2^{24}, 2^{17})$ | 12.37      | 14.69      | 98.14                          | 67.36      |
| $(2^{24}, 2^{20})$ | 15.65      | 16.57      | 98.14                          | 538.88     |
| $(2^{28}, 2^{10})$ | 458.17     | 455.55     | 1596.44                        | 0.59       |
| $(2^{28}, 2^{17})$ | 458.10     | 499.04     | 1596.44                        | 67.36      |
| $(2^{28}, 2^{20})$ | 465.23     | 523.17     | 1596.44                        | 538.88     |

Table 6: Time consumption compared with [SKR<sup>+</sup>24].  $(m, n)$  represents the server’s set size  $m$ , and the client’s set size  $n$ . PSI Init is a one-time computation and PSI Online is executed between the server and each individual client. All experiments are executed with 16 threads. The online computation time for [SKR<sup>+</sup>24] with a client size of  $2^{20}$  is derived from other tested results based on linear properties.

linearly with the client set size. For a server’s set size of  $2^{20}$ , ours C-VOLE PSI requires 1.03 s for client size  $2^{10}$  and 1.23 s for client size  $2^{20}$ . In comparison, OVUF-PSI requires 0.59 s for client size  $2^{10}$  and 538.88 s for client size  $2^{20}$ .

**Communication overhead.** For communication overhead, the initialization phase of both schemes requires the server to publish the commitment of its set elements. The communication overhead scales linearly with the server set size. However, in OVUF-PSI, each element in  $\mathbb{F}_{2^{128}}$  is hashed to 64-bit, while ours C-VOLE PSI can not use such hashing. The communication overhead of our initialization phase scales linearly with the length of  $\mathbf{p}$ , which is proportional to the server’s set size. Consequently, our C-VOLE PSI requires roughly 2.6 times more communication overhead than the OVUF-PSI during initialization. In the online phase, C-VOLE PSI includes generating the C-VOLE correlation of  $\mathbf{p}$  and having the client send back its set encodings. The online communication overhead is dominated by the client’s set size, requires 1.73 MB for a client size of  $2^{10}$  and 18.49 MB for a client size of  $2^{20}$ , with a fixed server size  $2^{20}$ . This significantly outperforms the OVUF-PSI online phase.

| Set size $(m, n)$  | C-VOLE PSI |             | OVUF-PSI [SKR <sup>+</sup> 24] |             |
|--------------------|------------|-------------|--------------------------------|-------------|
|                    | Init (MB)  | Online (MB) | Init (MB)                      | Online (MB) |
| $(2^{20}, 2^{10})$ | 21.94      | 1.73        | 8.38                           | 63.23       |
| $(2^{20}, 2^{17})$ | 21.94      | 3.81        | 8.38                           | 8091        |
| $(2^{20}, 2^{20})$ | 21.94      | 18.49       | 8.38                           | 64747       |
| $(2^{24}, 2^{10})$ | 353.94     | 1.76        | 134.18                         | 63.23       |
| $(2^{24}, 2^{17})$ | 353.94     | 3.84        | 134.18                         | 8091        |
| $(2^{24}, 2^{20})$ | 353.94     | 18.52       | 134.18                         | 64747       |
| $(2^{28}, 2^{10})$ | 5671.63    | 1.79        | 2147                           | 63.23       |
| $(2^{28}, 2^{17})$ | 5671.63    | 3.87        | 2147                           | 8091        |
| $(2^{28}, 2^{20})$ | 5671.62    | 18.55       | 2147                           | 64747       |

Table 7: Communication overhead compared with [SKR<sup>+</sup>24].  $(m, n)$  represents the server’s set size  $m$ , and client’s set size  $n$ . The online communication time for [SKR<sup>+</sup>24] with a client size of  $2^{20}$  is derived from other tested results based on linear properties.

## Acknowledgements

The work of Yu Yu is supported by the National Key Research and Development Program of China (Grant No. 2020YFA0309705), the National Natural Science Foundation of China (Grant Nos. 62125204 and 92270201), the Innovation Program for Quantum Science and Technology (No. 2021ZD0302901/2021ZD0302902), and the Major Program of Guangdong Basic and Applied Research (Grant No. 2019B030302008). The work of Xiao Wang is supported by NSF award #2236819. Yu Yu's work is also supported by the New Corner-stone Science Foundation through the XPLOER PRIZE.

## References

- [AAB<sup>+</sup>20] Abdelrahman Aly, Tomer Ashur, Eli Ben-Sasson, Siemen Dhooghe, and Alan Szepieniec. Design of symmetric-key primitives for advanced cryptographic protocols. *IACR Trans. Symm. Cryptol.*, 2020(3):1–45, 2020.
- [AGR<sup>+</sup>16] Martin R. Albrecht, Lorenzo Grassi, Christian Rechberger, Arnab Roy, and Tyge Tiessen. MiMC: Efficient encryption and cryptographic hashing with minimal multiplicative complexity. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part I*, volume 10031 of *LNCS*, pages 191–219, Hanoi, Vietnam, December 4–8, 2016. Springer Berlin Heidelberg, Germany.
- [BCG<sup>+</sup>17] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, and Michele Orrù. Homomorphic secret sharing: Optimizations and applications. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 2105–2122, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.
- [BCG<sup>+</sup>19] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 489–518, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.
- [BCG<sup>+</sup>22] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Nicolas Resch, and Peter Scholl. Correlated pseudorandomness from expand-accumulate codes. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 603–633, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- [BCGI18] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 896–912, Toronto, ON, Canada, October 15–19, 2018. ACM Press.
- [BDOZ11] Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. Semi-homomorphic encryption and multiparty computation. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 169–188, Tallinn, Estonia, May 15–19, 2011. Springer Berlin Heidelberg, Germany.

- [BFKL94] Avrim Blum, Merrick L. Furst, Michael J. Kearns, and Richard J. Lipton. Cryptographic primitives based on hard learning problems. In Douglas R. Stinson, editor, *CRYPTO'93*, volume 773 of *LNCS*, pages 278–291, Santa Barbara, CA, USA, August 22–26, 1994. Springer Berlin Heidelberg, Germany.
- [BGI15] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 337–367, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.
- [BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1292–1303, Vienna, Austria, October 24–28, 2016. ACM Press.
- [BMRS21] Carsten Baum, Alex J. Malozemoff, Marc B. Rosen, and Peter Scholl. Mac’n’cheese: Zero-knowledge proofs for boolean and arithmetic circuits with nested disjunctions. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 92–122, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.
- [CDKs24] Ran Cohen, Jack Doerner, Yashvanth Kondi, and abhi shelat. Secure multiparty computation with identifiable abort via vindicating release. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VIII*, volume 14927 of *LNCS*, pages 36–73, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.
- [DILO22a] Samuel Dittmer, Yuval Ishai, Steve Lu, and Rafail Ostrovsky. Authenticated garbling from simple correlations. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 57–87, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- [DILO22b] Samuel Dittmer, Yuval Ishai, Steve Lu, and Rafail Ostrovsky. Improving line-point zero knowledge: Two multiplications for the price of one. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 829–841, Los Angeles, CA, USA, November 7–11, 2022. ACM Press.
- [DIO21] Samuel Dittmer, Yuval Ishai, and Rafail Ostrovsky. Line-point zero knowledge and its applications. In Stefano Tessaro, editor, *ITC 2021*, volume 199 of *LIPIcs*, pages 5:1–5:24, Seattle, WA, USA, July 23–26, 2021. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik.
- [DKLs18] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. Secure two-party threshold ECDSA from ECDSA assumptions. In *2018 IEEE Symposium on Security and Privacy*, pages 980–997, San Francisco, CA, USA, May 21–23, 2018. IEEE Computer Society Press.
- [DPSZ12] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 643–662, Santa Barbara, CA, USA, August 19–23, 2012. Springer Berlin Heidelberg, Germany.

- [EPSW24] Daniel Escudero, Antigoni Polychroniadou, Yifan Song, and Chenkai Weng. Multi-verifier zero-knowledge proofs for any constant fraction of corrupted verifiers. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 4092–4106, Salt Lake City, UT, USA, October 14–18, 2024. ACM Press.
- [GGM84] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions (extended abstract). In *25th FOCS*, pages 464–479, Singer Island, Florida, October 24–26, 1984. IEEE Computer Society Press.
- [Gil99] Niv Gilboa. Two party RSA key generation. In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 116–129, Santa Barbara, CA, USA, August 15–19, 1999. Springer Berlin Heidelberg, Germany.
- [GKR<sup>+</sup>21] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schafneggger. Poseidon: A new hash function for zero-knowledge proof systems. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 519–535. USENIX Association, August 11–13, 2021.
- [GKS23] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schafneggger. Poseidon2: A faster version of the poseidon hash function. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *AFRICACRYPT 23*, volume 14064 of *LNCS*, pages 177–203, Sousse, Tunisia, July 19–21, 2023. Springer, Cham, Switzerland.
- [HSS20] Carmit Hazay, Peter Scholl, and Eduardo Soria-Vazquez. Low cost constant round MPC combining BMR and oblivious transfer. *Journal of Cryptology*, 33(4):1732–1786, October 2020.
- [JKO13] Marek Jawurek, Florian Kerschbaum, and Claudio Orlandi. Zero-knowledge using garbled circuits: how to prove non-algebraic statements efficiently. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 955–966, Berlin, Germany, November 4–8, 2013. ACM Press.
- [JKPT12] Abhishek Jain, Stephan Krenn, Krzysztof Pietrzak, and Aris Tentes. Commitments and efficient zero-knowledge proofs from learning parity with noise. In Xiaoyun Wang and Kazue Sako, editors, *ASIACRYPT 2012*, volume 7658 of *LNCS*, pages 663–680, Beijing, China, December 2–6, 2012. Springer Berlin Heidelberg, Germany.
- [Mer88] Ralph C. Merkle. A digital signature based on a conventional encryption function. In Carl Pomerance, editor, *CRYPTO’87*, volume 293 of *LNCS*, pages 369–378, Santa Barbara, CA, USA, August 16–20, 1988. Springer Berlin Heidelberg, Germany.
- [NNOB12] Jesper Buus Nielsen, Peter Sebastian Nordholt, Claudio Orlandi, and Sai Sheshank Burra. A new approach to practical active-secure two-party computation. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 681–700, Santa Barbara, CA, USA, August 19–23, 2012. Springer Berlin Heidelberg, Germany.
- [PVW08] Chris Peikert, Vinod Vaikuntanathan, and Brent Waters. A framework for efficient and composable oblivious transfer. In David Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 554–571, Santa Barbara, CA, USA, August 17–21, 2008. Springer Berlin Heidelberg, Germany.

- [RR] Peter Rindal and Lance Roy. libOTe: an efficient, portable, and easy to use Oblivious Transfer Library. <https://github.com/osu-crypto/libOTe>.
- [RR22a] Srinivasan Raghuraman and Peter Rindal. Blazing fast PSI from improved OKVS and subfield VOLE. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 2505–2517, Los Angeles, CA, USA, November 7–11, 2022. ACM Press.
- [RR22b] Srinivasan Raghuraman and Peter Rindal. VOLE-PSI. <https://github.com/Visa-Research/volepsi>, 2022.
- [RRT23] Srinivasan Raghuraman, Peter Rindal, and Titouan Tanguy. Expand-convolute codes for pseudorandom correlation generators from LPN. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 602–632, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
- [RS21] Peter Rindal and Phillipp Schoppmann. VOLE-PSI: Fast OPRF and circuit-PSI from vector-OLE. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 901–930, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
- [RS22] Rahul Rachuri and Peter Scholl. Le mans: Dynamic and fluid MPC for dishonest majority. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 719–749, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- [SGRR19] Phillipp Schoppmann, Adrià Gascón, Leonie Reichert, and Mariana Raykova. Distributed vector-OLE: Improved constructions and implementation. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 1055–1072, London, UK, November 11–15, 2019. ACM Press.
- [SKR<sup>+</sup>24] Yunqing Sun, Jonathan Katz, Mariana Raykova, Phillipp Schoppmann, and Xiao Wang. Actively secure private set intersection in the client-server setting. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 1478–1492, Salt Lake City, UT, USA, October 14–18, 2024. ACM Press.
- [WMK16] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. EMP-toolkit: Efficient Multi-Party computation toolkit. <https://github.com/emp-toolkit>, 2016.
- [WRK17] Xiao Wang, Samuel Ranellucci, and Jonathan Katz. Global-scale secure multiparty computation. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 39–56, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.
- [WYKW21] Chenkai Weng, Kang Yang, Jonathan Katz, and Xiao Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *2021 IEEE Symposium on Security and Privacy*, pages 1074–1091, San Francisco, CA, USA, May 24–27, 2021. IEEE Computer Society Press.
- [WYX<sup>+</sup>21] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. Mystique: Efficient conversions for zero-knowledge proofs with applications to machine learning. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 501–518. USENIX Association, August 11–13, 2021.



- [YSWW21] Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2986–3001, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press.
- [YWL<sup>+</sup>20] Kang Yang, Chenkai Weng, Xiao Lan, Jiang Zhang, and Xiao Wang. Ferret: Fast extension for correlated OT with small communication. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1607–1626, Virtual Event, USA, November 9–13, 2020. ACM Press.

## A Appendix

### A.1 Hybrid Commitment Parameters based on EA-Code

Boyle et al. [BCG<sup>+</sup>22] introduced a method of choosing conservative parameters for EA-code, which is secure against the current known attacks [RRT23]. They use a random walk on a Markov chain to determine the minimum distance for the EA code. As all variants of LPN generalize naturally to larger fields [BCG<sup>+</sup>22], we instantiate our commitment of  $\mathbb{F}_{2^{128}}$  based on the conservative parameters constraints of  $\mathbb{F}_2$  as follows.

#### A.1.1 Conservative parameters

Based on [BCG<sup>+</sup>22, Theorem 3.10], for  $\mathbf{H} \leftarrow \text{EAGen}(n, N, \frac{C \ln N}{N})$ ,  $\mathbf{H}$  achieves a minimum distance of  $\delta N$  with a probability of at least

$$\eta(\kappa) \geq 1 - 2RN^{-2\beta^2 C+2} = 1 - 2nN^{-2(1/2-\delta)^2 C+1} . \quad (1)$$

For an error distribution with an expected weight of  $t$ , we have

$$\varepsilon_d \leq (1 - 2t/N)^{\delta N} / 2 \leq e^{-2t\delta} . \quad (2)$$

We consider two scenarios: a failure probability of at most 0.01 (i.e.,  $\eta \geq 0.99$ ). Our chosen failure probability falls within the range of [0.0000148, 0.174], as indicated in the initial work [BCG<sup>+</sup>22, Figure 8], and is deemed reasonable. To achieve 128-bit security against linear tests, we have  $-\log(\varepsilon_d) = 128 - \log(N)$  because computing a dot-product requires  $N$  operations [BCG<sup>+</sup>22]. We first choose an appropriate value for  $t$  and then determine  $\delta$  using Equation (2), which is  $\delta \geq \frac{128 - \log(N)}{2 \log(e) \cdot t}$ . Next, to ensure  $\eta \geq 0.99$ , we use Equation (1) and choose  $C$  such that

$$\log(2n) + \log(N) \cdot (-2(1/2 - \delta)^2 C + 1) \leq \log(0.01) .$$

Finally, we check the parameters to satisfy the following conditions from [BCG<sup>+</sup>22, Theorem 3.10]:

$$R < \min \left\{ \frac{2}{\ln 2} \cdot \frac{1 - e^{-1}}{1 + e^{-1}} \cdot \beta^2, \frac{2}{e} \right\} \quad \text{and} \quad C > \frac{1}{\beta^2} .$$

shown in Theorem [BCG<sup>+</sup>22, Theorem 3.10], where  $\beta = 1/2 - \delta$ .

### A.1.2 Aggressive parameters

[BCG<sup>+</sup>22] proposed aggressive parameters by empirically estimating the average minimum row-weight of  $\mathbf{H} \leftarrow \text{EAGen}(n, N, \ell)$  using exponential regression in  $\log(n)$ , with  $N = 5n$  and  $\ell \in \{7, 9, 11\}$ . Specifically,  $\mathbf{H}$  achieves a minimum distance of  $\delta N$ , where

$$\delta \approx \begin{cases} 0.4646 \cdot e^{-0.1012 \cdot \log(n)}, & \text{if } \ell = 7 \\ 0.4415 \cdot e^{-0.0783 \cdot \log(n)}, & \text{if } \ell = 9 \\ 0.4337 \cdot e^{-0.06383 \cdot \log(n)}, & \text{if } \ell = 11. \end{cases}$$

Then, we determine  $t$  using Equation (2), which is  $t \geq \frac{128 - \log(N)}{2 \log(e) \cdot \delta}$ .

## A.2 Primal-LPN based C-VOLE

We give primal-LPN based C-VOLE construction in this section. It follows the same idea in Section 5, except minor changes regarding the format of primal-LPN commitment of vector  $\mathbf{e}_u$ . As we introduced, generating any C-VOLE of designated vector  $\mathbf{x}$  is start with generating C-VOLE of a designated regular- $w$  vector  $\mathbf{e}_u$ . As  $\mathbf{e}_u$  is committed as  $\text{com}_{\mathbf{e}_u} = \mathbf{e}_u \cdot \mathbf{A}_1 + \mathbf{r} \cdot \mathbf{A}_2 + \mathbf{e}_r$  and  $\text{decom}_{\mathbf{e}_u} = \{\mathbf{r}, \mathbf{e}_r\}$  under primal-LPN commitment in Section 4.1, we generate consistent  $[\![\mathbf{e}_u]\!]$  as follows: 1) generate  $[\![\mathbf{e}_u]\!]$  with  $\text{P}_A$  input  $\mathbf{e}_u$ , 2) generate  $[\![\mathbf{e}_r]\!]$  with  $\text{P}_A$  input  $\mathbf{e}_r$ , 3) generate  $[\![\mathbf{r}]\!]$  with  $\text{P}_A$  input  $\mathbf{r}$ , 4)  $\text{P}_A$  computes  $H_3(\text{M}[\text{com}_{\mathbf{e}_u}])$  and sends it to  $\text{P}_B$  for consistency check. The consistency check works as follows:

$$\begin{aligned} \text{M}[\text{com}_{\mathbf{e}_u}] &= \text{M}[\mathbf{e}_u] \cdot \mathbf{A}_1 + \text{M}[\mathbf{r}] \cdot \mathbf{A}_2 + \text{M}[\mathbf{e}_r] \\ \text{K}[\text{com}_{\mathbf{e}_u}] &= \text{K}[\mathbf{e}_u] \cdot \mathbf{A}_1 + \text{K}[\mathbf{r}] \cdot \mathbf{A}_2 + \text{K}[\mathbf{e}_r] \end{aligned}$$

If  $[\![\mathbf{e}_u]\!]$ ,  $[\![\mathbf{e}_r]\!]$ , and  $[\![\mathbf{r}]\!]$  are generated with  $\text{P}_A$  input consistent  $\mathbf{e}_u$ ,  $\mathbf{e}_r$ , and  $\mathbf{r}$  used in  $\text{com}_{\mathbf{e}_u}$ ,  $[\![\text{com}_{\mathbf{e}_u}]\!]$  should hold and be checked successfully by  $\text{P}_B$  that  $H_3(\text{M}[\text{com}_{\mathbf{e}_u}]) = H_3(\text{K}[\text{com}_{\mathbf{e}_u}] + \text{com}_{\mathbf{e}_u} \cdot \Delta)$ .

Then, we can recover  $[\![\mathbf{x}]\!]$  from  $[\![\mathbf{e}_u]\!]$  under  $\text{Hcom}_{\mathbf{x}}$  as easily as dual-LPN-based C-VOLE does in Section 5. The detailed protocol is shown in Figure 8. We also prove that the protocol is secure against malicious adversaries in Theorem 5.

**Theorem 5.** *Protocol  $\Pi_{\text{C-VOLE-primal}}$  in Figure 8 securely instantiated  $\mathcal{F}_{\text{C-VOLE}}$  in Figure 1 in  $(\mathcal{F}_{\text{MPVOLE}}, \mathcal{F}_{\text{VOLE}})$ -hybrid model against any malicious adversaries.*

*Proof.* Let  $\mathcal{A}$  be a PPT adversary that corrupt  $\text{P}_A$  or  $\text{P}_B$ . We construct a PPT simulator  $\mathcal{S}$  with access to  $\mathcal{F}_{\text{C-VOLE}}$  and simulates the adversary's view. We will prove that the joint distribution over the output of  $\mathcal{A}$  and the honest party in the real world is indistinguishable from the joint distribution over the output of  $\mathcal{S}$  and the honest party in the ideal world.

**Corrupted  $\text{P}_A$ .** Let  $\mathcal{S}$  access  $\mathcal{F}_{\text{C-VOLE}}$  as an honest  $\text{P}_A$  and interact with  $\mathcal{A}$  as an honest  $\text{P}_B$ .  $\mathcal{S}$  passes all communication between  $\mathcal{A}$  and environment  $\mathcal{Z}$ .

1.  $\mathcal{S}$  holds public commitment of  $\mathbf{x}$ , that  $\text{Hcom}_{\mathbf{x}} = (\text{com}_{\mathbf{e}_u}, \mathbf{x}^*)$ .  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{MPVOLE}}$  and receives (init) from  $\mathcal{A}$ .
2.  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{MPVOLE}}$  and receives (extend,  $\mathbf{e}'_u \in \mathbb{F}^{w \cdot 2^L}, L, w$ ) from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $\text{M}[\mathbf{e}'_u] \in \mathbb{F}^{w \cdot 2^L}$  from  $\mathcal{A}$ .  $\mathcal{S}$  computes  $\mathbf{x}' = \mathbf{e}_u \cdot \mathbf{H} - \mathbf{x}^*$ .
3.  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{MPVOLE}}$  and receives (extend,  $\mathbf{e}'_r \in \mathbb{F}^{t \cdot 2^{L'}}, L', t$ ) from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $\text{M}[\mathbf{e}'_r] \in \mathbb{F}^{t \cdot 2^{L'}}$  from  $\mathcal{A}$ .

**Protocol  $\Pi_{C-VOLE-primal}$**

**Parameters:** A public hybrid commitment of  $\mathbf{x} \in \mathbb{F}^m$  defined in Section 4.2, that  $\text{Hcom}_{\mathbf{x}} = (\text{com}_{e_u} \in \mathbb{F}^N, \mathbf{x}^* = e_u \cdot \mathbf{H} - \mathbf{x} \in \mathbb{F}^m)$ . A primal LPN commitment scheme defined in Section 4.1 that  $(\text{com}_{e_u} = e_u \cdot \mathbf{A}_1 + \mathbf{r} \cdot \mathbf{A}_2 + e_r, \text{decom}_{e_u} = \{\mathbf{r}, e_r\}) \leftarrow \text{Com}(e_u)$ , where  $e_u \in \mathbb{F}^{w \cdot 2^L}$ ,  $\mathbf{r} \in \mathbb{F}^k$ ,  $e_r \in \mathbb{F}^{t \cdot 2^{L'}}$ ,  $N = t \cdot 2^{L'}$ . A Collision-Resistant Hash Function  $H_3$ .

**Inputs:**  $P_A$  holds vector  $\mathbf{x} \in \mathbb{F}^m$ ,  $\text{Hdecom}_{\mathbf{x}} = (e_u \in \mathbb{F}^{w \cdot 2^L}, \mathbf{r} \in \mathbb{F}^k, e_r \in \mathbb{F}^{t \cdot 2^{L'}})$ .

**Compute:**

1. Both  $P_A$  and  $P_B$  call  $\mathcal{F}_{\text{MPVOLE}}$  with input (init).  $P_B$  receives  $\Delta \in \mathbb{F}$  from  $\mathcal{F}_{\text{MPVOLE}}$ .
2.  $P_A$  calls  $\mathcal{F}_{\text{MPVOLE}}$  with input (extend,  $e_u \in \mathbb{F}^{w \cdot 2^L}, L, w$ ).  $P_B$  calls  $\mathcal{F}_{\text{MPVOLE}}$  with input (extend,  $L, w$ ). The functionality returns  $M[e_u] \in \mathbb{F}^{w \cdot 2^L}$  to  $P_A$  and  $K[e_u] \in \mathbb{F}^{w \cdot 2^L}$  to  $P_B$  such that  $M[e_u] = K[e_u] + \Delta \cdot e_u \in \mathbb{F}^{w \cdot 2^L}$ . If either party receives **abort** from  $\mathcal{F}_{\text{MPVOLE}}$  in any of these executions, it aborts.
3.  $P_A$  calls  $\mathcal{F}_{\text{MPVOLE}}$  with input (extend,  $e_r \in \mathbb{F}^{t \cdot 2^{L'}}, L', t$ ).  $P_B$  calls  $\mathcal{F}_{\text{MPVOLE}}$  with input (extend,  $L', t$ ). The functionality returns  $M[e_r] \in \mathbb{F}^{t \cdot 2^{L'}}$  to  $P_A$  and  $K[e_r] \in \mathbb{F}^{t \cdot 2^{L'}}$  to  $P_B$  such that  $M[e_r] = K[e_r] + \Delta \cdot e_r \in \mathbb{F}^{t \cdot 2^{L'}}$ . If either party receives **abort** from  $\mathcal{F}_{\text{MPVOLE}}$  in any of these executions, it aborts.
4.  $P_A$  calls  $\mathcal{F}_{\text{VOLE}}$  with input (extend,  $\mathbf{r} \in \mathbb{F}^k, k$ );  $P_B$  calls  $\mathcal{F}_{\text{VOLE}}$  with input (extend,  $k$ ). The functionality returns  $M[\mathbf{r}] \in \mathbb{F}^k$  to  $P_A$  and  $K[\mathbf{r}] \in \mathbb{F}^k$  to  $P_B$ , such that  $M[\mathbf{r}] = K[\mathbf{r}] + \mathbf{r} \cdot \Delta$ . If either party receives **abort** from  $\mathcal{F}_{\text{VOLE}}$  in any of these executions, it aborts.
5.  $P_A$  computes  $M[\text{com}_{e_u}] = M[e_u] \cdot \mathbf{A}_1 + M[\mathbf{r}] \cdot \mathbf{A}_2 + M[e_r]$  and sends  $H_3(M[\text{com}_{e_u}])$  to  $P_B$ ;  $P_B$  computes  $K[\text{com}_{e_u}] = K[e_u] \cdot \mathbf{A}_1 + K[\mathbf{r}] \cdot \mathbf{A}_2 + K[e_r]$  and **value**  $= K[\text{com}_{e_u}] + \text{com}_{e_u} \cdot \Delta \in \mathbb{F}^N$ .  $P_B$  checks whether  $H_3(M[\text{com}_{e_u}]) = H_3(\text{value})$ . If not,  $P_B$  aborts.
6.  $P_A$  outputs  $(\mathbf{x}, M[\mathbf{x}])$ , where  $M[\mathbf{x}] = M[e_u] \cdot \mathbf{H} \in \mathbb{F}^m$ .  $P_B$  outputs  $(K[\mathbf{x}], \Delta)$ , where  $K[\mathbf{x}] = K[e_u] \cdot \mathbf{H} + \mathbf{x}^* \Delta \in \mathbb{F}^m$ .

Figure 8: The committed-VOLE protocol based on primal-LPN-based hybrid commitment

4.  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{VOLE}}$  and receives  $(\text{extend}, \mathbf{r}' \in \mathbb{F}^k, k)$  from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $\mathbf{M}[\mathbf{r}'] \in \mathbb{F}^k$  to  $\mathcal{A}$ .  $\mathcal{S}$  sends  $(\mathbf{x}', \text{decom}_{\mathbf{x}'} = (\mathbf{e}'_u, \mathbf{r}', \mathbf{e}'_r))$  to  $\mathcal{F}_{\text{C-VOLE}}$ . If  $\mathcal{S}$  receives **abort** from  $\mathcal{F}_{\text{C-VOLE}}$ ,  $\mathcal{S}$  aborts at step (5).
5.  $\mathcal{S}$  receives  $H_3(\mathbf{M}[\text{com}_{\mathbf{e}_u}])'$  from  $\mathcal{A}$ .  $\mathcal{S}$  checks if  $H_3(\mathbf{M}[\text{com}_{\mathbf{e}_u}])' = H_3(\mathbf{M}[\mathbf{e}'_u] \cdot \mathbf{A}_1 + \mathbf{M}[\mathbf{r}'] \cdot \mathbf{A}_2 + \mathbf{M}[\mathbf{e}'_r])$ . If not,  $\mathcal{S}$  aborts.
6.  $\mathcal{S}$  computes  $\mathbf{M}[\mathbf{x}'] = \mathbf{M}[\mathbf{e}'_u] \cdot \mathbf{H}$  and sends  $\mathbf{M}[\mathbf{x}']$  to  $\mathcal{F}_{\text{C-VOLE}}$ .  $\mathcal{S}$  outputs what  $\mathcal{A}$  outputs.

We are going to show the simulated execution is indistinguishable from real-world protocol execution.

**Hybrid  $\mathcal{H}_0$**  Same as real-world execution in  $\mathcal{F}_{\text{MPVOLE}}, \mathcal{F}_{\text{VOLE}}$  model.

**Hybrid  $\mathcal{H}_1$**  Same as Hybrid  $\mathcal{H}_0$  except  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{MPVOLE}}$  and  $\mathcal{F}_{\text{VOLE}}$ . Notice that  $\mathcal{S}$  does not simulate any message to  $\mathcal{A}$ .

**Hybrid  $\mathcal{H}_2$**  Same as Hybrid  $\mathcal{H}_1$  except  $\mathcal{S}$  aborts at step (5) if  $\mathcal{S}$  receives **abort** from  $\mathcal{F}_{\text{C-VOLE}}$  or the received  $H_3(\mathbf{M}[\text{com}_{\mathbf{e}_u}])' \neq H_3(\mathbf{M}[\mathbf{e}'_u] \cdot \mathbf{A}_1 + \mathbf{M}[\mathbf{r}'] \cdot \mathbf{A}_2 + \mathbf{M}[\mathbf{e}'_r])$ .

In hybrid  $\mathcal{H}_1$ , an honest  $\mathbf{P}_B$  aborts at step (5) if the received  $H_3(\mathbf{M}[\text{com}_{\mathbf{e}_u}])' \neq H_3(\mathbf{K}[\mathbf{e}'_u] \cdot \mathbf{A}_1 + \mathbf{K}[\mathbf{r}'] \cdot \mathbf{A}_2 + \mathbf{K}[\mathbf{e}'_r] + \text{com}_{\mathbf{e}_u} \cdot \Delta)$ . There are the following abort conditions with abort probability indistinguishable from hybrid  $\mathcal{H}_2$ :

1)  $\mathcal{A}$  uses  $\mathbf{e}'_u \neq \mathbf{e}_u$ ,  $\mathbf{e}'_r \neq \mathbf{e}_r$ , and  $\mathbf{r}' \neq \mathbf{r}$  to  $\mathcal{F}_{\text{MPVOLE}}$ .  $\mathcal{A}$  computes  $\mathbf{M}[\text{com}_{\mathbf{e}_u}]'$  honestly from  $\mathbf{M}[\mathbf{e}'_u] \cdot \mathbf{A}_1 + \mathbf{M}[\mathbf{r}'] \cdot \mathbf{A}_2 + \mathbf{M}[\mathbf{e}'_r]$ . Any  $(\mathbf{e}'_u, \mathbf{e}'_r, \mathbf{r}')$  inconsistent with  $(\mathbf{e}_u, \mathbf{e}_r, \mathbf{r})$  will result the inequality above with the probability of statistical binding failure mentioned in Theorem 1. In this hybrid, any inconsistency will be checked by the  $\mathcal{F}_{\text{C-VOLE}}$  with the same binding failure and result in an abort at step (5), which is indistinguishable from hybrid  $\mathcal{H}_1$ . 2)  $\mathcal{A}$  uses  $\mathbf{e}'_u \neq \mathbf{e}_u$ ,  $\mathbf{e}'_r \neq \mathbf{e}_r$ , and  $\mathbf{r}' \neq \mathbf{r}$  to  $\mathcal{F}_{\text{MPVOLE}}$ .  $\mathcal{A}$  samples  $\mathbf{M}[\text{com}_{\mathbf{e}_u}]'$  to  $\mathcal{S}$ . The consistency check passes if and only if  $\mathcal{A}$  samples  $\mathbf{M}[\text{com}_{\mathbf{e}_u}]' = \mathbf{K}[\mathbf{e}'_u] \cdot \mathbf{A}_1 + \mathbf{K}[\mathbf{r}'] \cdot \mathbf{A}_2 + \mathbf{K}[\mathbf{e}'_r] + \text{com}_{\mathbf{e}_u} \cdot \Delta$ . However, since  $\mathbf{K}[\mathbf{e}'_u]$ ,  $\mathbf{K}[\mathbf{e}'_r]$ ,  $\mathbf{K}[\mathbf{r}]$ , and  $\Delta$  are uniformly distributed over  $\mathbb{F}^{w \cdot 2^L}$ ,  $\mathbb{F}^{t \cdot 2^{L'}}$ ,  $\mathbb{F}^k$  and  $\mathbb{F}$ , respectively, the consistency check fails with all but negligible probability. In this hybrid, given inconsistent  $(\mathbf{e}'_u, \mathbf{e}'_r, \mathbf{r}')$  pair,  $\mathcal{S}$  aborts except the failure probability of binding, which is also negligible and thus indistinguishable from hybrid  $\mathcal{H}_1$ . 3)  $\mathcal{A}$  uses  $\mathbf{e}'_u = \mathbf{e}_u$ ,  $\mathbf{e}'_r = \mathbf{e}_r$ , and  $\mathbf{r}' = \text{vecr}$  to  $\mathcal{F}_{\text{MPVOLE}}$ .  $\mathcal{A}$  samples  $\mathbf{M}[\text{com}_{\mathbf{e}_u}]'$  to  $\mathcal{S}$ . The consistency check passes if and only if  $\mathcal{A}$  samples  $\mathbf{M}[\text{com}_{\mathbf{e}_u}]' = \mathbf{K}[\mathbf{e}'_u] \cdot \mathbf{A}_1 + \mathbf{K}[\mathbf{r}'] \cdot \mathbf{A}_2 + \mathbf{K}[\mathbf{e}'_r] + \text{com}_{\mathbf{e}_u} \cdot \Delta$ , which is equivalent to checking  $\mathbf{M}[\text{com}_{\mathbf{e}_u}]' = \mathbf{M}[\mathbf{e}'_u] \cdot \mathbf{A}_1 + \mathbf{M}[\mathbf{r}'] \cdot \mathbf{A}_2 + \mathbf{M}[\mathbf{e}'_r] + \text{com}_{\mathbf{e}_u} \cdot \Delta$  in this hybrid. Thus, the abort probability is indistinguishable from hybrid  $\mathcal{H}_1$ .

Therefore, this hybrid is identically distributed as  $\mathcal{H}_1$ .

**Corrupted  $\mathbf{P}_B$ .** Let  $\mathcal{S}$  access  $\mathcal{F}_{\text{C-VOLE}}$  as an honest  $\mathbf{P}_B$  and interact with  $\mathcal{A}$  as an honest  $\mathbf{P}_A$ .  $\mathcal{S}$  passes all communication between  $\mathcal{A}$  and environment  $\mathcal{Z}$ .

1.  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{MPVOLE}}$  and receives (init) from  $\mathcal{A}$ .  $\mathcal{S}$  holds public commitment of  $\mathbf{x}$ ,  $\mathbf{Hcom}_{\mathbf{x}} = (\text{com}_{\mathbf{e}_u}, \mathbf{x}^* = \mathbf{e}_u \cdot \mathbf{H} - \mathbf{x})$ .  $\mathcal{S}$  sends (init) to  $\mathcal{F}_{\text{C-VOLE}}$ .  $\mathcal{S}$  receives  $\Delta \in \mathbb{F}$  from  $\mathcal{A}$ .
2.  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{MPVOLE}}$  and receives (extend,  $L, w$ ) from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $\mathbf{K}[\mathbf{e}_u] \in \mathbb{F}^{w \cdot 2^L}$  from  $\mathcal{A}$ .
3.  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{MPVOLE}}$  and receives (extend,  $L', t$ ) from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $\mathbf{K}[\mathbf{e}_r] \in \mathbb{F}^{t \cdot 2^{L'}}$  from  $\mathcal{A}$ .
4.  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{VOLE}}$  and receives (extend,  $k$ ) from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $\mathbf{K}[\mathbf{r}] \in \mathbb{F}^k$  from  $\mathcal{A}$ .
5.  $\mathcal{S}$  computes  $H_3(\mathbf{M}[\text{com}_{\mathbf{e}_u}]) = H_3(\mathbf{K}[\mathbf{e}_u] \cdot \mathbf{A}_1 + \mathbf{K}[\mathbf{r}] \cdot \mathbf{A}_2 + \mathbf{K}[\mathbf{e}_r] + \text{com}_{\mathbf{e}_u} \cdot \Delta)$  and sends it to  $\mathcal{A}$ .  $\mathcal{S}$  aborts if  $\mathcal{A}$  aborts.
6.  $\mathcal{S}$  computes  $\mathbf{K}[\mathbf{x}] = \mathbf{K}[\mathbf{e}_u] \cdot \mathbf{H} + \mathbf{x}^* \cdot \Delta \in \mathbb{F}^m$  and sends  $(\Delta, \mathbf{K}[\mathbf{x}]) \in \mathbb{F}^{m+1}$  to  $\mathcal{F}_{\text{C-VOLE}}$ .  $\mathcal{S}$  outputs what  $\mathcal{A}$  outputs.

| Input          | Random sparse vector |     |     | Primal LPN commitment |     |      |                   |                               |
|----------------|----------------------|-----|-----|-----------------------|-----|------|-------------------|-------------------------------|
| $ \mathbf{x} $ | $ \mathbf{e}_u $     | $w$ | $L$ | $ \mathbf{e}_r $      | $t$ | $L'$ | $ \mathbf{r} (k)$ | $ \text{com}_{\mathbf{e}_u} $ |
| $2^{20}$       | $7 * 2^{20}$         | 224 | 15  | 1272                  | 159 | 3    | 433               | 1272                          |
| $2^{24}$       | $7 * 2^{24}$         | 224 | 19  | 1296                  | 162 | 3    | 434               | 1296                          |
| $2^{28}$       | $7 * 2^{28}$         | 224 | 23  | 1296                  | 162 | 3    | 434               | 1296                          |

Table 8: Primal-LPN-based hybrid commitment parameters selection of  $\mathbf{x}$  with different lengths. Achieves 128-bit security.

We are going to show the simulated execution is indistinguishable from real-world protocol execution.

**Hybrid  $\mathcal{H}_0$**  Same as real-world execution in  $\mathcal{F}_{\text{MPVole}}, \mathcal{F}_{\text{Vole}}$  model.

**Hybrid  $\mathcal{H}_1$**  Same as Hybrid  $\mathcal{H}_0$  except  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{MPVole}}$  and  $\mathcal{F}_{\text{Vole}}$ , and simulate messages to  $\mathcal{A}$ .

$\mathcal{S}$  simulates  $H_3(\text{M}[\text{com}_{\mathbf{e}_u}]) = H_3(\text{K}[\mathbf{e}_u] \cdot \mathbf{A}_1 + \text{K}[\mathbf{r}] \cdot \mathbf{A}_2 + \text{K}[\mathbf{e}_r] + \text{com}_{\mathbf{e}_u} \cdot \Delta)$  and sends it to  $\mathcal{A}$ . In the previous hybrid, an honest  $\text{P}_A$  computes  $\text{M}[\text{com}_{\mathbf{e}_u}] = \text{K}[\mathbf{e}_u] \cdot \mathbf{A}_1 + \text{K}[\mathbf{r}] \cdot \mathbf{A}_2 + \text{K}[\mathbf{e}_r] + \text{com}_{\mathbf{e}_u} \cdot \Delta$ , which holds the property that  $\text{M}[\mathbf{e}_u] = \text{K}[\mathbf{e}_u] + \mathbf{e}_u \cdot \Delta$ ,  $\text{M}[\mathbf{e}_r] = \text{K}[\mathbf{e}_r] + \mathbf{e}_r \cdot \Delta$ , and  $\text{M}[\mathbf{r}] = \text{K}[\mathbf{r}] + \mathbf{r} \cdot \Delta$ . Consequently, the  $\text{M}[\text{com}_{\mathbf{e}_u}]$  computed by simulator equals the one computed by honest  $\text{P}_A$  in real world. Thus, this hybrid is identical to the previous one.

This concludes the proof. □

### A.3 Performance of Primal-LPN based C-VOLE

**Complexity Analysis.** The primal-LPN-based C-VOLE protocol requires MPVole instantiations of  $\mathbf{e}_u \in \mathbb{F}^{w \cdot 2^L}$  and  $\mathbf{e}_r \in \mathbb{F}^{t \cdot 2^{L'}}$ , Vole instantiation of  $\mathbf{r} \in \mathbb{F}^k$ , together with a  $H(\text{M}[\text{com}_{\mathbf{e}_u}])$  for consistency check and a commitment with a size of  $N + m$ , where  $N = t \cdot 2^{L'}$ . According to the instantiation of MPVole protocol in Section A.5, the sparse vector of length  $w \cdot 2^L$  requires communication overhead of  $wL\kappa$ . For instantiation of Vole protocol in Section A.9, it is dominated by  $k \log |\mathbb{F}|$  for input vector with length  $k$ . Thus, the communication complexity is  $(wL + tL')\kappa + (k + N + m) \log |\mathbb{F}|$  in total.

**Parameter Selection of Hybrid Commitment.** Same as the dual-LPN-based hybrid commitment parameter selection, we use EA code to choose the  $(w, L)$  pair of  $\mathbf{e}_u \in \mathbb{F}^{w \cdot 2^L}$  for  $\mathbf{x} \in \mathbb{F}^m$  with 128-bit security level. Given the  $(w, L)$  pair, we fix the structure of  $\mathbf{e}_u$  and select parameters to construct a 128-bit secure commitment  $\text{com}_{\mathbf{e}_u}$ , where  $\text{com}_{\mathbf{e}_u} = \mathbf{e}_u \cdot \mathbf{A}_1 + \mathbf{r} \cdot \mathbf{A}_2 + \mathbf{e}_r$ . According to Theorem 1, we select the  $(k, t, L')$  parameters for the primal LPN commitment. For each  $\mathbf{x} \in \mathbb{F}^m$  of different lengths, groups of parameters  $(w, L, k, t, L')$  are computed. Based on the complexity analysis above, we minimize the communication cost for primal-LPN-based C-Vole protocol and present the commitment parameters that achieve 128 bit statistically binding and computational hiding for  $\mathbf{x} \in \{\mathbb{F}^{2^{20}}, \mathbb{F}^{2^{24}}, \mathbb{F}^{2^{28}}\}$  in Table 8.

**Performance Evaluation.** The performance of primal-LPN-based C-Vole also consists of two parts: computing the one-time commitment of sender's input vector  $\mathbf{x}$  and a online computation with any individual receiver. The time consumption and communication overhead of these two phases are shown in Figure 9a and Figure 9b respectively.

For one-time commitment, the sender generates a primal-LPN-based hybrid commitment of input vector  $\mathbf{x} \in \mathbb{F}^m$  and publishes it to all other parties. The computation cost for the sender is 0.59 s for the computation of a vector of length  $m = 2^{20}$  and 340.46 s for  $m = 2^{28}$  using 4 threads,

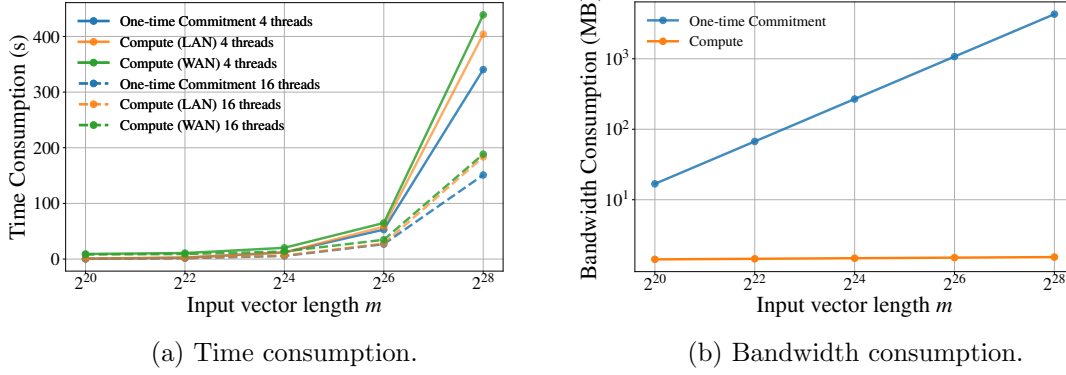


Figure 9: **Performance of the C-VOLE protocol based on the primal-LPN assumption in LAN and WAN settings.**

| Input          | Intermediate sparse vector |     |     | Dual LPN commitment |     |      |                                  |
|----------------|----------------------------|-----|-----|---------------------|-----|------|----------------------------------|
| $ \mathbf{x} $ | $ \mathbf{e}_u $           | $w$ | $L$ | $ \mathbf{e}_r $    | $t$ | $L'$ | $ \text{com}_{\mathbf{e}_u} (n)$ |
| $2^{20}$       | $5 * 2^{20}$               | 640 | 13  | 1640                | 165 | 4    | 882                              |
| $2^{24}$       | $5 * 2^{24}$               | 640 | 17  | 2688                | 168 | 4    | 885                              |
| $2^{28}$       | $5 * 2^{28}$               | 640 | 21  | 2736                | 171 | 4    | 888                              |

Table 9: Dual-LPN-based hybrid commitment parameters selection of  $\mathbf{x}$  with different lengths. Using aggressive parameters for EA-code. Commitment achieves 128-bit security level.

while it takes 0.30 s for  $m = 2^{20}$  and 151.06 s for  $m = 2^{28}$  using 16 threads. The communication overhead of publishing the primal-LPN-based hybrid commitment is 16.80 MB for  $m = 2^{20}$  and 4295.0 MB for  $m = 2^{28}$ . For  $m = 2^{20}$ , the compute phase takes 0.83 s under a LAN network and 9.19 s under a WAN network with a bandwidth of 50 Mbps when using 4 threads. With 16 threads, the compute phase takes 0.61 seconds under LAN network and 8.13 seconds under WAN network. For  $m = 2^{28}$ , the compute phase with 4 threads takes 404.11 s under a LAN network and 435.81 s under a WAN network. With 16 threads, the corresponding times are 183.47 s under a LAN network and 188.56 s under a WAN network. Regarding the online communication overhead, the cost is 1.73 Mb for  $m = 2^{20}$  and 1.79 MB for  $m = 2^{28}$ .

**Comparison with Dual-LPN-based C-VOLE.** As we can see from the performance evaluation of dual-LPN-based C-VOLE and primal-LPN-based C-VOLE, both schemes perform well with the chosen hybrid commitment parameters. Considering the cumulative effect of both communication and computation overhead, the scheme based on primal-LPN shows slightly better concrete performance than the one based on dual-LPN in our experiments.

#### A.4 Performance Evaluation of C-VOLE with Aggressive EA-code

According to Section A.1.2, we also evaluate the performance of our C-VOLE protocol over field  $\mathbb{F}_2^{128}$  using an aggressive EA-code, which remains secure against attacks [RRT23] applicable only to  $\mathbb{F}_2$ . We took  $\ell = 9$  and compute the hamming weight  $t = 640$ . Taking dual-LPN commitment as an example, Table 9 presents the parameters for commitments that achieve 128 bit security level.

**Performance Evaluation.** The performance of dual-LPN-based C-VOLE using aggressive EA-code also shown in two parts: computing the one-time commitment of sender’s input vector  $\mathbf{x}$  and a online computation with any individual receiver. The time consumption and communication

overhead of these two phases are shown in Figure 10a and Figure 10b respectively.

For one-time commitment, the computation cost for the sender is 0.07 s for the computation of a vector of length  $m = 2^{20}$  and 36.08 s for  $m = 2^{28}$  using 4 threads, while it takes 0.07 s for  $m = 2^{20}$  and 20.33 s for  $m = 2^{28}$  using 16 threads. The communication overhead of publishing the commitment is 16.79 MB for  $m = 2^{20}$  and 4294.98 MB for  $m = 2^{28}$ . For  $m = 2^{20}$ , the compute phase takes 0.36 s under a LAN network and 9.09 s under a WAN network with a bandwidth of 50 Mbps when using 4 threads. With 16 threads, the compute phase takes 0.26 seconds under LAN network and 7.84 seconds under WAN network. For  $m = 2^{28}$ , the compute phase with 4 threads takes 41.16 s under a LAN network and 52.79 s under a WAN network. With 16 threads, the corresponding times are 27.84 s under a LAN network and 33.89 s under a WAN network. Regarding the online communication overhead, the cost is 1.99 Mb for  $m = 2^{20}$  and 2.16 MB for  $m = 2^{28}$ .

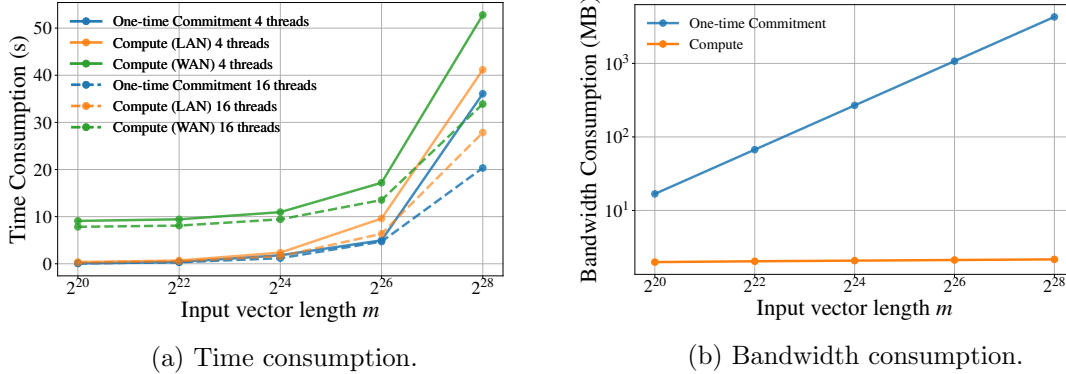


Figure 10: **Performance of the C-VOLE protocol based on the dual-LPN assumption using aggressive EA-code in LAN and WAN settings.**

**Comparison with Dual-LPN-based C-VOLE with conservative parameters.** As we can see from the performance evaluation, the aggressive parameters result in significantly more efficient time consumption, which is roughly 10 times better for  $m = 2^{28}$ , although this comes at the cost of slightly worse online communication.

## A.5 Construction of MPVOLE

Here, we briefly introduce the construction of protocol  $\Pi_{\text{MPVOLE}}$  in Figure 11, which securely instantiates the  $\mathcal{F}_{\text{MPVOLE}}$  functionality in Figure 3. This protocol is a slightly variation of the single-point subfield-VOLE protocol from [WYKW21], where the consistency check is slightly adjust to suit the VOLE protocol. A brief description of the protocol is as follows:

The  $\Pi_{\text{MPVOLE}}$  generates VOLE relation of sparse multi-point vector  $\mathbf{e}_u \in \mathbb{F}^{t \cdot 2^h}$ ,  $n = 2^h$ , which satisfies a specific form that it is a concatenation of  $t$  single-point vectors  $\mathbf{e}_u^i, i \in [t]$ , each with length  $n$ . We let  $\alpha_i$  to represent the index of non-zero element that  $\alpha_i \in [n]$  and  $\beta_i$  to denote the value of the non-zero element in block  $\mathbf{e}_u^i, i \in [t]$ . This protocol first utilize  $\mathcal{F}_{\text{VOLE}}$  over field  $\mathbb{F}$  to generate VOLE of  $\beta_i, i \in [t]$ , as selected by  $\mathbf{P}_A$ . Then, for each single-point block  $\mathbf{e}_u^i, i \in [t]$ , this protocol follows the design of [YWL<sup>+</sup>20]. It first let  $\mathbf{P}_B$  run GGM algorithm [GGM84], which is an algorithm building pseudorandom function by using a PRG, detailed in Section A.7, to obtain pseudorandom output  $\mathbf{v}_i$ ,  $\mathbf{P}_A$  run GGM' algorithm, also detailed in Section A.7, to obtain  $\mathbf{w}_i, i \neq \alpha_i$ . Then  $\mathbf{P}_B$  sends  $d_i$  to help  $\mathbf{P}_A$  recover  $\mathbf{w}_i$ . For consistency check, this protocol first generates a random value  $r_0 \leftarrow \mathbb{F}$  and its VOLE relation  $\mathbf{M}[r_0] = \mathbf{K}[r_0] + r_0 \cdot \Delta$ . With sampled  $\chi_i[j], i \in [t], j \in [n]$ ,



$P_A$  computes linear combination of  $w_i[j], i \in [t], j \in [n]$  and  $P_B$  computes linear combination of  $v_i[j], i \in [t], j \in [n]$ .  $P_A$  uses  $r_0$  to mask the sum of  $\beta_i \cdot \chi_i[\alpha_i], i \in [t]$  and sends the masked value to  $P_B$ . Then, the Equation 3 holds if both parties acts honestly.

$$\begin{aligned} & \sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] w_i[j] - M[r_0] \\ &= \sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] v_i[j] - K[r_0] + \left( \sum_{i \in [t]} \beta_i \chi_i[\alpha_i] - r_0 \right) \Delta \end{aligned} \quad (3)$$

We present the detailed protocol in Figure 11. We also prove the protocol securely instantiate  $\mathcal{F}_{MPVOLE}$  in  $(\mathcal{F}_{RVOLE}, \mathcal{F}_{VOLE}, \mathcal{F}_{OT})$  hybrid model against malicious adversary in Theorem 6. Moreover, the construction of  $\Pi_{VOLE}$  is introduced in Section A.9.

## A.6 Security Analysis of MPVOLE

**Theorem 6.** *Protocol  $\Pi_{MPVOLE}$  in Figure 11 securely instantiated  $\mathcal{F}_{MPVOLE}$  in Figure 3 in  $(\mathcal{F}_{RVOLE}, \mathcal{F}_{VOLE}, \mathcal{F}_{OT})$ -hybrid model against malicious adversaries.*

*Proof.* Let  $\mathcal{A}$  be a PPT adversary that corrupt  $P_A$  or  $P_B$ . We construct a PPT simulator  $\mathcal{S}$  with access to  $\mathcal{F}_{MPVOLE}$  and simulates the adversary's view. We will prove that the joint distribution over the output of  $\mathcal{A}$  and the honest party in the real world is indistinguishable from the joint distribution over the output of  $\mathcal{S}$  and the honest party in the ideal world.

**Corrupted  $P_A$ .** Let  $\mathcal{S}$  access  $\mathcal{F}_{MPVOLE}$  as an honest  $P_A$  and interact with  $\mathcal{A}$  as an honest  $P_B$ .  $\mathcal{S}$  passes all communication between  $\mathcal{A}$  and environment  $\mathcal{Z}$ .

For Initialize phase,  $\mathcal{S}$  simulates as follows:

1.  $\mathcal{S}$  emulates  $\mathcal{F}_{RVOLE}$  and receives (init) from  $\mathcal{A}$ .  $\mathcal{S}$  sends (init) to  $\mathcal{F}_{MPVOLE}$ .

For Extend phase,  $\mathcal{S}$  simulates as follows:

2.  $\mathcal{S}$  emulates  $\mathcal{F}_{RVOLE}$  and receives (extend, 1) from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $r_0 \in \mathbb{F}$ ,  $M[r_0] \in \mathbb{F}$  from  $\mathcal{A}$ .
3.  $\mathcal{S}$  emulates  $\mathcal{F}_{VOLE}$  and receives (extend,  $\beta, t$ ) from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $M[\beta] \in \mathbb{F}^t$  from  $\mathcal{A}$ .
4. For each iteration  $i \in [t]$ ,  $\mathcal{S}$  emulates  $\mathcal{F}_{OT}$  and receives  $\bar{a}_i[k], k \in [h]$ .  $\mathcal{S}$  reconstructs  $\alpha_i, i \in [t]$ .  $\mathcal{S}$  samples  $K_{i, \bar{a}_i[k]}^k \leftarrow \mathbb{F}, k \in [h]$  and sends them to  $\mathcal{A}$ .  $\mathcal{S}$  reconstructs  $e_u$  and sends (extend,  $e_u, h, t$ ) to  $\mathcal{F}_{MPVOLE}$ .  $\mathcal{S}$  also runs  $\{v_i[j]\}_{j \neq \alpha_i} := \text{GGM}'(\alpha_i, \{K_{i, \bar{a}_i[k]}^k\}_{k \in [h]})$ .
5. For each iteration  $i \in [t]$ ,  $\mathcal{S}$  samples  $d_i \leftarrow \mathbb{F}$  and sends it to  $\mathcal{A}$ .  $\mathcal{S}$  computes  $w_i[j] := v_i[j]$  for  $j \neq \alpha_i$  and  $w_i[\alpha_i] := M[\beta_i] - \left( d_i + \sum_{j \neq \alpha_i} w_i[j] \right)$ .
6.  $\mathcal{S}$  receives  $(\{\chi_i[j]\}_{i \in [t], j \in [n]}, x^*)$  from  $\mathcal{A}$ .  $\mathcal{S}$  checks whether  $x^* = \sum_{i \in [t]} \beta_i \cdot \chi_i[\alpha_i] - r_0$ . If it is,  $\mathcal{S}$  computes  $V_B = \sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] \cdot w_i[j] - M[r_0]$  and sends  $H(V_B)$  to  $\mathcal{A}$ . Otherwise,  $\mathcal{S}$  sends  $H(V_B) \leftarrow \mathbb{F}$  to  $\mathcal{A}$ .
7.  $\mathcal{S}$  constructs  $M[e_u] = w_1 || \dots || w_t$  to  $\mathcal{F}_{MPVOLE}$ .  $\mathcal{S}$  outputs what  $\mathcal{A}$  outputs.

We are going to show the simulated execution is indistinguishable from the real protocol execution.

**Hybrid  $\mathcal{H}_0$**  Same as real-world execution in  $(\mathcal{F}_{RVOLE}, \mathcal{F}_{VOLE}, \mathcal{F}_{OT})$  model.

**Protocol  $\Pi_{\text{MPVOLE}}$**

**Parameters:** GGM:  $(\mathbf{v}, \{S_{i,0}, S_{i,1}\}_{i \in [\log N]}) \leftarrow \text{GGM}(N, s), s \leftarrow \{0, 1\}^\kappa, \mathbf{v} \in \mathbb{F}^N, S_{i,0} \in \{0, 1\}^\kappa, S_{i,1} \in \{0, 1\}^\kappa$ ; GGM':  $\{w_h\}_{h \in [N]/\{\alpha\}} \leftarrow \text{GGM}'(\alpha, \{S_{i,\alpha_i}\}_{i \in [\log N]}), w_h \in \{0, 1\}^\kappa$ . Both algorithms are detailed in Section A.7.

**Inputs:** For each extend sparse execution,  $P_A$  holds regular sparse multi-point vector  $\mathbf{e}_u \in \mathbb{F}^{t \cdot 2^h}, n = 2^h$ .  $P_A$  partition  $\mathbf{e}_u \in \mathbb{F}^{t \cdot 2^h}$  into  $t$  blocks, that for each block  $\mathbf{e}_u^i \in \mathbb{F}^n, i \in [t]$ , it has one non-zero element  $\beta_i \in \mathbb{F}$  at position  $\alpha_i \in [n]$ .  $P_A$  represents  $\alpha_i$  as bit vectors, namely  $\mathbf{a}_i \in \{0, 1\}^h$ . Both  $P_A$  and  $P_B$  have integers  $t \in \mathbb{N}$  and  $n = 2^h$  for some  $h \in \mathbb{N}$ .

**Initialize:**

1. Both  $P_A$  and  $P_B$  sends (init) to  $\mathcal{F}_{\text{RVOLE}}$ .  $P_B$  receives  $\Delta \in \mathbb{F}$  from  $\mathcal{F}_{\text{RVOLE}}$ .

**Extend:**

2. Both  $P_A$  and  $P_B$  call  $\mathcal{F}_{\text{RVOLE}}$  with (extend, 1). The functionality sends  $r_0 \in \mathbb{F}, M[r_0] \in \mathbb{F}$  to  $P_A$ , and  $K[r_0] \in \mathbb{F}$  to  $P_B$ , such that  $M[r_0] = K[r_0] + r_0 \cdot \Delta$ .
3.  $P_A$  call  $\mathcal{F}_{\text{VOLE}}$  with (extend,  $\beta, \mathbf{t}$ ), where  $\beta$  is a concatenation of  $\beta_i$ s, and  $P_B$  call  $\mathcal{F}_{\text{VOLE}}$  with (extend,  $\mathbf{t}$ ). The functionality sends  $M[\beta] \in \mathbb{F}^t$  to  $P_A$ , and  $K[\beta] \in \mathbb{F}^t$  to  $P_B$ , such that  $M[\beta_i] = K[\beta_i] + \beta_i \Delta$  for each  $i \in [t]$ .
4. For each iteration  $i \in [t]$ ,  $P_B$  samples seed  $l_i \leftarrow \{0, 1\}^\kappa$ , runs  $\text{GGM}(1^n, l_i)$  to obtain  $(\{v_i[j]\}_{j \in [n]}, \{(K_{i,0}^k, K_{i,1}^k)\}_{k \in [h]})$ , and sets  $\mathbf{v}_i[j] := v_i[j]$  for  $j \in [n]$ . For  $k \in [h]$ ,  $P_B$  call  $\mathcal{F}_{\text{OT}}$  with input  $(K_{i,0}^k, K_{i,1}^k)$ ;  $P_A$  call  $\mathcal{F}_{\text{OT}}$  with input  $\bar{a}_i[k]$  and get  $K_{i,\bar{a}_i[k]}^k$ . Then  $P_A$  runs  $\{v_i[j]\}_{j \neq \alpha_i} := \text{GGM}'(\alpha_i, \{K_{i,\bar{a}_i[k]}^k\}_{k \in [h]})$ .
5. For each iteration  $i \in [t]$ ,  $P_B$  sends  $d_i := K[\beta_i] - \sum_{j \in [n]} \mathbf{v}_i[j] \in \mathbb{F}$  to  $P_A$ . Then,  $P_A$  defines  $\mathbf{w}_i \in \mathbb{F}^n$  as the vector with  $\mathbf{w}_i[j] := v_i[j]$  for  $j \neq \alpha_i$  and  $\mathbf{w}_i[\alpha_i] := M[\beta_i] - (d_i + \sum_{j \neq \alpha_i} \mathbf{w}_i[j])$ . Note that  $\mathbf{w}_i = \mathbf{v}_i + \Delta \cdot \mathbf{e}_u^i$ .
6.  $P_A$  and  $P_B$  check **GGM-consistency** as follows.
  - (a)  $P_A$  samples  $\chi_i[j] \leftarrow \mathbb{F}$  for  $i \in [t], j \in [n]$ .  $P_A$  then computes  $x^* := \sum_{i \in [t]} \beta_i \cdot \chi_i[\alpha_i] - r_0 \in \mathbb{F}$  and sends  $(\{\chi_i[j]\}_{i \in [t], j \in [n]}, x^*)$  to  $P_B$ , who computes  $y := K[r_0] - \Delta \cdot x^*$ .
  - (b)  $P_A$  computes  $V_A := \sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] \cdot \mathbf{w}_i[j] - M[r_0] \in \mathbb{F}$ , while  $P_B$  computes  $V_B := \sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] \cdot \mathbf{v}_i[j] - y \in \mathbb{F}$ . Then  $P_B$  sends  $H(V_B)$  to  $P_A$ .  $P_A$  aborts if  $H(V_A) \neq H(V_B)$ .
7.  $P_A$  outputs  $M[\mathbf{e}_u] = \mathbf{w}_1 || \dots || \mathbf{w}_t \in \mathbb{F}^{tn}$  and  $P_B$  outputs  $K[\mathbf{e}_u] = \mathbf{v}_1 || \dots || \mathbf{v}_t \in \mathbb{F}^{tn}$ .

Figure 11: Multi-point VOLE protocol.

**Hybrid  $\mathcal{H}_1$**  Same as Hybrid  $\mathcal{H}_0$  except  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{RVOLE}}, \mathcal{F}_{\text{VOLE}}, \mathcal{F}_{\text{OT}}$  and simulate messages to  $\mathcal{A}$ .

For step (4),  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{OT}}$  and sends  $K_{i,\bar{a}_i[k]}^k \leftarrow \mathbb{F}, k \in [h]$  to  $\mathcal{A}$ . In Hybrid  $\mathcal{H}_0$ , an honest  $P_B$  computes  $K_{i,\bar{a}_i[k]}^k, k \in [h]$  according to GGM algorithm with random seed  $l_i \leftarrow \{0, 1\}^\kappa$ . By leveraging PRG over random seed  $l_i$ ,  $K_{i,\bar{a}_i[k]}^k, k \in [h]$  are indistinguishable from random value over  $\mathbb{F}$ .

For step (5),  $\mathcal{S}$  samples  $d_i \leftarrow \mathbb{F}, i \in [t]$  to  $\mathcal{A}$ . In Hybrid  $\mathcal{H}_0$ , an honest  $P_B$  computes  $d_i = K[\beta_i] - \sum_{j \in [n]} \mathbf{v}_i[j]$  to  $\mathcal{A}$ . Since  $K[\beta_i]$  is unknown to  $\mathcal{A}$ ,  $d_i$  is uniform distributed to  $\mathcal{A}$ .

For step (6),  $\mathcal{S}$  receives  $(\{\chi_i[j]\}_{i \in [t], j \in [n]}, x^*)$  from  $\mathcal{A}$  and sends  $H(V_B)$  to  $\mathcal{A}$ .  $\mathcal{S}$  sets  $H(V_B) =$

$H(\sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] \cdot \mathbf{w}_i[j] - \mathbf{M}[r_0])$  if  $x^* = \sum_{i \in [t]} \beta_i \cdot \chi_i[\alpha_i] - r_0$ .  $\mathcal{S}$  sets  $H(V_B) \leftarrow \mathbb{F}$  if  $x^* \neq \sum_{i \in [t]} \beta_i \cdot \chi_i[\alpha_i] - r_0$ . In Hybrid  $\mathcal{H}_0$ , an honest  $\mathbf{P}_B$  computes  $V_B = \sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] \cdot \mathbf{v}_i[j] - y$ , where  $y = \mathbf{K}[r_0] - \Delta \cdot x^*$ , and sends  $H(V_B)$  to  $\mathcal{A}$ . If  $x^* = \sum_{i \in [t]} \beta_i \cdot \chi_i[\alpha_i] - r_0 + \text{rand}$ ,  $\text{rand} \in \mathbb{F}$ ,  $y = \mathbf{M}[r_0] - \sum_{i \in [t]} \beta_i \cdot \chi_i[\alpha_i] \cdot \Delta - \text{rand} \cdot \Delta$ . We have the following equation holds.

$$\begin{aligned}
V_B &= \sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] \cdot \mathbf{v}_i[j] - y \\
&= \sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] \mathbf{v}_i[j] - \mathbf{M}[r_0] + \sum_{i \in [t]} \beta_i \chi_i[\alpha_i] \Delta + \text{rand} \Delta \\
&= \sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] \cdot \mathbf{w}_i[j] - \sum_{i \in [t]} \beta_i \cdot \chi_i[\alpha_i] \cdot \Delta - \mathbf{M}[r_0] \\
&\quad + \sum_{i \in [t]} \beta_i \cdot \chi_i[\alpha_i] \cdot \Delta + \text{rand} \cdot \Delta \\
&= \sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] \cdot \mathbf{w}_i[j] - \mathbf{M}[r_0] + \text{rand} \cdot \Delta \\
&= V_A + \text{rand} \cdot \Delta
\end{aligned}$$

If  $\text{rand} = 0$ , we have  $V_A = V_B$ . Thus, the simulation is identical to the real world protocol execution. Otherwise,  $\mathcal{A}$  will receives  $H(V_B) = H(V_A + \text{rand} \cdot \Delta)$ , which is indistinguishable from a random value in the hybrid of random oracle model.

Thus, this hybrid is identical to the previous one.

**Corrupted  $\mathbf{P}_B$ .** Let  $\mathcal{S}$  access  $\mathcal{F}_{\text{MPVole}}$  as an honest  $\mathbf{P}_B$  and interact with  $\mathcal{A}$  as an honest  $\mathbf{P}_A$ .  $\mathcal{S}$  passes all communication between  $\mathcal{A}$  and environment  $\mathcal{Z}$ . For Initialize phase,  $\mathcal{S}$  simulates as follows:

1.  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{Rvole}}$  and receives (init) from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $\Delta$  from  $\mathcal{A}$ .

For Extend phase,  $\mathcal{S}$  simulates as follows:

2.  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{Rvole}}$  and receives (extend, 1) from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $\mathbf{K}[r_0] \leftarrow \mathbb{F}$  from  $\mathcal{A}$ .
3.  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{Vole}}$  and receives (extend, t) from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $\mathbf{K}[\beta] \leftarrow \mathbb{F}^t$  from  $\mathcal{A}$ .  $\mathcal{S}$  samples  $\beta_i, i \in [t]$  and computes  $\mathbf{M}[\beta_i] = \mathbf{K}[\beta_i] + \beta_i \cdot \Delta, i \in [t]$ .
4. For each iteration  $i \in [t]$ ,  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{OT}}$  and receives  $(K_{i,0}^k, K_{i,1}^k), k \in [h]$ .
5. For each iteration  $i \in [t]$ ,  $\mathcal{S}$  receives  $d_i \leftarrow \mathbb{F}$  from  $\mathcal{A}$ . For each  $\alpha_i \in [n]$ ,  $\mathcal{S}$  computes  $\mathbf{w}_i^{\alpha_i}$  as follows:

- (a) Computes  $\{v_i^{\alpha_i}[j]\}_{j \neq \alpha_i} := \text{GGM}'(\alpha_i, \{K_{i,\bar{\alpha}_i[k]}^k\}_{k \in [h]})$  and sets  $\mathbf{w}_i^{\alpha_i}[j] = v_i^{\alpha_i}[j]$  for  $j \neq \alpha_i$
- (b) Sets  $\mathbf{w}_i^{\alpha_i}[\alpha_i] := \mathbf{M}[\beta_i] - (d_i + \sum_{j \neq \alpha_i} \mathbf{w}_i^{\alpha_i}[j])$ .

6. (a)  $\mathcal{S}$  samples  $(\{\chi_i[j] \leftarrow \mathbb{F}\}_{i \in [t], j \in [n]}, x^* \leftarrow \mathbb{F})$  to  $\mathcal{A}$ .

(b)  $\mathcal{S}$  receives  $H(V_B)$  from  $\mathcal{A}$ .  $\mathcal{S}$  is able to extract  $V_B$  as  $\mathcal{S}$  emulate random oracle  $H$  for  $\mathcal{A}$ .

For each  $\{\alpha_i \in [n]\}_{i \in [t]}$ , denoted as  $\alpha \in \mathcal{N}^t, \mathcal{N} := [n]$ ,  $\mathcal{S}$  computes  $V_A^\alpha$  as follows:  $V_A^\alpha = \sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] \cdot \mathbf{w}_i^{\alpha_i}[j] - \mathbf{M}[r_0]$ . Define  $I := \{\alpha \in \mathcal{N}^t | V_A^\alpha = V_B\}$ . We also define  $I = I_1 \times \dots \times I_t$ , where  $I_i := \{\alpha_i \in \mathcal{N} | V_A^\alpha = V_B\}$ .  $\mathcal{S}$  aborts if  $I = \emptyset$ .

$\mathcal{S}$  chooses an arbitrary  $\alpha \in I$  and computes vector  $\mathbf{v}_i$  for each  $i \in [t]$  as follows:

- (a) Sets  $\mathbf{v}_i[j] := \mathbf{w}_i^{\alpha_i}[j]$  for  $j \neq \alpha_i$
- (b) Sets  $\mathbf{v}_i[\alpha_i] := \mathbf{K}[\beta_i] - d_i - \sum_{j \neq \alpha_i} \mathbf{v}_i[j]$

$\mathcal{S}$  send  $\{I_1, \dots, I_t\}$  to  $\mathcal{F}_{\text{MPVOLE}}$ . If it receives **abort**,  $\mathcal{S}$  aborts.

7.  $\mathcal{S}$  constructs  $\mathbf{K}[e_u] = \mathbf{v}_1 || \dots || \mathbf{v}_t$  to  $\mathcal{F}_{\text{MPVOLE}}$ .  $\mathcal{S}$  outputs what  $\mathcal{A}$  outputs.

We are going to show the simulated execution is indistinguishable from the real protocol execution.

**Hybrid  $\mathcal{H}_0$**  Same as real-world execution in  $(\mathcal{F}_{\text{RVOLE}}, \mathcal{F}_{\text{VOLE}}, \mathcal{F}_{\text{OT}})$  model.

**Hybrid  $\mathcal{H}_1$**  Same as Hybrid  $\mathcal{H}_0$  except  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{RVOLE}}, \mathcal{F}_{\text{VOLE}}, \mathcal{F}_{\text{OT}}$  and simulate messages to  $\mathcal{A}$ .

For step (6),  $\mathcal{S}$  samples  $(\{\chi_i[j] \leftarrow \mathbb{F}\}_{i \in [t], j \in [n]}, x^* \leftarrow \mathbb{F})$  to  $\mathcal{A}$ . In Hybrid  $\mathcal{H}_0$ , an honest  $\mathbf{P}_A$  samples  $\{\chi_i[j] \leftarrow \mathbb{F}\}_{i \in [t], j \in [n]}$ , and computes  $x^* = \sum_{i \in [t]} \beta_i \cdot \chi_i[\alpha_i] - r_0$ . Since  $r_0$  is output by  $\mathcal{F}_{\text{VOLE}}$  to  $\mathbf{P}_A$  and is uniformly distributed against  $\mathcal{A}$ ,  $x^*$  is uniformly distributed against  $\mathcal{A}$ , which is indistinguishable from this hybrid. Thus, this hybrid is identical to the previous one.

**Hybrid  $\mathcal{H}_2$**  Same as Hybrid  $\mathcal{H}_1$  except  $\mathcal{S}$  aborts at step (6) if  $I = \emptyset$  or received **abort** from  $\mathcal{F}_{\text{MPVOLE}}$ .

For step (6),  $\mathcal{S}$  extracts set  $I$  and send it to  $\mathcal{F}_{\text{MPVOLE}}$ , which is equivalent to  $\mathcal{A}$  samples selective failure attack on  $\mathbf{P}_A$ 's chosen non-zero index  $\alpha_i, i \in [t]$ . If  $\mathcal{S}$  receives **abort** from  $\mathcal{F}_{\text{MPVOLE}}$ , there exist  $i, i \in [t], \alpha_i \notin I_i$ , that is equivalent to  $\alpha \notin I$ . In the real world protocol execution, if  $V_B \neq V_A^\alpha$ ,  $\mathbf{P}_A$  aborts. This is equivalent to  $\alpha \notin I$ . Thus,  $\mathcal{F}_{\text{MPVOLE}}$  aborts if and only if the real world protocol execution aborts. This hybrid is identical to the previous one.

Then, we will prove the outputs of  $\mathcal{A}$  and honest party in the real world is indistinguishable from the outputs of  $\mathcal{S}$  and honest party in the ideal world.

We will prove that except with probability  $1/|\mathbb{F}|$ , all choices of  $\alpha_i \in I_i$  in the above simulation lead to the same vector  $\mathbf{v}_i$ .

Since  $V_A^\alpha = V_A^{\alpha'} = V_B$ , we have:

$$\begin{aligned}
V_A^\alpha &= V_A^{\alpha'} \\
\sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] \cdot \mathbf{w}_i^{\alpha_i}[j] - \mathbf{M}[r_0] &= \sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] \cdot \mathbf{w}_i^{\alpha'_i}[j] - \mathbf{M}[r_0] \\
\sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] \cdot \mathbf{w}_i^{\alpha_i}[j] - y - \sum_{i \in [t]} \beta_i \chi_i[\alpha_i] \Delta &= \sum_{i \in [t]} \sum_{j=0}^{n-1} \chi_i[j] \cdot \mathbf{w}_i^{\alpha'_i}[j] - y - \sum_{i \in [t]} \beta_i \chi_i[\alpha'_i] \Delta \\
&= \sum_{i \in [t]} \left( \sum_{j \neq \alpha_i, \alpha'_i} \chi_i[j] (\mathbf{w}_i^{\alpha_i}[j] - \mathbf{w}_i^{\alpha'_i}[j]) + \chi_i[\alpha_i] (\mathbf{w}_i^{\alpha_i}[\alpha_i] \right. \\
&\quad \left. - \mathbf{w}_i^{\alpha'_i}[\alpha_i] - \beta_i \Delta) + \chi_i[\alpha'_i] (\mathbf{w}_i^{\alpha_i}[\alpha'_i] - \mathbf{w}_i^{\alpha'_i}[\alpha'_i] + \beta_i \Delta) \right) \\
&= 0
\end{aligned}$$

Since  $\{\chi_i[j]\}_{i \in [t], j \in [n]}$  are uniformly distributed and independent from  $w_i^{\alpha_i}$ ,  $w_i^{\alpha'_i}$ , and  $\Delta$ , we have except with probability  $1/|\mathbb{F}|$ ,

$$w_i^{\alpha_i}[j] = w_i^{\alpha'_i}[j], j \neq \alpha_i, \alpha'_i \quad (4)$$

$$w_i^{\alpha_i}[\alpha_i] - w_i^{\alpha'_i}[\alpha_i] = w_i^{\alpha'_i}[\alpha'_i] - w_i^{\alpha_i}[\alpha'_i] = \Delta\beta_i \quad (5)$$

Thus, we are able to obtain for any  $\alpha_i, \alpha'_i \in I_i$ ,  $v_i^{\alpha_i}[j] = v_i^{\alpha'_i}[j], j \neq \{\alpha_i, \alpha'_i\}$  from Equation 4. According to  $w_i^{\alpha_i}[\alpha_i] = v_i^{\alpha_i}[\alpha_i] + \Delta\beta_i$  and Equation 5, we are able to get  $v_i^{\alpha_i}[\alpha_i] = w_i^{\alpha'_i}[\alpha_i] = v_i^{\alpha'_i}[\alpha_i]$  and similarly  $v_i^{\alpha'_i}[\alpha'_i] = w_i^{\alpha_i}[\alpha'_i] = v_i^{\alpha_i}[\alpha'_i]$ .  $\square$

## A.7 GGM Algorithm

We detailed the GGM algorithm [BGI15, BGI16, BCG<sup>+</sup>17] and its evaluation in this section to help with the construction of  $\Pi_{\text{MPVole}}$ .

- **GGM( $N, s$ ):** On input  $N = 2^d$  for some  $d \in \mathbb{N}$  and  $s \in \{0, 1\}^\kappa$ , this algorithm executes as follows:
  1. Define  $s_0^0 := s$ . For  $i \in [1, d]$  and  $j \in [0, 2^{i-1})$  compute  $(s_{2j}^i, s_{2j+1}^i) := G(s_j^{i-1})$ , where  $G : \{0, 1\}^\kappa \rightarrow \{0, 1\}^{2\kappa}$  is a PRG.
  2. Compute the leaves as  $(s_{2j}^d, s_{2j+1}^d) := G'(s_j^{d-1})$  for  $j \in [0, 2^{d-1})$ , where  $G' : \{0, 1\}^\kappa \rightarrow \{0, 1\}^{2\kappa}$  is another PRG. Define a vector  $\mathbf{v}$  as  $v_h := s_{h-1}^d$  for  $h \in [N]$ .
  3. For  $i \in [d]$ , compute  $S_{i,0} := \sum_{j \in [0, 2^{i-1})} s_{2j}^i$  and  $S_{i,1} := \sum_{j \in [0, 2^{i-1})} s_{2j+1}^i$ , where for  $i \in [1, d)$ , the addition is defined over  $\mathbb{F}_{2^\kappa}$ ; for  $i = d$ , the addition is defined over  $\mathbb{F}$ .
  4. Output  $(\mathbf{v}, \{(S_{i,0}, S_{i,1})\}_{i \in [d]})$ .
- **GGM'( $\alpha, \{S_{i,\bar{\alpha}_i}\}_{i \in [d]}$ ):** On input  $\alpha \in [N]$  and a set  $\{S_{i,\bar{\alpha}_i}\}_{i \in [d]}$  where  $\bar{\alpha}_i = \alpha_i \oplus 1 \in \{0, 1\}$  for  $i \in [d]$  and  $\alpha = \sum_{i=1}^d \alpha_i \cdot 2^{i-1} + 1 \in [N]$ , for  $i \in [d]$ , this algorithm does the following:
  1. Define an  $i$ -bit string  $\alpha_i^* := \alpha_1 \cdots \alpha_{i-1} \bar{\alpha}_i$ .
  2. If  $i = 1$ , define  $s_{\bar{\alpha}_1}^1 := S_{1,\bar{\alpha}_1}$ .
  3. If  $i \geq 2$ , for  $j \in [0, 2^{i-1}), j \neq \alpha_1 \cdots \alpha_{i-1}$ , compute  $(s_{2j}^i, s_{2j+1}^i) := G(s_j^{i-1})$  if  $i < d$  and  $(s_{2j}^d, s_{2j+1}^d) := G'(s_j^{d-1})$  otherwise.
  4. Compute  $s_{\alpha_i^*}^i := S_{i,\bar{\alpha}_i} + \left( \sum_{j \in [0, 2^{i-1}), j \neq \alpha_1 \cdots \alpha_{i-1}} s_{2j+\bar{\alpha}_i}^i \right)$ , where the addition is defined over  $\mathbb{F}_{2^\kappa}$  if  $i \in [1, d)$  and over  $\mathbb{F}$  otherwise.
  5. Define  $w_h := s_{h-1}^d$  for  $h \in [N], h \neq \alpha$ , and output  $\{w_h\}_{h \in [N] \setminus \{\alpha\}}$ .

## A.8 Random VOLE

**Random Vector Oblivious Linear Evaluation.** Random Vector Oblivious Linear Evaluation (RVOLe) functionality is a two party functionality that samples a vector  $\mathbf{x} \leftarrow \mathbb{F}^m$  to  $\mathcal{P}_A$ , a field element  $\Delta \leftarrow \mathbb{F}$  and a vector  $\mathbf{K}[\mathbf{x}] \leftarrow \mathbb{F}^m$  to  $\mathcal{P}_B$ , and delivers  $\mathbf{M}[\mathbf{x}] = \mathbf{K}[\mathbf{x}] + \mathbf{x} \cdot \Delta \in \mathbb{F}^m$  to  $\mathcal{P}_A$ . RVOLe can be efficiently generated by LPN-based approaches [BCGI18, WYKW21, YWL<sup>+</sup>20, SGRR19]. This paper takes the functionality of random VOLE depicted in Figure 12 as the fundamental building block to construct committed VOLE.

**Functionality  $\mathcal{F}_{\text{RVOLE}}$**

**Initialize:** Upon receiving (init) from both  $P_A$  and  $P_B$ , sample  $\Delta \leftarrow \mathbb{F}$  and send it to  $P_B$ . The functionality stores  $\Delta$ . If  $P_B$  is malicious, receive  $\Delta \in \mathbb{F}$  from  $P_B$ .

**Extend:** Upon receiving (extend,  $m$ ) from both  $P_A$  and  $P_B$ :

1. Samples  $K[x] \leftarrow \mathbb{F}^m$ . If  $P_B$  is malicious, receive  $K[x] \in \mathbb{F}^m$  from adversary.
2. Samples  $x \leftarrow \mathbb{F}^m$  and computes  $M[x] := K[x] + \Delta \cdot x \in \mathbb{F}^m$ . If  $P_A$  is malicious, receives  $(x \in \mathbb{F}^m, M[x] \in \mathbb{F}^m)$  from the adversary, and recomputes  $K[x] := M[x] - \Delta \cdot x \in \mathbb{F}^m$ .
3. Sends  $(x, M[x])$  to  $P_A$  and  $(K[x])$  to  $P_B$ .

Figure 12: The random VOLE functionality.

## A.9 Protocol of VOLE

We present a simple VOLE protocol constructed from  $\mathcal{F}_{\text{RVOLE}}$ . In this protocol,  $P_A$  holds the designated input vector  $b \in \mathbb{F}^m$ . After calling  $\mathcal{F}_{\text{RVOLE}}$  on both sides,  $P_A$  obtains random  $(r, M[r]) \in \mathbb{F}^{2m}$  and  $P_B$  obtains  $(\Delta, K[r]) \in \mathbb{F}^{m+1}$ , such that  $M[r] = K[r] + r \cdot \Delta$ . Then,  $P_A$  sends the difference  $f$  between  $r$  and  $b$ . Consequently, both parties are able to obtain the VOLE correlation of the designated vector  $b$ , where  $M[b] = M[r]$  and  $K[b] = K[r] + f \cdot \Delta$ .

This protocol is able to securely instantiate  $\mathcal{F}_{\text{VOLE}}$  in the  $\mathcal{F}_{\text{RVOLE}}$  hybrid model. A simple proof sketch is provided here. For corrupted  $P_A$ , a simulator  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{VOLE}}$  and receives  $r \in \mathbb{F}^m$ ,  $M[r] \in \mathbb{F}^m$  from  $\mathcal{A}$ .  $\mathcal{S}$  receives  $f \in \mathbb{F}^m$  from  $\mathcal{A}$ . It is able to recover  $b = r - f$ , and  $M[b] = M[r]$ , and sends  $M[b]$  to  $\mathcal{F}_{\text{VOLE}}$ . The simulator does not need to simulate any message to the adversary. For corrupted  $P_B$ ,  $\mathcal{S}$  receives  $K[r] \in \mathbb{F}^m$  from  $\mathcal{A}$  and samples  $f \leftarrow \mathbb{F}^m$  to  $\mathcal{A}$ , which is indistinguishable from the  $f$  in real protocol. Then,  $\mathcal{S}$  is able to compute  $K[b] = K[r] + f \cdot \Delta$  to  $\mathcal{F}_{\text{VOLE}}$ . The above simulation could simulate the view of adversary in real protocol. Also,  $\mathcal{S}$  output what  $\mathcal{A}$  outputs could ensure the same output distribution of the real protocol.

## A.10 Proof of Theorem 4

*Proof.* Let  $\mathcal{A}$  be a PPT adversary that corrupt  $P_1$  or  $P_j$ . We construct a PPT simulator  $\mathcal{S}$  with access to  $\mathcal{F}_{\text{PSI}}$  and simulates the adversary's view. We will prove that the joint distribution over the output of  $\mathcal{A}$  and the honest party in the real world is indistinguishable from the joint distribution over the output of  $\mathcal{S}$  and the honest party in the ideal world.

**Corrupted  $P_1$ .** Let  $\mathcal{S}$  access  $\mathcal{F}_{\text{PSI}}$  as an honest  $P_1$  and interact with  $\mathcal{A}$  as an honest  $P_2$ .  $\mathcal{S}$  passes all communication between  $\mathcal{A}$  and environment  $\mathcal{Z}$ .

- (1)  $\mathcal{S}$  emulates random oracle  $H_1$  and receives elements  $x_i, i \in [n]$  from  $\mathcal{A}$ .  $\mathcal{S}$  samples  $H_1(x_i) \leftarrow \mathbb{F}, i \in [n]$  and sends them to  $\mathcal{A}$ .
- (2)  $\mathcal{S}$  receives  $\text{Hcom}_{\mathbf{p}} \in \mathbb{F}^N$  from  $\mathcal{A}$ .
- (3)  $\mathcal{S}$  samples  $c^s \leftarrow \mathbb{F}$  and sends it to  $\mathcal{A}$ .
- (4)  $\mathcal{S}$  receives  $w^r \in \mathbb{F}, r \in \{0, 1\}^\kappa$  from  $\mathcal{A}$ .
- (5)  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{C-VOLE}}$  and receives (init,  $\mathbf{p}, \text{Hdecom}_{\mathbf{p}}$ ), where  $\mathbf{p} \in \mathbb{F}^m$ ,  $\text{Hdecom}_{\mathbf{p}} \in \mathbb{F}^{m'}$  from  $\mathcal{A}$ .  $\mathcal{S}$  checks whether  $(\text{Hcom}_{\mathbf{p}}, \text{Hdecom}_{\mathbf{p}})$  opens to  $\mathbf{p}$ . If not,  $\mathcal{S}$  sends abort to  $\mathcal{A}$ . Otherwise,  $\mathcal{S}$  waits

to receive  $M[p] \leftarrow \mathbb{F}^m$  from  $\mathcal{A}$ .  $\mathcal{S}$  checks whether  $\text{Decode}(\mathbf{p}, x_i, r) = H(x_i)$ . If it is,  $\mathcal{S}$  inserts  $x_i$  to set  $X$ .  $\mathcal{S}$  sends  $(\text{init}, X)$  to  $\mathcal{F}_{\text{PSI}}$  and waits to receive  $(\text{setid}, Z)$ .

- (6)  $\mathcal{S}$  samples  $w^s \leftarrow \mathbb{F}$  and programs random oracle  $H_1$  that  $c^s = H_1(w^s)$ .  $\mathcal{S}$  sends  $w^s$  to  $\mathcal{A}$ .  $\mathcal{S}$  computes  $w = w^s + w^r$ .
- (7)  $\mathcal{S}$  samples  $ey_i \leftarrow \{0, 1\}^{\text{out}}$  for each  $y_i \in Y$  and inserts  $ey_i$  to set  $EY$ .  $\mathcal{S}$  sends  $EY$  to  $\mathcal{A}$ .
- (8)  $\mathcal{S}$  emulates random oracle  $H_2$  and receives query  $(q_i, x_i)$  from  $\mathcal{A}$ .  $\mathcal{S}$  first checks whether the query is fresh.
  - (a) If it is,  $\mathcal{S}$  checks whether  $x_i \in Z$  and  $q_i = \text{Decode}(M[p], x_i, r) + w$ . If it is,  $\mathcal{S}$  samples  $ex_i$  from  $EY/EX$  and sends it to  $\mathcal{A}$ .  $\mathcal{S}$  records the triple  $(q_i, x_i, ex_i)$  and inserts  $ex_i$  to  $EX$ . If not,  $\mathcal{S}$  samples  $ex_i \leftarrow \{0, 1\}^{\text{out}} / (EY \cup EX)$  to  $\mathcal{A}$ , records the triple, and inserts  $ex_i$  to  $EX$ .
  - (b) If not,  $\mathcal{S}$  checks records and sends corresponding  $ex_i$  to  $\mathcal{A}$ .

$\mathcal{S}$  aborts if  $\mathcal{A}$  aborts and outputs what  $\mathcal{A}$  outputs.

While  $\mathcal{S}$  access  $\mathcal{F}_{\text{PSI}}$  as an honest  $P_1$  and interact with  $\mathcal{A}$  as an honest  $P_j, j \in [3, n]$ .  $\mathcal{S}$  simulates the **Extend** phase. The simulation is same as above except step (5):

- (5)  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{C-VOLE}}$  and receives  $(\text{init}, \mathbf{p}, \text{Hdecom}_{\mathbf{p}})$ , where  $\mathbf{p} \in \mathbb{F}^m, \text{Hdecom}_{\mathbf{p}} \in \mathbb{F}^{m'}$  from  $\mathcal{A}$ .  $\mathcal{S}$  checks whether  $(\text{Hcom}_{\mathbf{p}}, \text{Hdecom}_{\mathbf{p}})$  opens to  $\mathbf{p}$ . If not,  $\mathcal{S}$  sends **abort** to  $\mathcal{A}$ . Otherwise,  $\mathcal{S}$  waits to receive  $M[p] \leftarrow \mathbb{F}^m$  from  $\mathcal{A}$ .  $\mathcal{S}$  sends  $(\text{init}, \text{setid})$  to  $\mathcal{F}_{\text{PSI}}$  and waits to receive  $Z$ .

We are going to show the simulated execution is indistinguishable from real-world protocol execution.

**Hybrid  $\mathcal{H}_0$**  Same as real-world execution in  $(\mathcal{F}_{\text{C-VOLE}}, H_1, H_2)$  model.

**Hybrid  $\mathcal{H}_1$**  Same as Hybrid  $\mathcal{H}_0$  except  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{C-VOLE}}$ , random oracle  $H_1, H_2$ , and simulates the messages to  $\mathcal{A}$  as follows:

For step (1),  $\mathcal{S}$  emulates  $H_1$  and samples  $H_1(x_i) \leftarrow \mathbb{F}, i \in [n]$  to  $\mathcal{A}$ . In Hybrid  $\mathcal{H}_0$ ,  $H_1(x_i) \leftarrow \mathbb{F}, i \in [n]$  are generated and outputted by a random oracle, which is uniform distributed and indistinguishable from this hybrid.

For step (3),  $\mathcal{S}$  samples  $c^s \leftarrow \mathbb{F}$  to  $\mathcal{A}$ . In hybrid  $\mathcal{H}_0$ , Random oracle  $H_1$  outputs  $c^s$ , which is uniformly distributed over  $\mathbb{F}$  and indistinguishable from this hybrid.

For step (6),  $\mathcal{S}$  samples  $w^s \leftarrow \mathbb{F}$  to  $\mathcal{A}$ .  $\mathcal{S}$  also programs random oracle  $H_1$  that  $c^s = H_1(w^s)$ . In hybrid  $\mathcal{H}_0$ , an honest  $P_2$  samples  $w^s \leftarrow \mathbb{F}$  and computes  $c^s = H_1(w^s)$ . The distribution of  $w^s$  is indistinguishable from this hybrid.

For step (7),  $\mathcal{S}$  programs  $EY$  to size  $|Y|$  with random elements in  $\{0, 1\}^{\text{out}}$ . In hybrid  $\mathcal{H}_0$ , an honest  $P_2$  computes  $ey_i$  from random oracle for each element  $y_i \in Y_2$ . Thus,  $EY$  is indistinguishable from this hybrid.

For step (8),  $\mathcal{S}$  emulates  $H_2$  and programs  $H_2$  that for  $x_i \in Z, q_i = \text{Decode}(M[p], x_i, r) + w$ , it sends  $ex_i \in EY$  to  $\mathcal{A}$ . Thus, the output of  $\mathcal{A}$  is same as interacting with an honest  $P_2$ .

Thus, this hybrid is identical to the previous one.

**Corrupted  $P_j$ .** Let  $\mathcal{S}$  access  $\mathcal{F}_{\text{PSI}}$  as an honest  $P_j$  and interact with  $\mathcal{A}$  as an honest  $P_1$ .  $\mathcal{S}$  passes all communication between  $\mathcal{A}$  and environment  $\mathcal{Z}$ .

- (1-2)  $\mathcal{S}$  samples  $\text{Hcom}_{\mathbf{p}} \leftarrow \mathbb{F}^N$ .  $\mathcal{S}$  publishes  $\text{Hcom}_{\mathbf{p}}$ .

- (3)  $\mathcal{S}$  emulates  $H_1$  and receives  $w^s$  from  $\mathcal{A}$ .  $\mathcal{S}$  samples  $c^s \leftarrow \mathbb{F}$  to  $\mathcal{A}$ .  $\mathcal{S}$  receives  $c'^s \leftarrow \mathbb{F}$  from  $\mathcal{A}$ . If  $c'^s \neq c^s$ ,  $\mathcal{S}$  aborts in step (6).



- (4)  $\mathcal{S}$  samples  $w^r \leftarrow \mathbb{F}, r \leftarrow \{0, 1\}^\kappa$  and sends them to  $\mathcal{A}$ .
  - (5)  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{C-VOLE}}$  and receives (init) from  $\mathcal{A}$ .  $\mathcal{S}$  waits to receive  $(\Delta, \mathbf{K}[\mathbf{p}]) \leftarrow \mathbb{F}^{m+1}$  from  $\mathcal{A}$ .
  - (6)  $\mathcal{S}$  receives  $w'^s$  from  $\mathcal{A}$  and checks whether  $w'^s = w^s$ . If not,  $\mathcal{S}$  aborts.
  - (7)  $\mathcal{S}$  emulates  $H_1, H_2$ , and receives query as follows:
    - (a)  $\mathcal{S}$  receives  $y_i$  from  $\mathcal{A}$ . If the query is fresh,  $\mathcal{S}$  samples  $H_1(y_i) \leftarrow \mathbb{F}$  to  $\mathcal{A}$  and stores the pair  $(y_i, H_1(y_i))$ . If not,  $\mathcal{S}$  checks the records and sends corresponding  $H_1(y_i)$  to  $\mathcal{A}$ .
    - (b)  $\mathcal{S}$  receives query  $(q_i, y_i)$  from  $\mathcal{A}$ . If the query is fresh,  $\mathcal{S}$  samples  $ey_i \leftarrow \{0, 1\}^{\text{out}}$  to  $\mathcal{A}$  and records triple  $(y_i, q_i, ey_i)$ . If not,  $\mathcal{S}$  checks the records and sends corresponding  $ey_i$  to  $\mathcal{A}$ .
- $\mathcal{S}$  receives  $EY$  from  $\mathcal{A}$ . For each  $ey_i \in EY$ ,  $\mathcal{S}$  checks whether it is recorded and  $q_i = \text{Decode}(\mathbf{K}[\mathbf{p}], y_i, r) + \Delta H_1(y_i) + w$ . If it is,  $\mathcal{S}$  inserts  $y_i$  to set  $Y$  and sends  $(\text{intersect}, Y)$  to  $\mathcal{F}_{\text{PSI}}$ .  $\mathcal{S}$  receives  $\text{setid}$  from  $\mathcal{F}_{\text{PSI}}$ .

While  $\mathcal{S}$  access  $\mathcal{F}_{\text{PSI}}$  as an honest  $P_j$  and interact with  $\mathcal{A}$  as an honest  $P_1$ .  $\mathcal{S}$  simulates the **Extend** phase. The simulation is same as above except:  $\mathcal{S}$  extracts set  $Y$  and sends  $(\text{intersect}, Y, \text{setid})$  to  $\mathcal{F}_{\text{PSI}}$  at step (7).

We are going to show the simulated execution is indistinguishable from real-world protocol execution.

**Hybrid  $\mathcal{H}_0$**  Same as real-world execution in  $(\mathcal{F}_{\text{C-VOLE}}, H_1, H_2)$  model.

**Hybrid  $\mathcal{H}_1$**  Same as Hybrid  $\mathcal{H}_0$  except  $\mathcal{S}$  emulates  $\mathcal{F}_{\text{C-VOLE}}$ , random oracle  $H_1, H_2$ , and simulates the messages to  $\mathcal{A}$  as follows:

For step (1-2),  $\mathcal{S}$  samples  $\mathbf{Hcom}_{\mathbf{p}} \leftarrow \mathbb{F}^N$ , and publishes it. In Hybrid  $\mathcal{H}_0$ ,  $\mathbf{Hcom}_{\mathbf{p}}$  generated from commitment scheme. According to Theorem 2,  $\mathbf{Hcom}_{\mathbf{p}}$  is computational indistinguishable from pseudorandom vector over  $\mathbb{F}^N$ , which is indistinguishable from this hybrid.

For step (3),  $\mathcal{S}$  emulates  $H_1$  and samples  $c^s \leftarrow \mathbb{F}$  for each  $w^s$  to  $\mathcal{A}$ . In hybrid  $\mathcal{H}_0$ ,  $c^s$  is uniformly distributed over  $\mathbb{F}$  and indistinguishable from this hybrid.

For step (4),  $\mathcal{S}$  samples  $w^r \leftarrow \mathbb{F}, r \leftarrow \{0, 1\}^k$  as an honest  $P_1$  does in hybrid  $\mathcal{H}_0$ .

For step (7),  $\mathcal{S}$  emulates  $H_1, H_2$ , and samples  $H_1(y_i) \leftarrow \mathbb{F}, ey_i \leftarrow \{0, 1\}^{\text{out}}$  to  $\mathcal{A}$ , which is indistinguishable from the output of a random oracle.  $\mathcal{S}$  records all the programmed pairs and triple.  $\mathcal{S}$  receives  $EY$  from  $\mathcal{A}$  and checks each  $ey_i \in EY$  whether been recorded and the previous query is computed correctly to extract  $\mathcal{A}$ 's private set elements and sends them to  $\mathcal{F}_{\text{PSI}}$ . Thus, the output of  $\mathcal{S}$  and honest party is same as the output of  $\mathcal{A}$  and honest party.

Thus, this hybrid is identical to the previous one.

**Hybrid  $\mathcal{H}_2$**  Same as Hybrid  $\mathcal{H}_1$  except  $\mathcal{S}$  aborts at step (6) if  $c'^s \neq c^s$  or  $w'^s \neq w^s$ . It is indistinguishable from  $P_1$  aborts if  $c^s \neq H_1(w^s)$  in hybrid  $\mathcal{H}_1$ . Thus, this hybrid is identical to the previous one.

This concludes the proof. □